Technology
Arts Sciences
TH Köln

Potenziale von CSS-Layouttechnologien im Responsive Webdesign

Welche gestalterischen und technischen Potenziale bieten CSS-Layouttechnologien im Responsive Webdesign und für welche Layoutanforderungen eignen sich welche Technologien am besten?

Praxisprojekt

ausgearbeitet von

Anna Stein
Matrikelnr.: 11237022

vorgelegt an der

Technischen Hochschule Köln
Campus Gummersbach
Fakultät für Informatik und
Ingenieurwissenschaften

im Studiengang
Medieninformatik

Prüfer: Prof. Christian Noss
Technische Hochschule Köln

Gummersbach, im Juli 2026

Die Gestaltung responsiver Webseiten wird durch eine wachsende Anzahl an CSS-Layouttechnologien zunehmend komplex, da in der Praxis oft Unsicherheit darüber besteht, welches Werkzeug für welche Layoutanforderung am besten geeignet ist. Diese Arbeit untersucht die gestalterischen und technischen Potenziale von Flexbox, CSS Grid, CSS Subgrid sowie den responsiven Steuerungsprinzipien Media Queries und Container Queries im Kontext des Responsive Webdesigns. Auf Basis eines eigens entwickelten, sechsdimensionalen Analysemodells werden die Technologien hinsichtlich Layout und Wirkung, Modularität, technischer Effizienz, Wartbarkeit, Nachhaltigkeit und Barrierefreiheit sowie responsiver Anpassungsfähigkeit systematisch untersucht. Anschließend werden mithilfe eines Human-Centered-Design-Ansatzes und standardisierter Spezifikationswerkzeuge aus dem Requirements Engineering sieben konkrete Layoutanforderungen aus typischen Nutzungskontexten hergeleitet. Ein Verknüpfungsmodell ordnet diese Anforderungen den jeweils geeignetsten Technologien zu und bildet die Grundlage für eine prototypische Demonstrationswebseite, die die identifizierten Zusammenhänge interaktiv erfahrbar macht. Die Ergebnisse zeigen, dass sich moderne responsive Webseiten nicht mehr sinnvoll mit einer einzelnen Technologie umsetzen lassen, sondern eine funktionale Arbeitsteilung zwischen makro- und mikroebenenbezogenen Layoutlösungen erfordern.

Inhaltsverzeichnis

1. Einleitung **1**
- 1.1. Problemstellung . . . 1
- 1.2. Zielsetzung . . . 1

2. Historische Entwicklung **3**
- 2.1. Seitenlayouts mit `<table>`-Tags . . . 3
- 2.2. Einführung des Tabellenlosen Webdesigns . . . 4
- 2.3. Entwicklung responsiver Webseiten . . . 4
- 2.4. Entwicklung moderner Layoutsysteme . . . 5

3. Theoretische Grundlagen **6**
- 3.1. Grundlagen von Cascading Style Sheets (CSS) . . . 6
- 3.2. Bedeutung von Layout-Technologien im Frontend-Development . . . 6
- 3.3. Responsive Webdesign . . . 7

4. Methodik **8**
- 4.1. Ziel der Analyse . . . 8
- 4.2. Entwicklung des Analysemodells . . . 8
  - 4.2.1. Gesichtspunkte zur Layouttechnologieanalyse . . . 8
  - 4.2.2. Gesichtspunkte zur Analyse der responsiven Steuerungsprinzipien 10
- 4.3. Methodischer Rahmen zur Herleitung von Layoutanforderungen . . . 12
  - 4.3.1. Nutzungskontext und Erfordernisse nach ISO/IEC 25064 . . . 12
  - 4.3.2. Systemanforderungen nach der Rupp'schen Satzschablone . . . 13
- 4.4. Verknüpfungsmodell zur Zuordnung von Layoutanforderungen und Layouttechnologien . . . 13

5. Analyse der CSS-Technologien **15**
- 5.1. Analyse Flexbox . . . 15
  - 5.1.1. Layout und Wirkung . . . 15
  - 5.1.2. Modularität und Wartbarkeit . . . 16
  - 5.1.3. Technische Effizienz und Performance . . . 16
  - 5.1.4. Wartbarkeit und Developer Experience . . . 17
  - 5.1.5. Nachhaltigkeit und Barrierefreiheit . . . 17
  - 5.1.6. Responsive Anpassungsfähigkeit . . . 18
- 5.2. Analyse CSS-Grid . . . 19
  - 5.2.1. Layout und Wirkung . . . 19
  - 5.2.2. Modularität und Wartbarkeit . . . 19
  - 5.2.3. Technische Effizienz und Performance . . . 19
  - 5.2.4. Wartbarkeit und Developer Experience . . . 20
  - 5.2.5. Nachhaltigkeit und Barrierefreiheit . . . 22

5.2.6. Responsive Anpassungsfähigkeit . . . 22
5.3. Analyse CSS-Subgrid . . . 22
5.3.1. Layout und Wirkung . . . 22
5.3.2. Modularität und Wartbarkeit . . . 23
5.3.3. Technische Effizienz und Performance . . . 23
5.3.4. Wartbarkeit und Developer Experience . . . 24
5.3.5. Nachhaltigkeit und Barrierefreiheit . . . 25
5.3.6. Responsive Anpassungsfähigkeit . . . 25
5.4. Analyse Media-Queries . . . 26
5.4.1. Steuerungslogik und Layout-Wirkung . . . 26
5.4.2. Komponentenarchitektur und Modularität . . . 27
5.4.3. Technische Effizienz und Performance . . . 27
5.4.4. Wartbarkeit und Developer Experience . . . 27
5.4.5. Nachhaltigkeit und Barrierefreiheit . . . 28
5.5. Analyse Container-Queries . . . 28
5.5.1. Steuerungslogik und Layout-Wirkung . . . 28
5.5.2. Komponentenarchitektur und Modularität . . . 28
5.5.3. Technische Effizienz und Performance . . . 29
5.5.4. Wartbarkeit und Developer Experience . . . 29
5.5.5. Nachhaltigkeit und Barrierefreiheit . . . 30

6. Herleitung der Layoutanforderungen im responsive Webdesign 31
6.1. Dimensionierung und proportionale Raumverteilung . . . 31
6.2. Ergonomie und Interaktionsflächen auf Touch-Endgeräten . . . 31
6.3. Größenunterschiede von Medien auf Webseiten . . . 32
6.4. Kontextübergreifende Modularität in Design-Systemen . . . 33
6.5. Varianz und Dynamik datengetriebener Inhalte . . . 33
6.6. Divergenz zwischen visueller Hierarchie und Semantik . . . 34
6.7. Software-Qualität und strukturelle Wartbarkeit . . . 34

7. Auswertung und Einordnung der Ergebnisse 36
7.1. Entscheidungsmodell Ergebnisse . . . 36
7.2. Auswertung . . . 38

8. Prototypische Umsetzung einer Demonstrationswebseite 40
8.1. Zielsetzung der Demonstrationswebseite . . . 40
8.2. Designprozess . . . 40
8.2.1. Definition der Webkomponenten . . . 41
8.2.2. Darstellung der Responsivität und Code-Struktur . . . 46
8.2.3. Figma Umsetzung . . . 48
8.2.4. Programmieren des Figma-Designs . . . 50

9. Fazit 52
9.1. Zusammenfassung der Ergebnisse . . . 52
9.2. Beantwortung der Forschungsfrage . . . 52
9.3. Reflexion der Analyse und Methodik . . . 53

10. Ausblick **55**

Abbildungsverzeichnis **57**

Literaturverzeichnis **63**

Anhang **64**

A. Anhang 6 Herleitung der Layoutanforderungen im responsive Webdesign 65

B. Anhang 7 Auswertung und Einordnung der Ergebnisse 66

B.1. Entscheidungsmodell Ergebnisse 7.1 66

C. Anhang 8 Prototypische Umsetzung einer Demonstrationswebseite . . . 70

C.1. Designprozess 8.2 70

C.2. Figma Umsetzung 8.2.3 74

1. Einleitung

Die Gestaltung responsiver Webseiten stellt Entwickler heute vor vielfältige Herausforderungen. Vor allem im Hinblick auf die wachsende Smartphone-Nutzerzahl, die in Deutschland 2025 bei etwa 71 Millionen lag (Department, 2025), müssen Webseiten heute auf unterschiedlichsten Endgeräten und Bildschirmgrößen nutzbar sein. Gleichzeitig müssen sie den hohen Anforderungen an Benutzerfreundlichkeit, Wartbarkeit und Gestaltung gerecht werden (Bhanarkar u. a., 2023). Cascading Style Sheets (CSS) bilden dabei die Grundlage für die visuelle Gestaltung und Strukturierung von Webseiten (Wilson, 2023). Durch die kontinuierliche Weiterentwicklung von CSS wurden in den vergangenen Jahren zahlreiche neue Layouttechnologien eingeführt, die deutlich umfangreichere Möglichkeiten zur Positionierung von Inhalten bieten als klassische Ansätze, wie `<table>`-Tags im HTML (Hazaël-Massieux, 2005).

1.1. Problemstellung

Zu den bedeutensten CSS-Layouttechnologien zählen insbesondere Flexbox, CSS Grid und Subgrid (Andrew, 2017) sowie die responsiven Steuerungsprinzipien Media Queries und Container Queries (Schoner, 2024). Diese Technologien ermöglichen die Umsetzung komplexer, flexibler und responsiver Layoutstrukturen im Web (Wilson, 2023). Gleichzeitig führt die zunehmende Anzahl verfügbarer Technologien dazu, dass die Auswahl geeigneter Werkzeuge für konkrete responsive Layoutanforderungen immer anspruchsvoller wird. In der Praxis besteht daher häufig Unsicherheit darüber, welche Technologie für bestimmte responsive Gestaltungs- und Layoutanforderungen den größten Nutzen bietet (UnravelJS, 2026).
Die Auseinandersetzung mit diesem Thema besitzt sowohl für die Webentwicklung als auch für das Webdesign eine hohe Relevanz. Die gezielte Auswahl geeigneter CSS-Technologien kann die Entwicklung effizienter gestalten, die Wartbarkeit von Webseiten verbessern und die Umsetzung moderner Benutzeroberflächen erleichtern (Bhanarkar u. a., 2023). Vor diesem Hintergrund besteht die Motivation dieses Praxisprojekts darin, die Potenziale der ausgewählten CSS-Layouttechnologien und der responsiven Steuerungsprinzipien systematisch zu untersuchen und deren Einsatzmöglichkeiten im Responsive Webdesign zu ergründen.

1.2. Zielsetzung

Ziel des Praxisprojekts ist es, die gestalterischen und technischen Potenziale ausgewählter CSS-Layouttechnologien zu analysieren und miteinander zu vergleichen. Auf strategischer Ebene soll eine fundierte Grundlage geschaffen werden, die Entwicklern bei der Auswahl geeigneter Technologien für unterschiedliche Layoutanforderungen un-

terstützt. Operativ erfolgt hierzu eine strukturierte Analyse der Technologien Flexbox, CSS Grid, Subgrid, Media Queries und Container Queries anhand definierter Gesichtspunkte. Außerdem werden diese Ergebnisse mit erarbeiteten Layoutanforderungen im responsive Webdesign verknüpft, um ihre Zusammenhänge darstellen zu können und somit ein Verküpfungsmodell zu erstellen. Dieses soll zeigen, welche Layoutanforderungen mit welcher Technologie gelöst werden können. Die gewonnenen Erkenntnisse werden anschließend in Form einer prototypischen Umsetzung ausgewählter Layout-Features in Form einer Demonstrationswebseite veranschaulicht.
Aus dieser Zielsetzung ergibt sich die folgende Forschungsfrage:

„Welche gestalterischen und technischen Potenziale bieten CSS-Layouttechnologien im Responsive Webdesign und für welche Layoutanforderungen eignen sich welche Technologien am besten?"

Der Mehrwert der Arbeit liegt daher in der Entwicklung einer strukturierten Vergleichs- und Bewertungsgrundlage, die Entwicklern und Designern bei der Auswahl geeigneter CSS-Technologien für responsive Layoutsysteme unterstützen soll.

2. Historische Entwicklung

Um zu verstehen, weshalb die Cascading Style Sheets (CSS) überhaupt eingeführt wurden, lohnt sich ein Blick auf die früheren Gestaltungspraktiken im Webdesign und wie diese sich im Laufe der Jahre weiterentwickelt haben.

2.1. Seitenlayouts mit `<table>`-Tags

Im Werk ,,Creating Killer Web Sites“ von David Siegel wird die Nutzung von HTML-Tabellen zur Layoutgestaltung erwähnt (Siegel, 1996). Nach der Einführung des `<table>`-Tags im HTML in den 1990er Jahren, wurde diese Vorgehensweise zur Seitengestaltung populär und viel genutzt (Hoffmann, 2017). Diese Tabellendarstellung war jedoch im Zusammenhang mit der Softwareentwicklung für die Darstellung tabellarischer Daten entwickelt worden und nicht für die Gestaltung von Seitenlayouts. Trotzdem sahen viele Webdesigner den Vorteil darin, dieses Feature zur Visuallisierung komplexerer Webseitenlayouts zu verwenden, da HTML ursprünglich nicht in der Lage war dies zu tun (Mohamed u. a., 2014).

Bei der Verwendung von `<table>` kam es jedoch schnell dazu, dass viele verschachtelte Tabellen im HTML Dokument entstanden und somit den Code unübersichtlich und komplex machten (Mohamed u. a., 2014). Dies machte unteranderem die Wartbarkeit dieses Codes herausfordernd. Außerdem sorgte seine Verwendung dafür, dass Suchmaschinen und Screenreader den Content der Webseiten nicht verstanden haben, da Syntax und Semantik nicht von einander getrennt werden konnten. Diese Herausforderung hatte vor allem zum Nachteil, dass Menschen mit einer Sehbehinderung keine Möglichkeit hatten sich den Inhalt der Webseite durch einen Screenreader darstellen zu lassen. Somit litt unter anderem die Zugänglichkeit der Webseiten darunter (Think11, 2020). Dieses Problem stellt ebenfalls einen Verstoß gegen das Erfolgskriterium 1.3.1 der Web Content Accessibility Guidelines (WCAG) 2.2 dar. Dieses besagt: ,,Informationen und Bestandteile der Benutzerschnittstelle müssen den Benutzern so präsentiert werden, dass diese sie wahrnehmen können.”(W3C, 2024).

Desweiteren gab es keine einheitliche Methode, um Webseiten in verschiedenen Browsern zu dieser Zeit, identisch anzuzeigen (Hellbusch, 2021). Eine Seite konnte in einem Browser korrekt aussehen, jedoch in einem anderen komplett zerfallen (Hoffmann, 2017).

2.2. Einführung des Tabellenlosen Webdesigns

Die Lösung zu dem oben beschriebenen Problem ist das sogenannte ,,Tabellenlose Webdesign". Das ,,Tabellenlose Webdesign"beschreibt unter Anderem die Verwendung von CSS-Technologien, statt dem `<table>`-Tag, zur Gestaltung von Webseitlayouts (Hazaël-Massieux, 2005). Das Trennen der gestalterischen Aspekten vom HTML machte es möglich, den Code wartbarer zu machen, um Fehler schnell zu beheben (Mohamed u. a., 2014).
Problematisch war anfangs jedoch, dass CSS nicht von allen Browsern untertstützt wurde und Designer sich dieser Sache nicht bewusst waren. Jedoch hatte das CSS-basierte Design den Vorteil, dass wenn einmal die style-sheets einer Webseite im Browser geladen wurden, diese automatisch gechached wurden und somit das schnellere Laden einer Webseiten beim nächsten Besuch ermöglichte (Mohamed u. a., 2014)

2.3. Entwicklung responsiver Webseiten

Der Begriff ,,Responsive Web Design"(RWD) wurde erstmalig im Jahr 2010 vom Webdesigner und Entwickler Ethan Marcotte geprägt. Marcotte orientierte sich dabei an architektonischen Konzepten wie flexiblen Rasterstrukturen, proportionalen Layouts sowie adaptiven Gestaltungssystemen, die sich dynamisch an ihre Umgebung anpassen können. Diese Ansätze übertrug er auf die Webentwicklung mit dem Ziel, Webseiten flexibel an unterschiedliche Endgeräte und Bildschirmgrößen anzupassen (Mohamed u. a., 2014).

Vor der Entwicklung responsiver Webseiten wurde häufig vom sogenannten ,,Mobile Web Design"gesprochen. Dabei wurden oftmals separate Webseitenversionen für mobile Endgeräte entwickelt, wodurch zusätzlicher Entwicklungs- und Wartungsaufwand entstand. Mit der zunehmenden Verbreitung mobiler Endgeräte wie Smartphones und Tablets stiegen jedoch die Anforderungen an flexible und geräteunabhängige Weboberflächen erheblich. Webseiten mussten zunehmend auf unterschiedlichsten Bildschirmgrößen sowie Auflösungen funktionieren und gleichzeitig eine konsistente Benutzererfahrung gewährleisten (MDN Web Docs, 2025b).

Responsive Webdesign beschreibt daher einen Gestaltungs- und Entwicklungsansatz, bei dem sich Layout, Inhalte und Navigation einer Webseite dynamisch an die jeweilige Nutzungsumgebung anpassen. Ziel ist es, Webseiten so zu entwickeln, dass diese sowohl auf kleinen Smartphone-Displays als auch auf großen Desktop-Bildschirmen nutzerfreundlich dargestellt werden können (Mohamed u. a., 2014).

Mit der zunehmenden Bedeutung mobiler Endgeräte etablierte sich außerdem der sogenannte Mobile-First-Ansatz. Dabei wird das grundlegende Layout zunächst für kleine Bildschirmgrößen entwickelt und anschließend schrittweise für größere Ansichten erweitert. Dieser Ansatz verbessert insbesondere die Performance, Wartbarkeit sowie Benutzerfreundlichkeit moderner Webseiten (IONOS Redaktion, 2024).

2.4. Entwicklung moderner Layoutsysteme

Die Entwicklung moderner Layoutsysteme entstand aus der Notwendigkeit, Webseiten flexibler, wartungsfreundlicher und responsiver gestalten zu können. Frühere Webseitenlayouts wurden häufig mithilfe sogenannter Float-Layouts umgesetzt. Dabei wurden Inhalte durch die CSS-Eigenschaft `float` nebeneinander positioniert. Ursprünglich war diese Technik jedoch nicht für vollständige Webseitenlayouts vorgesehen, sondern hauptsächlich für das Umfließen von Texten um Bilder. Komplexe Layoutstrukturen führten dadurch oftmals zu schwer wartbarem und unübersichtlichem CSS-Code (Marcotte, 2010).

Mit der Einführung von Flexbox entstand erstmals ein Layoutsystem, das speziell für die flexible Anordnung von Elementen innerhalb einer Dimension entwickelt wurde. Flexbox erleichtert insbesondere die horizontale und vertikale Ausrichtung von Inhalten sowie die dynamische Verteilung verfügbarer Bereiche innerhalb eines Containers. Dadurch konnten viele Einschränkungen früherer Float-basierter Layouttechniken überwunden werden (MDN Web Docs, 2025a).

Einen weiteren Fortschritt stellte die Einführung von CSS Grid Layout dar. Im Gegensatz zu Flexbox ermöglicht CSS Grid die Gestaltung zweidimensionaler Layoutstrukturen mit Zeilen und Spalten. Dadurch lassen sich komplexe Webseitenlayouts deutlich strukturierter und effizienter umsetzen (MDN Web Docs, 2026a). Moderne CSS-Technologien wie Grid und Flexbox bilden heute die Grundlage vieler responsiver Webanwendungen.

Parallel dazu entwickelte sich die Frontend-Entwicklung zunehmend hin zu komponentenbasierten Layoutsystemen. Benutzeroberflächen werden dabei aus modularen und wiederverwendbaren Komponenten aufgebaut, die unabhängig voneinander funktionieren und gestaltet werden können. Besonders moderne Frameworks und Designsysteme verfolgen diesen Ansatz, um Webseiten skalierbarer, flexibler und wartbarer zu gestalten. Technologien wie Container Queries unterstützen diese Entwicklung zusätzlich, da Komponenten nicht mehr ausschließlich auf die globale Bildschirmgröße reagieren müssen, sondern sich abhängig von ihrer jeweiligen Containergröße anpassen können (Gupta, 2025).

3. Theoretische Grundlagen

Im folgenden werden die theoretischen Grundlagen moderner CSS-Technologien und des responsiven Webdesigns vermittelt. Dabei werden zunächst die grundlegenden Konzepte von Cascading Style Sheets, sowie deren Bedeutung für die Gestaltung moderner Webseiten erläutert. Zudem wird aufgezeigt, weshalb moderne Layout-Technologien eine zentrale Rolle in der heutigen Frontend-Entwicklung aus Sicht des responsiven Designs einnehmen.

3.1. Grundlagen von Cascading Style Sheets (CSS)

Cascading Style Sheets (CSS) ist eine Stylesheet-Sprache, welche dazu genutzt wird das Aussehen und die Formatierung von HTML-Dateien zu definieren und die semantische Struktur von der visuellen Darstellung dieser zu trennen. CSS dient der visuellen Gestaltung und Formatierung von Webseiten sowie der Definition von Layoutstrukturen digitaler Benutzeroberflächen. Das sogenannte Kaskadenprinzip, welches im Namen steckt, beschreibt die Priorisierung von CSS-Regeln. Dabei entscheidet die Spezifität von Selektoren sowie die Reihenfolge der Definitionen darüber, welche Gestaltungsvorgaben angewendet werden (World Wide Web Consortium (W3C), 2022a).

Im Laufe der Zeit wurde CSS kontinuierlich weiterentwickelt und um neue Funktionen erweitert. Während frühere CSS-Versionen hauptsächlich grundlegende Gestaltungsmöglichkeiten bereitstellten, bieten moderne CSS-Technologien inzwischen umfangreiche Werkzeuge zur Umsetzung komplexer und dynamischer Layouts. Beispielsweise können mit aktuellen CSS-Technologien adaptive Layoutsysteme entwickelt werden, bei denen sich Navigationsbereiche, Inhaltselemente und Bilddarstellungen automatisch an unterschiedliche Bildschirmgrößen und Containerstrukturen anpassen. Dadurch lassen sich responsive Webseiten effizienter, flexibler und wartungsfreundlicher umsetzen (MDN Web Docs, 2025b, 2026b).

3.2. Bedeutung von Layout-Technologien im Frontend-Development

Moderne Frontend-Architekturen basieren zunehmend auf komponentenorientierten Entwicklungsansätzen. Benutzeroberflächen werden dabei aus wiederverwendbaren UI-Komponenten aufgebaut, die unabhängig voneinander funktionieren und gestaltet werden müssen. Insbesondere Container Queries ermöglichen es, Komponenten abhängig von ihrer Containergröße statt der globalen Viewportgröße responsiv zu gestalten (Gupta, 2025)

Vor Allem CSS Grid 5.2 und Flexbox 5.1 ermöglichen die Erstellung flexibler und modularer Layoutsysteme, während Media 5.4- und Container-Queries 5.5 eine adaptive Darstellung von Inhalten abhängig von Bildschirmgröße oder Containerdimensionen unterstützen (MDN Web Docs, 2026b, 2025b). Dadurch können Weboberflächen effizienter entwickelt sowie besser skaliert und gewartet werden. CSS-Layoutechnologien bilden somit eine zentrale Grundlage der Frontend-Entwicklung und eröffnen neue gestalterische sowie technische Potenziale im Responsive Webdesign.

3.3. Responsive Webdesign

Responsive Webdesign beschreibt einen Gestaltungsansatz zur Entwicklung von Webseiten, die sich flexibel an unterschiedliche Bildschirmgrößen, Auflösungen und Endgeräten anpassen. Ziel ist es, eine konsistente Benutzererfahrung auf Smartphones, Tablets, Laptops, Desktop-Computern sowie weiteren internetfähigen Geräten zu gewährleisten (MDN Web Docs, 2025b).

Die Relevanz responsiver Webseiten hat in den letzten Jahren deutlich zugenommen. Da ein Großteil des weltweiten Web-Traffics mittlerweile über mobile Endgeräte erfolgt, verfolgt Google seit 2024 einen Mobile-First-Indexierungsansatz, bei dem primär die mobile Version einer Webseite für das Ranking berücksichtigt wird (Think11, 2026). Dadurch gewinnt die mobile Optimierung von Webseiten zunehmend an Bedeutung.

Eine zentrale Grundlage des Responsive Webdesigns stellen sogenannte Fluid Layouts 6.1 dar. Dabei werden Layoutstrukturen nicht mit festen Pixelwerten definiert, sondern mithilfe relativer Einheiten wie Prozentwerten, `rem`, `vw` oder `vh` flexibel gestaltet. Dadurch können sich Inhalte dynamisch an unterschiedliche Bildschirmgrößen anpassen.
Ergänzend dazu werden sogenannte Breakpoints verwendet. Breakpoints definieren bestimmte Bildschirmbreiten oder Containergrößen, ab denen sich das Layout einer Webseite verändert. Diese Breakpoints werden mithilfe von Media-Queries in das CSS eingebaut. Durch Media Queries 5.4 können beispielsweise Navigationsstrukturen angepasst, Inhalte neu angeordnet oder Schriftgrößen verändert werden (MDN Web Docs, 2025b).

Trotz der technischen Fortschritte moderner CSS-Technologien stellt die Entwicklung responsiver Webseiten weiterhin eine Herausforderung dar: Unterschiedliche Bildschirmgrößen, Seitenverhältnisse sowie Eingabemethoden erfordern flexible und skalierbare Layoutsysteme. Insbesondere komplexe Benutzeroberflächen stoßen bei klassischen Responsive-Design-Ansätzen häufig an ihre Grenzen (**?**). Globale viewportbasierte Anpassungen mithilfe von Media Queries reichen oftmals nicht aus, um komponentenbasierte Layouts effizient zu gestalten (Carter, 2025). Dadurch entstehen zusätzliche Anforderungen an die Wiederverwendbarkeit und Modularität von Benutzeroberflächen. Desweiteren müssen moderne Webseiten nicht nur auf verschiedenen Endgeräten funktionieren, sondern gleichzeitig Aspekte wie Performance, Barrierefreiheit, Wartbarkeit sowie Benutzerfreundlichkeit berücksichtigen (**?**).

4. Methodik

Um im Anschluss die CSS-Layouttechnologien hinsichtlich ihrer gestalterischen und technischen Potenziale analysieren zu können, wird vorerst eine methodische Vorgehensweise für die Analyse dieser Technologien erarbeitet. Dazu werden Gesichtspunkte aufgestellt, die für die Analyse der Layouttechnologien und der responsiven Steuerungsprinzipien, Media- und Container-Queries, angewandt werden. Daraufhin werden Layoutanforderungen im Responsive Webdesign hergeleitet, um abschließend die CSS-Layouttechnologien mit den Layoutanforderungen verknüpfen zu können.

4.1. Ziel der Analyse

Mit der Analyse der ausgewählten CSS-Layouttechnologien Flexbox, Grid und Subgrid und den responsiven Steuerungsprinzipien Media- und Container-Queuries, sollen gestalterische, sowie technische Potenziale dieser hinsichtlich Responsivität ausgearbeitet werden. Durch die identifizierten Potenziale soll es auf Grundlage von identischen Gesichtspunkten möglich gemacht werden, die CSS-Layouttechnologien vergleichen zu können. Für die responsiven Steuerungsprinzipien werden andere bzw. abgewandelte Gesichtspunkte aufgestellt, da Queries selbst keine Layouts erzeugen, sondern diese nur steuern. Auf Grundlage der Analyse und den hergeleiteten Layoutanforderungen wird anschließend eine Verknüpfungsmatrix entwickelt, welche als Entscheidungsgrundlage für Entwickler und Designer dienen soll. Mit dieser Verknüpfung soll die Forschungsfrage beantwortet werden, mit welchen CSS-Layouttechnologien, welche Layoutanforderungen aus dem Responsive Webdesign gelöst werden können.

4.2. Entwicklung des Analysemodells

Die Analyse besteht aus drei Schritten: Zum einen aus der Analyse der CSS-Layouttechnologien selbst und der Analyse der responsiven Steuerungsprinzipien. Anschließend folgt der Abgleich der Eigenschaften der Layouttechnologien mit den voher hergeleiteten Layoutanforderungen im Responsive Webdesign, woraus abschließend das Entscheidungsmodell für Entwickler und Designer abgeleitet werden soll.

4.2.1. Gesichtspunkte zur Layouttechnologieanalyse

Das Analysemodell basiert auf einer Reihe von Untersuchungskriterien, die sowohl technische als auch gestalterische Aspekte aktueller CSS-Layouttechnologien berücksichtigen. Für jede Layouttechnologie werden dieselben sechs Bewertungsdimensionen analysiert, um eine einheitliche Betrachtung und spätere Vergleichbarkeit der Ergebnisse sicherzustellen.

Die Analyse erfolgt anhand der folgenden Dimensionen bzw. Gesichtspunkten:

- **Layout und Wirkung:**
  In dieser Dimension wird das gestalterische Potenzial analysiert. Hier werden die gestalterischen Möglichkeiten und die Auswirkungen auf die visuelle Struktur eines Layouts untersucht. Man stellt sich die Frage, welche Art von Layoutstrukturen die Layoutechnologie unterstützt und welche Gestaltungsmöglichkeiten diese für die Anordnung und Hierarchisierung von Inhalten mit sich bringt. Außerdem soll herausgearbeitet werden, welche Layoutmuster und Gestaltungsszenarien sich für die Technologie besonders gut eignen.

- **Modularität und Wiederverwendbarkeit:**
  Diese Dimension verfolgt das Ziel der Untersuchung der Eignung für komponentenbasierte und skalierbare Webentwicklung. Es wird analysiert wie gut die Technologie die Entwicklung unabhängiger und wiederverwendbarer Komponenten unterstützt und inwieweit sich diese in Design-Systeme integrieren lässt. Des weiteren stellt man sich der Frage, wie flexibel die Technologie in unterschiedlichen Layoutkontexten eingesetzt werden kann.

- **Technische Effizienz:**
  In diesem Kontext werden die technischen Auswirkungen auf Struktur, Ressourcenverbrauch und Implementierungsaufwand betrachtet. Hier sind Themen, wie der Einfluss der Technologie auf die Komplexität von HTML- und CSS-Strukturen zu beachten. Auch werden die Auswirkungen der Technologie auf Rendering- und Layoutberechnungen im Browser ergründet. Dabei schaut man ebenfalls auf die mögliche Reduktion des Einsatzes zusätzlicher Technologien, wie JavaSript, oder die Möglichkeit der Vereinfachung bestehender Lösungen.

- **Wartbarkeit und Developer Experience:**
  Ebenso zu beachten ist die Untersuchung der Entwicklungs- und Wartungsfreundlichkeit der zu analysierenden CSS-Layouttechnologie. Fragen, wie das Verständnis und die Nachvollziehbarkeit der Arbeitsweise der Technologie für Entwickler stehen hierbei im Vordergrund. Außerdem untersucht man die Technologie hinsichtlich ihrerer Erweiterbarkeit, Anpassbarkeit und langfristiger Pflege. Ein wichtiger Aspekt ist im Folgenden auch die vorhandene Unterstützung von Browser-Tools und Entwicklungswerkzeugen bei der Implementierung und Fehlersuche.

- **Nachhaltigkeit und Barrierefreiheit:**
  Dieser Gesichtspunkt untersucht die langfristige Einsetzbarkeit und Zugänglichkeit der Technologie. Fragen wie: ,,Welche Auswirkungen hat die Technologie auf semantische HTML-Strukturen und die Zugänglichkeit von Webseiten?", sollen hier beantwortet werden. Auch die Zukunftssicherheit und Browserunterstützung wird in dieser Dimension berücksichtigt. Welche potenziellen Herausforderungen oder Einschränkungen sich hinsichtlich Barrierefreiheit und langfristiger Wartung ergeben, sind ebenfalls Bestandteile dieses Gesichtspunktes.

- **Responsive Anpassungsfähigkeit:**
  Zum Abschluss soll für jede Layouttechnologie das Potenzial im RWD analysiert werden. Dies stellt eine fundierte Grundlage für die Erarbeitung der Forschungsfrage dar. Man untersucht hier die Möglichkeiten, die die Technologie für Anpassungen an verschiedene Bildschirmgrößen und Geräte aufweist. Außerdem wird untersucht auf welcher Ebene die Reaktion der Größennveränderung stattfindet. Dazu zählen Veränderungen auf Viewport-, Container- oder Inhalts-Ebene.

Jede Technologie wird anhand der in Abbildung 4.1 gezeigten Vorlage analysiert.

<Technologie>	
Gesichtspunkt	**Eigenschaft**
Layout und Wirkung	
Modularität und Wiederverwendbarkeit	
Technische Effizienz und Performance	
Wartbarkeit und Developer Experience	
Nachhaltigkeit und Barrierefreiheit	
Responsive Anpassungsfähigkeit	

Abbildung 4.1.: Vorlage zur Analyse der CSS-Layouttechnologien

4.2.2. Gesichtspunkte zur Analyse der responsiven Steuerungsprinzipien

Da Media- und Container-Queries im Gegensatz zu Flexbox oder Grid keine eigenständigen Layoutstrukturen generieren, sondern als bedingte Kontrollmechanismen bestehende Stile manipulieren (Andrew, 2017), erfordert deren Untersuchung eine modifizierte Kriterienmatrix. Das Ziel dieser spezifischen Analysephase besteht darin, die Funktionsweise, den Wirkungsgrad und die architektonische Ausrichtung der responsiven Steuerungsprinzipien zu ergründen. Die Untersuchung erfolgt anhand von fünf angepassten Bewertungsdimensionen, um eine präzise theoretische Grundlage für das spätere Verknüpfungsmodell zu schaffen.

Die Analyse der Steuerungsprinzipien gliedert sich in die folgenden Dimensionen:

- **Steuerungslogik und Layout-Wirkung:**
  In dieser Dimension wird das grundlegende operative Prinzip und der jeweilige

Anpassungsmaßstab der Query-Technologie untersucht. Hierbei steht die Frage im Vordergrund, auf Basis welcher Referenzwerte die Layout-Bedingungen schalten. Zudem wird analysiert, welche spezifischen Möglichkeiten das Prinzip zur dynamischen Restrukturierung und Hierarchisierung von Inhalten bietet und für welche architektonischen Makro- oder Mikroszenarien es sich primär eignet.

- **Komponentenarchitektur und Modularität:**
  Dieses Untersuchungskriterium evaluiert die Eignung der Steuerungsprinzipien für moderne, komponentenbasierte Architekturen und Design-Systeme. Es wird analysiert, inwieweit das jeweilige Prinzip die Entwicklung plattformunabhängiger UI-Komponenten begünstigt. Darüber hinaus wird ergründet, wie effektiv die Technologie gekapselte Module vor visuellen Defekten schützt, wenn diese flexibel in stark variierende Layoutkontexte, wie beispielsweise schmale Sidebars oder breite Inhaltsbereiche, eingebettet werden.

- **Technische Effizienz und Performance:**
  In diesem Kontext stehen die softwarearchitektonischen Auswirkungen auf das Stylesheet sowie die Browser-Performance im Fokus. Die Analyse hinterfragt den Einfluss des Steuerungsprinzips auf die Komplexität, Kapselung und potenzielle Redundanz des resultierenden CSS-Codes. Ebenso werden die Auswirkungen der Viewport- und Container-Berechnungen auf die Rendering-Performance und Resize-Events der Browser-Engine untersucht und geprüft, inwiefern JavaScript-basierte Überwachungsmethoden obsolet werden.

- **Wartbarkeit und Developer Experience:**
  Dieser Gesichtspunkt widmet sich der kognitiven Verständlichkeit und der langfristigen Wartbarkeit aus Entwicklersicht. Im Mittelpunkt steht die Frage, wie transparent und nachvollziehbar die Kausalität der Layout-Bedingungen im Code-Kontext bleibt. Des Weiteren wird evaluiert, wie skalierbar und fehlerresistent sich responsive Regelwerke bei System-Updates global erweitern lassen und welche native Unterstützung moderne Browser-Entwicklungswerkzeuge bei der Fehlersuche bieten.

- **Nachhaltigkeit und Barrierefreiheit:**
  Die finale Dimension betrachtet die technologische Nachhaltigkeit sowie die Barrierefreiheit. Hierbei werden spezifische Risiken für die Zugänglichkeit untersucht, die durch reines CSS-gesteuertes, visuelles Umordnen von Inhalten entstehen können. Ergänzend wird der aktuelle Reifegrad des flächendeckenden Browser-Supports analysiert und beleuchtet, wie sensitiv das Prinzip auf benutzerspezifische Barrierefreiheits-Einstellungen reagiert.

Die systematische Erfassung der Steuerungsprinzipien erfolgt analog zu den Layouttechnologien anhand der in Abbildung 4.2 dargestellten Struktur.

<responsives Steuerungsprinzip>	
Gesichtspunkt	**Eigenschaft**
Steuerungslogik und Layout-Wirkung	
Komponentenarchitektur und Modularität	
Technische Effizienz und Performance	
Wartbarkeit und Developer Experience	
Nachhaltigkeit und Barrierefreiheit	

Abbildung 4.2.: Vorlage zur Analyse der responsiven Steuerungsprinzipien

4.3. Methodischer Rahmen zur Herleitung von Layoutanforderungen

Um die Anforderungen an moderne Layouttechnologien systematisch und nachvollziehbar zu bestimmen, basiert das methodische Vorgehen dieser Arbeit auf drei aufeinander aufbauenden Schritten. Diese basieren auf dem Prozess des Human-Centered Designs (HCD) sowie standardisierten Spezifikationswerkzeugen aus dem Requirements Engineering (Rupp u. die SOPHISTen, 2020). Dieser Prozess überführt Nutzungskontexte aus dem Problemraum in präzise Definitionen für den Lösungsraum (ISO, 2013).

4.3.1. Nutzungskontext und Erfordernisse nach ISO/IEC 25064

Der Ausgangspunkt jeder Anforderung liegt im spezifischen Nutzungskontext. Dieser beschreibt die tatsächlichen Bedingungen, unter denen Endanwender oder Entwickler mit einer Software interagieren (ISO, 2013). Dazu zählen zum Beispiel Geräteeigenschaften, Umgebungsfaktoren oder Wartungsszenarien.
Aus diesen Kontexten werden im ersten Schritt die Erfordernisse extrahiert. Gemäß der Norm ISO/IEC 25064 beschreibt ein Erfordernis die notwendige Voraussetzung, die einem Benutzer in einer bestimmten Situation zur Verfügung stehen muss, um ein angestrebtes Ziel zu erreichen (ISO, 2013). Um eine qualitative und einheitliche Formulierung zu garantieren, werden alle Erfordernisse in dieser Arbeit über die Erfordernis-Satzschablone spezifiziert, welche in Abbildung 4.3 zu sehen ist. Die Schablone erzwingt eine strikte Zuweisung von Benutzerrolle, der benötigten Information oder Funktionalität sowie dem damit verbundenen Einsatzzweck.

Als <spezifischer Benutzer> muss man <X> <wissen/verfügbar haben>, um <Y> <entscheiden/tun> zu können.

Abbildung 4.3.: Syntaxstruktur der Erfordernis-Satzschablone (User Experience Qualifications Board (UXQB), 2023)

4.3.2. Systemanforderungen nach der Rupp'schen Satzschablone

Erfordernisse sind per Definition technologieneutral und beschreiben rein die Nutzerperspektive (ISO, 2013). Um diese in konkrete funktionale Vorgaben für die Layoutanforderung im responsive Webdesign zu übersetzen, müssen sie in Anforderungen transformiert werden. Anforderungen legen verbindlich fest, welche Eigenschaften oder Fähigkeiten ein System besitzen muss (Rupp u. die SOPHISTen, 2020).
Für eine syntaktisch fehlerfreie und unmissverständliche Definition wird die Satzschablone nach Rupp et al. verwendet (Rupp u. die SOPHISTen, 2020). Diese standardisierte Struktur aus dem Requirements Engineering zwingt dazu, die Verbindlichkeit sowie die funktionale Kernleistung des Systems präzise zu benennen. Die syntaktische Struktur dieser Schablone ist in Abbildung 4.4 dargestellt.

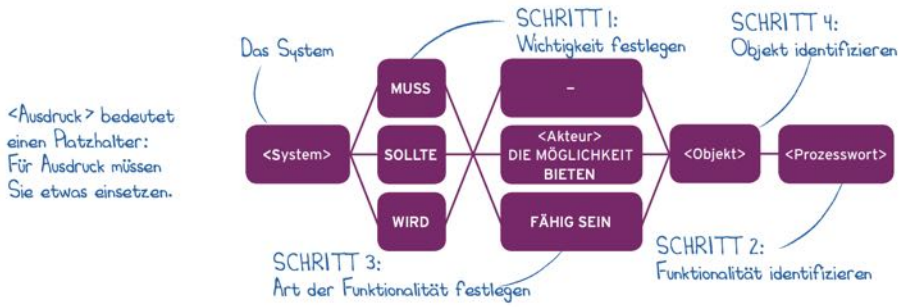


Abbildung 4.4.: Schablone für funktionale Anforderungen ohne Bedingung (Rupp u. die SOPHISTen, 2020)

4.4. Verknüpfungsmodell zur Zuordnung von Layoutanforderungen und Layouttechnologien

Anhand der Analyseergebnisse der CSS-Layoutechnologien, sowie der Herleitung der Layoutanforderungen im Responsive Webdesign, soll als ein Ergebnis der Arbeit eine Entscheidungsmatrix aufgestellt werden, die die CSS-Layoutechnologien mit den Layoutanforderungen verknüpft und darstellt, in wie weit sich die Technologie für die Anforderung eignet. Die Vorlage ist in Abbildung 4.5 zu sehen. Dabei wird für jede Layoutanforderung eine eigene Tabelle erstellt und für jede CSS-Layouttechnologie eine Bewertung von Null bis Zwei Punkten zur Eignung dieser Technologie für die Umsetzung der Layoutanforderung abgegeben. Diese Bewertung wird begründet dargestellt,

sodass am Ende der Tabelle eine Auswahl getroffen werden kann, welche Technologie nun für die Layoutanforderung empfohlen wird.

Entscheidungsmatrix: <Layoutanforderung>			
Layoutanforderung	**CSS Layout-Technologie**	**Eignung**	**Technologische Begründung (Warum passt es?)**
<Name Layoutanforderung>	Flexbox		
	Grid		
	Subgrid		
	Media Queries		
	Container Queries		
	AUSWAHL: CSS-Grid + Media-Queries		**<Begründung Auswahl>**
Legende für Eignung			
++ -> Optimale Lösung. Die Technologie wurde nativ für dieses Problem entwickelt. + -> Bedingt geeignet. Lösung ist möglich, erfordert aber Workarounds oder bietet nicht die volle Flexibilität. o -> Nicht geeignet. Die Technologie löst das Problem strukturell nicht.			

Abbildung 4.5.: Vorlage Entscheidungsmatrix

5. Analyse der CSS-Technologien

Aufbauend auf der in Kapitel 4 erarbeiteten Methodik erfolgt im nachfolgenden Kapitel die systematische Analyse der ausgewählten CSS-Layouttechnologien sowie der responsiven Steuerungsprinzipien.
Das Ziel dieses Kapitels ist es, die inhärenten Eigenschaften, Leistungsgrenzen und softwarearchitektonischen Auswirkungen aller fünf Kerntechnologien detailliert offenzulegen. Die daraus gewonnenen analytischen Erkenntnisse dienen als direkte theoretische und empirische Grundlage für das nachfolgende Verknüpfungsmodell, welches die technologischen Eigenschaften mit den praxisnahen Layoutanforderungen des Responsive Webdesigns zusammenführt.

5.1. Analyse Flexbox

5.1.1. Layout und Wirkung

Hinsichtlich der unterstützten Layoutstrukturen charakterisiert das World Wide Web Consortium (W3C) (2025a) Flexbox als ein primär eindimensionales Layoutmodell. Die Anordnung von Elementen erfolgt gerichtet entlang einer Haupt- und Querachse (siehe 5.1), was sich für lineare Inhaltsstrukturen und komponentenorientierte Layouts eignet (MDN Web Docs, 2025a). Um komplexe, zweidimensionale Rasterstrukturen zu realisieren, mussten Entwickler in der Praxis auf die Verschachtelung mehrerer Flex-Container zurückgreifen, was sich jedoch als kompliziert und fehleranfällig erweist (Andrew, 2017). Die technologischen Gestaltungsmöglichkeiten bieten eine flexible Ausrichtung, Verteilung und proportionale Größenanpassung von Elementen im Raum. Hierdurch lassen sich Inhalte visuell priorisieren und klare Hierarchien durch unterschiedliche Größenverhältnisse oder gezielte Positionierungen etablieren (Haim, 2026). In komponentenbasierten Benutzeroberflächen eignet sich diese Technologie daher besonders gut für elastische Layoutmuster wie Navigationsleisten, Formulare, Kartenlayouts oder Toolbars, bei denen die Raumaufteilung ohne striktes Vorwissen über die exakte Elementenanzahl erfolgen muss (Coyier, 2026).

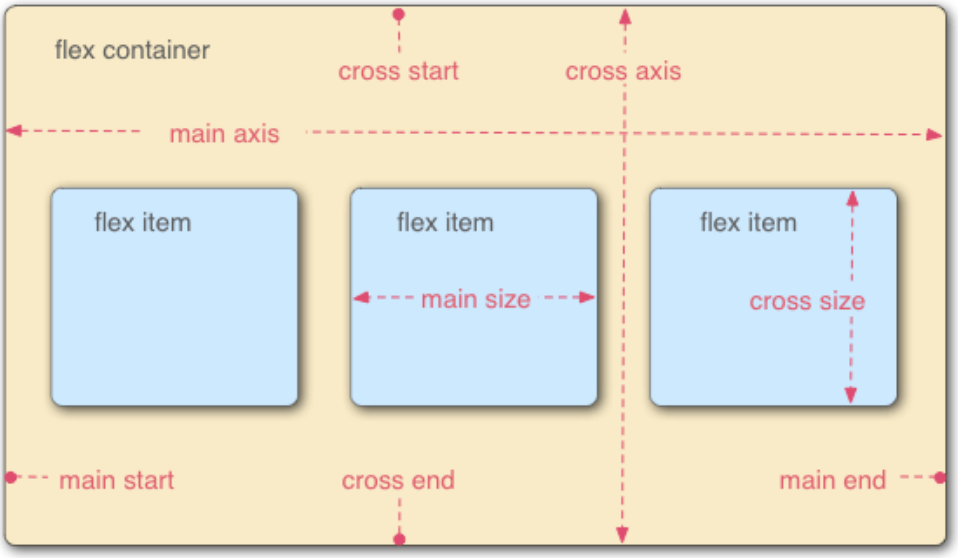


Abbildung 5.1.: Abbildung des Flex-Models aus der MDN-Dokumentation (MDN Web Docs, 2025a)

5.1.2. Modularität und Wartbarkeit

Flexbox bietet eine Unterstützung für die Entwicklung unabhängiger und wiederverwendbarer Komponenten, da das System einem primär `"content-driven"`-Ansatz folgt. Einzelne Komponenten können nativ unabhängig von der Gesamtseitenstruktur gestaltet und mehrfach wiederverwendet werden, da sie Abstände und notwendige Umbrüche elastisch auf Basis ihres konkreten Inhalts und vollständig ohne feste Pixelwerte berechnen (Coyier, 2026). Aus diesem Grund bildet Flexbox einen zentralen Bestandteil moderner Design-Systeme. Die offizielle Spezifikation der World Wide Web Consortium (W3C) (2025b) stellt direkt zu Beginn klar, dass Flexbox für die Koexistenz in komplexen Webanwendungen entwickelt wurde und explizit dafür vorgesehen ist, elastische Layouts in kleinerem und komponentenorientiertem Maßstab innerhalb übergeordneter Layouts bereitzustellen.

5.1.3. Technische Effizienz und Performance

Die softwaretechnische Effizienz von Flexbox äußert sich primär in einer Reduktion zusätzlicher, rein visueller Hilfselemente im HTML-Markup. Viele komplexe Layoutanforderungen lassen sich direkt umsetzen, wodurch das Markup sauber bleibt. Lediglich bei hochgradig verschachtelten Strukturen kann eine stärkere Kaskadierung von Containern erforderlich werden, was die DOM-Komplexität steigert (Andrew, 2017). Hinsichtlich der Rendering- und Layoutberechnungen im Browser verarbeiten moderne Browser-Engines Flexbox-Eigenschaften hocheffizient. Dynamische Layoutänderungen bei Interaktionen oder Viewport-Modifikationen werden performant verarbeitet, sodass im produktiven Betrieb nur geringe Performance-Einbußen messbar sind (Mozilla Developer Network (MDN), 2026b). Als technologischer Substitut vereinfacht

Flexbox bestehende Web-Architekturen maßgeblich. Zahlreiche Layoutaufgaben, die ursprünglich über rechenintensive JavaScript-Berechnungen oder aufwendige, fragile CSS-Konstruktionen realisiert werden mussten, können nativ über die Layout-Engine gelöst werden (Hayward, 2020).

5.1.4. Wartbarkeit und Developer Experience

Das zugrundeliegende Konzept von Flexbox zeichnet sich durch eine hohe Verständlichkeit und Nachvollziehbarkeit aus. Die lineare Anordnung von Elementen entlang einer klar definierten Achse bietet ein intuitives mentales Modell, das von Entwicklern schnell erlernt und adaptiert werden kann (Coyier, 2026). Layouts lassen sich folglich mit geringem Aufwand erweitern, anpassen und langfristig pflegen. Werden einer Komponente neue Elemente hinzugefügt, integrieren sich diese aufgrund der elastischen Verteilungslogik meist vollautomatisch in die bestehende Layoutstruktur, was die Wartbarkeit im Projektverlauf stärkt (Andrew, 2017). Die Developer Experience wird zudem durch umfangreiche Entwicklungswerkzeuge in modernen Browsern begünstigt. Integrierte DevTools visualisieren Flex-Container sowie deren spezifische Parameter interaktiv, was die Implementierung und gezielte Fehlersuche bei komplexen Verhaltungsweisen erheblich erleichtert (siehe Abbildung 5.2).

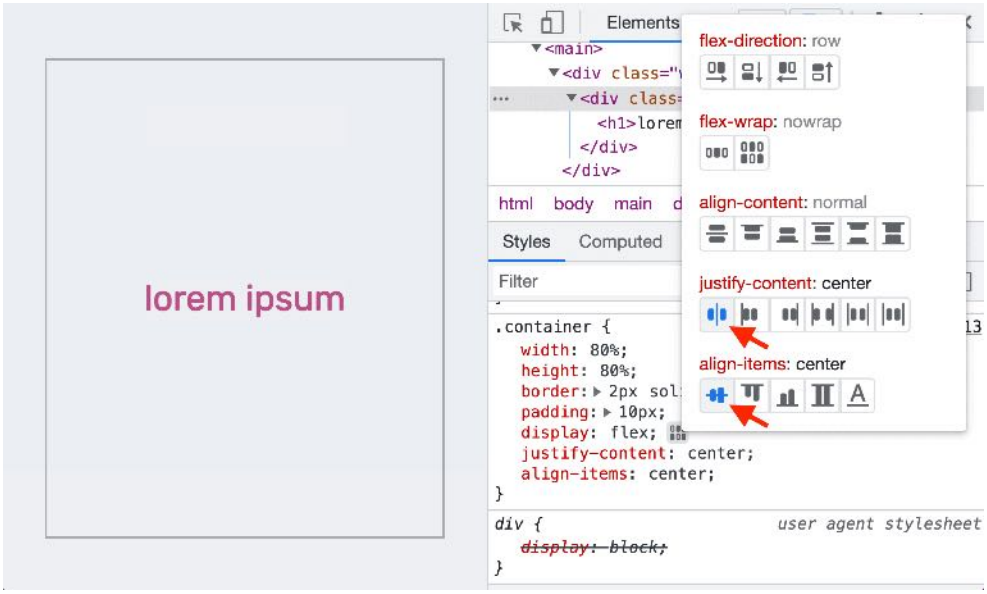


Abbildung 5.2.: DevTool-Option von Chrome zur Einstellung von Flexbox-Eigenschaften (Chrome for Developers, 2024)

5.1.5. Nachhaltigkeit und Barrierefreiheit

Unter Nachhaltigkeitsgesichtspunkten unterstützt Flexbox die strikte Trennung von visuellem Layout und inhaltlicher Struktur, was den Erhalt semantischer HTML-Strukturen fördert. Die technologische Zukunftssicherheit ist uneingeschränkt gegeben, da Flexbox von allen modernen Browsern vollständig und fehlerfrei unterstützt wird

(siehe Abbildung 5.3). Als etablierter Standard und offizielle W3C Empfehlung bildet die Technologie einen unverzichtbaren Kernbestandteil des modernen CSS-Standards, auf dem ein Großteil des aktuellen Webs aufbaut, weshalb eine technologische Veraltung ausgeschlossen ist (World Wide Web Consortium (W3C), 2025a).



Abbildung 5.3.: Diagramm zur Veranschaulichung der Browserunterstützung von Flexbox (Alexis Deveria, 2026a)

Hinsichtlich der Barrierefreiheit birgt Flexbox jedoch identische Risiken wie das CSS-Grid (5.2.5). Potenzielle Einschränkungen für unterstützende Technologien entstehen, wenn die visuelle Reihenfolge von Elementen über die Eigenschaft `order` massiv von der tatsächlichen Hierarchie im HTML-Quelltext entkoppelt wird. Da Screenreader und die sequentielle Tastaturnavigation ausschließlich der logischen DOM-Struktur folgen, führt eine unbedachte visuelle Umordnung zu gravierenden Navigationsbarrieren für beeinträchtigte Nutzer (World Wide Web Consortium (W3C), 2015).

5.1.6. Responsive Anpassungsfähigkeit

Innerhalb des Responsive Designs unterstützt Flexbox die adaptive Oberflächengestaltung, indem es die Größe und Verteilung von Elementen automatisch an den jeweils verfügbaren Platz anpasst. Hierdurch entstehen hochgradig flexible Layouts mit vergleichsweise geringem Implementierungsaufwand. Die Reaktion auf Größenänderungen erfolgt dabei auf der Ebene des Containers. Die Anpassung der Kindelemente wird indirekt über den zur Verfügung stehenden Raum des umschließenden Flex-Containers gesteuert und verfügt über keine eigenen, integrierten Breakpoint-Mechanismen (Andrew, 2017). Das Prinzip eignet sich folglich für responsive Komponenten und kleinere Layoutbereiche auf der Mikro-Ebene (Frost, 2016). Um eine skalierbare und vollumfängliche Responsive-Design-Lösung zu realisieren, wird Flexbox in modernen Architekturen zusammenwirkend mit übergeordneten Steuerungsprinzipien wie Media Queries (5.4) oder Container Queries (5.5) kombiniert.

5.2. Analyse CSS-Grid

5.2.1. Layout und Wirkung

Das CSS-Grid wurde für die Entwicklung von zweidimensionalen Layoutsystemen geschaffen, um die Ausrichtung von Elementen in Zeilen und Spalten zu ermöglichen. Daher ist diese Layouttechnologie besonders für das Gestalten von Gesamtseitenlayouts geeignet (Wilson, 2023). Das World Wide Web Consortium (W3C) (2015) spezifiziert Grid explizit als zweidiemsionales System, um die Beschränkungen linearer Inhalte zu überwinden. Durch die Zweidiemnsionalität ist die gleichzeitige Kontrolle über Spalten (`columns`) und Zeilen (`rows`) möglich, was die visuelle Hierarchisierung in horizontaler und vertikaler Ebene garantiert (World Wide Web Consortium (W3C), 2015). Grid-Expertin Rachel Andrew zeigt auf, dass diese Technologie komplexe Prototypen ohne mathetatische Prozentrechnungen visuell greifbar macht. Mit diesen Eignschaften kann man mit Grid also vor allem Makro-Layouts gestalten, aber auch komplexe, gitterbasierte Anordnungen, aus denen asymetrische und überlappende Layouts enstehen können (Andrew, 2017).

5.2.2. Modularität und Wartbarkeit

CSS-Grid ermöglicht es, das Layout innerhalb einer einzigen Komponente im Mikro-Layout völlig unahängig von der übergeordneten Seitenstruktur zu definieren (Andrew, 2017). Sein Layout-Driven-Ansatz, anders als bei Flexbox, sorgt dafür, dass das Raster die Platzierung von Elementen erzwingt, weshalb wiederverwendete Komponenten in verschiedenen Kontexten ihre strukturelle Integrität behalten, ohne von unvorhersehbaren Textlängen verzerrt zu werden (House, 2021). Seit der flächendeckenden Einführung von CSS-Subgrid, welches im Unterkapitel 5.3 näher betrachtet wird, können verschachtelte Kindelemente das Raster der Elternkomponente erben. Dies erlaubt die Erstellung modularer Teaser-oder Card-Komponenten, deren innere Elemente, wie zum Beispiel Header oder Footer auch bei Wiederverwendung auf einer Linie fluchten (Mozilla Developer Network (MDN), 2025b). Diese Layouttechnologie stellt einen etablierten Grundpfeiler moderner Layout-Komponenten dar, da sie das veraltete, starre 12-Spalten-Modell in Design-Systemen abgelöst hat. Dadurch können Entwickler heute standartisierte `Grid-` und `Layout`-Basiskomponenten bereitstellen, um konsistente UI-Muster zu erzwingen (Frost, 2022). Die Technologie ist hochgradig flexibel, da sie sowohl raumgreifende Gesamtraster auf der Makro-Ebene, als auch für die differenzierte Strukturierung innerhalb kleinster UI-Module eingesetzt werden kann (Andrew, 2017).

5.2.3. Technische Effizienz und Performance

Da CSS Grid ein echtes zweidimensionales System ist, entfällt die Notwendigkeit, für Spalten und Zeilen kaskadierende Hilfscontainer im HTML anzulegen. Das Markup bleibt dadurch flach, sauber und semantisch (Andrew, 2017). Des weiteren muss bei einer Änderung der Bildschirmgröße der Browser dank der Grid-Logik das Layout nicht mehrfach kaskadierend berechnen, da alle Elemente fest in ihrem zugewiesenen Raster verankert sind (Mozilla Developer Network (MDN), 2026b). Auch vermeidet

CSS-Grid Layout-Sprünge: Durch die explizite Definition von Rastergrößen, zum Beispiel über `grid-template-rows`, reserviert der Browser bereits während des Ladens den exakten Platz für Medien und Inhalte, was das risikoreiche nachträgliche Verschieben von Layout-Blöcken verhindert (House, 2021). Komplexe Design-Muster, wie gleich hohe Karten in einer Reihe, symmetrische Versatz-Layouts oder dynamische Ausrichtungen erfordern mit Grid keinerlei JavaScript-Bibliotheken mehr (Andrew, 2017). Außerdem können durch flexible Grid-Automatismen wie `repeat(auto-fit, minmax(200px,1fr))` der Umbruch von Elementen rein mathematisch anhand des verfügbaren Platzes berechnet werden. Viele starre Viewport-Breakpoints, die im Unterkapitel 5.4 näher erläutert werden, werden dadurch obsolet (House, 2021).

5.2.4. Wartbarkeit und Developer Experience

Über die Eigenschaft `grid-template-areas` lässt sich die zweidimensionale Layoutstruktur namentlich im CSS-Code „zeichnen“ (z. B. "header headersidebar main"). Dieses anschauliche mentale Modell orientiert sich an klassischen Print- und Magazin-Layouts, wodurch die Absicht des Entwicklers sofort verständlich wird und sich die kognitive Komplexität bei der Wartung deutlich verringert (Andrew, 2017). Da die Layout-Steuerung zentral über das Eltern-Element erfolgt und verschachtelte Hilfs-Layoutboxen im HTML entfallen, lassen sich auch responsive Umstrukturierungen, wie zum Beispiel durch Media Queries, effizient an einer einzigen Stelle im Code durchführen (Andrew, 2017). Dynamisch hinzugefügte Inhalte lässt das System über Regeln wie `grid-auto-rows` automatisch mitwachsen (World Wide Web Consortium (W3C), 2015). Browser-DevTools unterstützen die Developer Experience zusätzlich, indem sie Rasterlinien und benannte Bereiche visuell einblenden (siehe Abbildung 5.4) (Chrome for developers, 2021).
Außerdem bieten moderne Browser, wie Chrome visuelle Layout-Editoren mit denen sich CSS-Grid-Eigenschaften wie `align-items` oder `justify-content` per Mausklick testen lassen. Dieses sieht man in der Abbildung 5.5.

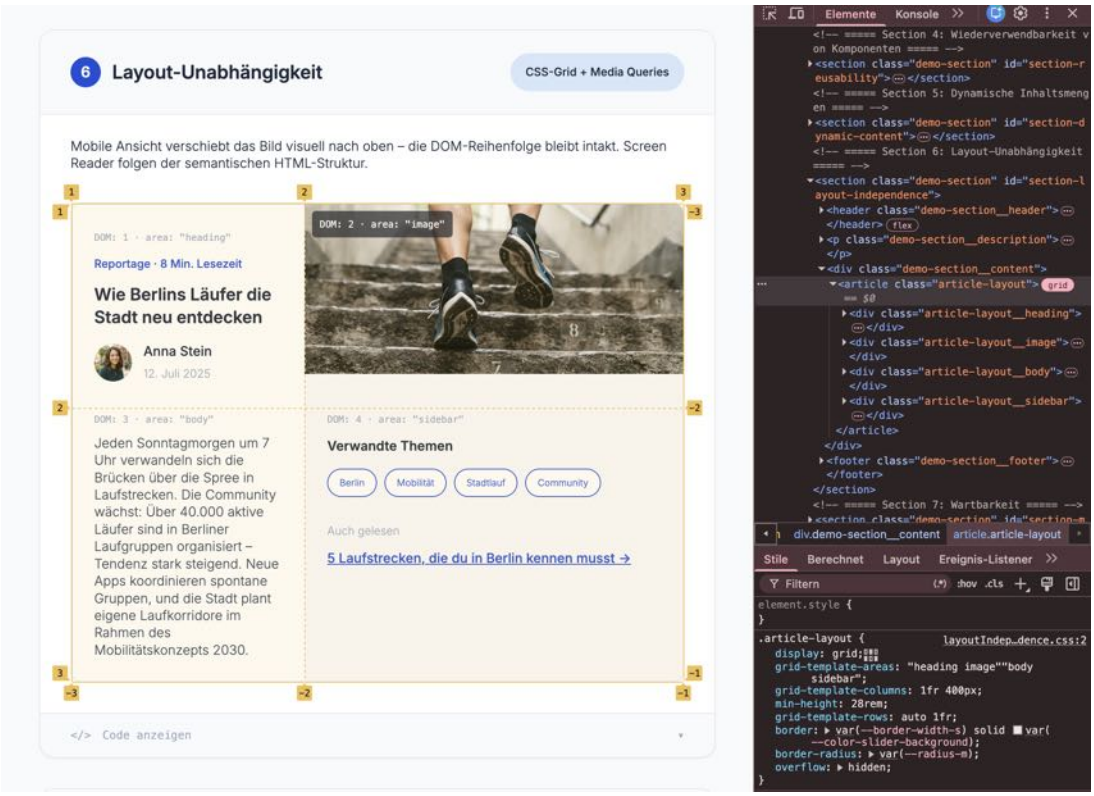


Abbildung 5.4.: DevTool-Option von Chrome zur Anzeige der verwendeten Grids am Beispiel von der Webkomponente der eigenen Ausarbeitung 8

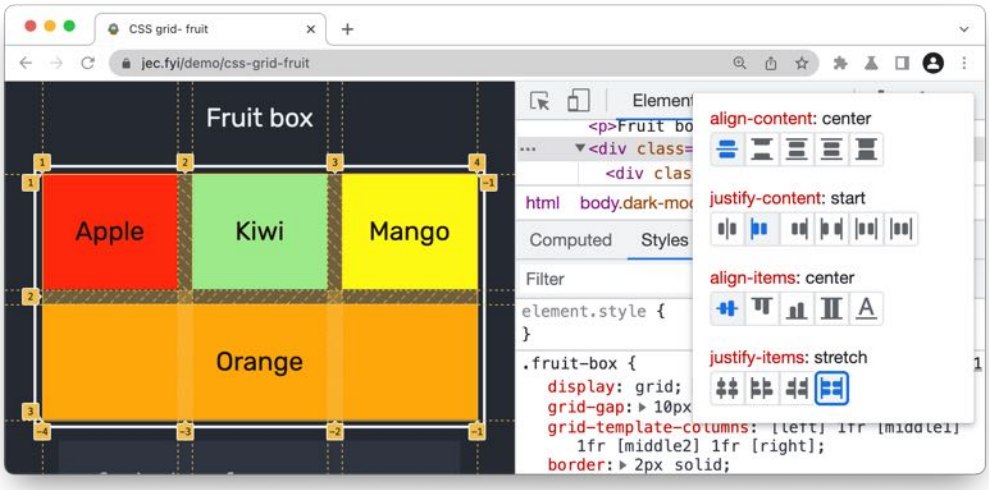


Abbildung 5.5.: DevTool-Layouteditor von Chrome zur Echtzeitänderung von CSS-Grid-Eigenschaften (Chrome for developers, 2021)

5.2.5. Nachhaltigkeit und Barrierefreiheit

Das gestalterische Potenzial der Quelltext-Unabhängigkeit moderner Layouttechnologien birgt jedoch ein erhebliches technisches und barrierefreies Risiko. Wie das World Wide Web Consortium (W3C) (2015) explizit deklariert, sind visuelle Neuanordnungen mittels Eigenschaften wie `order` oder Rasterplatzierungen kein Ersatz für eine korrekte, logische Reihenfolge des HTML-Quelltexts. Dies begründet sich darin, dass nicht-visuelle Medien, wie beispielsweise Screenreader zur Sprachausgabe für sehbehinderte Menschen, sowie die sequentielle Tastaturnavigation (`Tab`-Taste) ausschließlich der strukturellen Hierarchie des DOM-Strukturen folgen und CSS-Positionierungen ignorieren. Wird ein Element im HTML-Code folglich am Ende des Dokuments platziert, jedoch per CSS Grid visuell an den Anfang verschoben, liest das Sprachsystem dieses dennoch als letztes Element vor. Für die Entwicklung responsiver Webseiten resultiert daraus die fundamentale Anforderung, dass die logische Barrierefreiheit stets im Markup verankert sein muss und CSS Grid ausschließlich zur adaptiven Oberflächenpräsentation genutzt werden soll (Mozilla Developer Network (MDN), 2025a).

5.2.6. Responsive Anpassungsfähigkeit

CSS Grid weist hinsichtlich seiner responsiven Anpassungsfähigkeit eine technologische Dualität auf: Während grundlegende, strukturelle Layout-Sprünge auf der Makro-Ebene typischerweise via Viewport-Bedingungen mit Media Queries (mehr dazu im Unterkapitel 5.4) getriggert werden müssen, erlaubt die Technologie auf der Mikro-Ebene eine intelligente, media-query-freie Flexibilität. Durch mathematische Ausdrücke, wie `grid-template-columns: repeat(auto-fit, minmax(300px, 1fr))` erfolgt die Anpassung folglich auf Basis des lokal verfügbaren Kontextes (House, 2021).

5.3. Analyse CSS-Subgrid

5.3.1. Layout und Wirkung

Als Erweiterung des klassischen Grids (Grid Level 2) unterstützt CSS-Subgrid die Umsetzung tiefenverschachtelter, mehrdimensionaler Rasterstrukturen. Kindelemente eines Grids müssen dadurch kein eigenes, isoliertes Raster aufbauen, sondern ordnen sich nahtlos in die Zeilen- und Spaltenstruktur des übergeordneten Eltern-Rasters ein (World Wide Web Consortium (W3C), 2025c). Die Technologie bietet hierzu die Option einer einseitigen oder zweiseitigen Achsenvererbung. Entwickler können wahlweise nur die Spalten (`grid-template-columns: subgrid`), nur die Zeilen (`grid-template-rows: subgrid`) oder beide Achsen gleichzeitig vom übergeordneten Eltern-Grid vererben lassen (Mozilla Developer Network (MDN), 2025b). Durch diesen direkten Bezug auf das implizite und explizite Elternraster übernimmt das Subgrid exakt dessen Spurgrößen, Liniennamen und Abstände, wodurch komplexe, verschachtelte Teilstrukturen ohne mathematische Näherungswerte realisiert werden können (Andrew, 2017).
Hinsichtlich der gestalterischen Hierarchisierung garantiert diese Logik horizontale und vertikale Fluchtlinien über Komponentengrenzen hinweg, da die Kindelemente eigenständiger Bausteine Zugriff auf dieselben Rasterlinien besitzen (House, 2021). Zudem bleiben im Subgrid vererbte Rasterlinien und benannte Bereiche des äußeren

Rasters erhalten, was eine intuitive visuelle Zuordnung tief verschachtelter Fragmente erlaubt (World Wide Web Consortium (W3C), 2025b). Diese Eigenschaften befähigen CSS-Subgrid für gleichmäßige Card- und Teaser-Gantry-Layouts, bei denen innere Segmente wie Headerzeile, Überschrift und Footer-Button trotz variierender Textlängen horizontal exakt bündig fluchten sollen (House, 2021). Ebenso eignet sich die Technologie für komplexe Dashboard-UIs und Datenformulare, um Beschriftungen und Eingabefelder innerhalb verschachtelter, modularer Webkomponenten symmetrisch auszurichten, ohne starre, feste Spaltenbreiten pro Komponente erzwingen zu müssen (Mozilla Developer Network (MDN), 2025b).

5.3.2. Modularität und Wartbarkeit

Im Kontext modularer Softwarearchitekturen begünstigt CSS-Subgrid eine gekapselte Modulentwicklung. Die visuelle Ausrichtung innerhalb eines inneren Mikro-Layouts erfolgt völlig ohne hardcodierte Dimensionen (House, 2021). Ein zentraler Vorteil ist die inhaltsgetriebene Höhensynchronisation modularer Einheiten: Bei einem dynamischen Textzuwachs innerhalb eines Elements ziehen die Zeilenhöhen benachbarter, ansonsten völlig eigenständiger Komponenten vollautomatisch und synchron mit (Andrew, 2017). Die Komponente passt sich somit kontextbezogen an die jeweilige Umgebung an und richtet sich erst beim konkreten Einbetten in das übergeordnete Grid an diesem aus (House, 2021).
Für moderne Design-Systeme liefert Subgrid eine native Lösung, um die innere Struktur isolierter Card-Komponenten bündig aneinander auszurichten (Mozilla Developer Network (MDN), 2025b). Weil benannte Rasterlinien des Hauptlayouts tief in die Komponentenhierarchie durchgereicht und wiederverwendet werden können, fungiert das System als Vererbungswerkzeug für strukturelle Design-Tokens (World Wide Web Consortium (W3C), 2025c). Diese Makro-Mikro-Verknüpfung stellt sicher, dass tief verschachtelte UI-Fragmente direkt mit dem globalen Seitenraster interagieren können (Andrew, 2017). Diese Verschachtelungsunabhängigkeit bleibt auch dann verlustfrei erhalten, wenn das Element durch beliebig viele funktionale DOM-Wrapper-Elemente umschlossen ist (World Wide Web Consortium (W3C), 2025c).

5.3.3. Technische Effizienz und Performance

Die softwaretechnische Effizienz von CSS-Subgrid äußert sich primär in einer drastischen HTML-Verschlankung. Die Notwendigkeit rein visueller Hilfs-HTML-Strukturen innerhalb modularer Komponenten wird vollständig eliminiert (Andrew, 2017). Gleichzeitig wird die CSS-Code-Redundanz minimiert, da separate, wiederholende Spalten- und Zeilenberechnungen für Kindelemente durch die einfache Vererbung per `subgrid`-Wert entfallen (World Wide Web Consortium (W3C), 2025c). Die gesamte strukturelle Layout-Logik wird somit am übergeordneten Grid-Container zentralisiert, was zu sauberem, selbstdokumentierendem Code führt (House, 2021).
Aus Sicht der Browser-Performance zeichnet sich die Technologie durch eine effiziente Single-Pass-Performance aus. Die Browser-Engine berechnet die gesamte verschachtelte Struktur in einer einzigen Layout-Phase, wodurch kaskadierende, rechenintensive Reflows vermieden werden. Zudem trägt Subgrid zur Vermeidung von Layout-

Sprüngen bei, da bei dynamischen API-Inhaltsänderungen eine sofortige Platzreservierung und synchrone Anpassung aller Teilstrukturen stattfindet. Durch diese Automatismen ermöglicht Subgrid einen vollständigen Verzicht auf fehleranfällige JavaScript-Ausrichtungsskripte (House, 2021) und reduziert zeitgleich die Anzahl isolierter, komplexer Breakpoints für verschachtelte Teilstrukturen (Andrew, 2017).

5.3.4. Wartbarkeit und Developer Experience

Das mentale Modell von CSS-Subgrid basiert auf einem konsistenten Vererbungskonzept. Da es die bekannte, native CSS-Vererbungslogik auf das zweidimensionale Raster überträgt, ist die Arbeitsweise für Entwickler sehr intuitiv nachvollziehbar (World Wide Web Consortium (W3C), 2025c). Es erfordert keine abstrakte Geometrie-Logik oder mathematische Umrechnungen, da bestehende Rastermaße klar übernommen werden (House, 2021). Das System bietet folglich einen zentralen Refactoring-Punkt: Globale Design-Updates, wie die Änderung von Spaltenbreiten oder Abständen, müssen ausschließlich am äußeren Eltern-Grid vorgenommen werden; ein manuelles Nachjustieren von Kindkomponenten entfällt (Andrew, 2017). Bei dynamischem Content-Wachstum ist eine hohe Wartbarkeit garantiert, da tiefe Verschachtelungen fehlerfrei mitskalieren (House, 2021). Moderne Browser-Entwicklungswerkzeuge unterstützen diesen Prozess durch eine native DevTools-Erkennung, die Subgrids über dedizierte Badges im DOM-Baum kennzeichnet, während farbige Linien-Overlays die exakte visuelle Nachverfolgung der Vererbungslinien im Viewport ermöglichen (siehe Abbildung 5.6) (Mozilla Foundation & Mozilla Corporation, 2026).

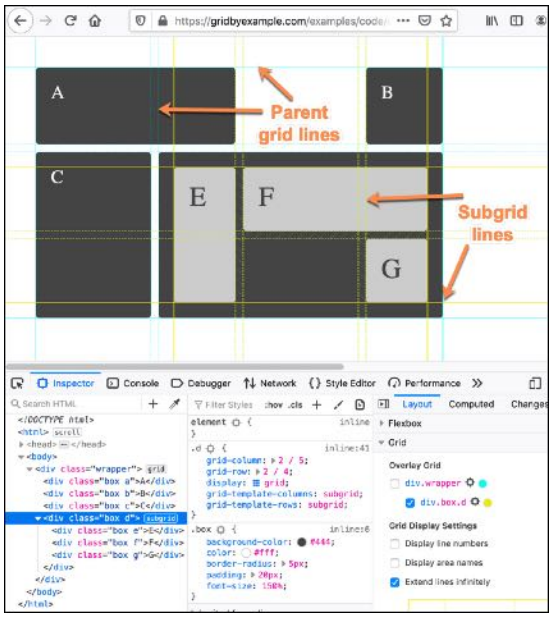


Abbildung 5.6.: Visuelle Inspektion eines CSS Subgrids in den Firefox DevTools inklusive Badge-Kennzeichnung und Overlay-Linien (Mozilla Foundation & Mozilla Corporation, 2026)

5.3.5. Nachhaltigkeit und Barrierefreiheit

Unter dem Gesichtspunkt der Barrierefreiheit leistet CSS-Subgrid einen wesentlichen Beitrag zum Erhalt der logischen Dokumentenhierarchie. Entwickler sind nicht mehr gezwungen, semantische HTML-Elemente, wie `<article>`, `<section>` oder `<aside>` strukturell aufzubrechen oder flacher zu gestalten, nur um sie in ein starres Rastersystem zu zwingen (World Wide Web Consortium (W3C), 2025c). Durch den resultierenden Verzicht auf Layout-Hilfs-Wrapper bleibt der DOM-Baum sauber und linear. Hilfstechnologien, wie Screenreader, können die Inhalte somit fehlerfrei in der korrekten, logischen Reihenfolge erfassen und wiedergeben.
Die technologische Zukunftssicherheit ist hoch, da CSS-Subgrid als offizieller W3C-Standard (Grid Level 2) fester Bestandteil des CSS-Kerns ist und keine Gefahr der Veraltung besteht (World Wide Web Consortium (W3C), 2025c). Der moderne Browser-Support ist plattformübergreifend flächendeckend gegeben (siehe Abbildung 5.7) (Mozilla Developer Network (MDN), 2025b). Herausforderungen ergeben sich jedoch beim Missbrauch der ,,Source-Order Independence“: Wird die visuelle Anordnung im Subgrid zu drastisch von der HTML-Reihenfolge entkoppelt, drohen gravierende Accessibility-Mängel für Tastaturnutzer (World Wide Web Consortium (W3C), 2025c). Bei Altsystemen muss die Abwärtskompatibilität über Fallbacks abgefangen werden (House, 2021). Zudem besteht ein Risiko für Kaskaden-Fehler bei der Fehlersuche, da Änderungen am weit entfernten Eltern-Grid unerwartete visuelle Auswirkungen in tief verschachtelten Subgrids auslösen können (Andrew, 2017).



Abbildung 5.7.: Diagramm zur Veranschaulichung der Browserunterstützung von Subgrid (Alexis Deveria, 2026b)

5.3.6. Responsive Anpassungsfähigkeit

Innerhalb responsiver Layoutkonzepte ermöglicht CSS-Subgrid ein synchrones Mitskalieren aller verschachtelten Module direkt über das fluide Haupt-Raster (House, 2021). Da fluide Einheiten des Eltern-Rasters, wie `fr` oder `minmax()`, automatisch übernommen werden, erfordern Kindkomponenten keine eigenen responsiven Deklarationen (House, 2021). Die Reaktion auf Größenänderungen erfolgt somit primär auf der

Makro-Ebene des zentralen äußeren Containers (Andrew, 2017), während die physikalischen Änderungen über die kaskadierende Vererbung unmittelbar an die Mikroebene weitergegeben werden (Mozilla Developer Network (MDN), 2025b).
Daraus resultiert eine erhebliche Media-Query-Minimierung, da die automatische Rasterkoppelung manuelle Breakpoints für Subkomponenten weitgehend überflüssig macht (Andrew, 2017). Durch eine bewusste Achsenunabhängigkeit können Entwickler das Verhalten geräteabhängig steuern; beispielsweise über eine fluide, einspaltige Vererbung auf mobilen Displays und ein starres, mehrspaltiges Raster auf Desktop-Bildschirmen (Mozilla Developer Network (MDN), 2025b). Die softwaretechnische Code-Effizienz führt letztlich zu einer enormen Zeiteinsparung und Fehlerreduktion beim Refactoring, da komplexe manuelle Breitenberechnungen für Kindelemente obsolet werden (House, 2021).

5.4. Analyse Media-Queries

5.4.1. Steuerungslogik und Layout-Wirkung

Das funktionale Prinzip der Media-Queries basiert auf einem reinen Viewport-Fokus. Die Schaltung responsiver Bedingungen orientiert sich ausschließlich an den globalen Dimensionen des Browserfensters oder des physischen Geräte-Bildschirms (World Wide Web Consortium (W3C), 2026). Neben den reinen Abmessungen erlaubt das Prinzip die Abfrage gerätespezifischer Merkmale, wie die Erkennung physikalischer Medientypen, bspw. `print` versus `screen`, sowie spezifischer Hardware-Eigenschaften, darunter die Pixeldichte oder die Touch-Fähigkeit eines Endgeräts (Frain, 2015). Systemisch bedingt weist das Prinzip jedoch eine vollständige Ignoranz gegenüber der tatsächlichen Breite oder der konkreten Platzierung einer Komponente innerhalb einer umschließenden Elternkomponente auf; es existiert keinerlei lokale Kontext-Sensitivität (Marcotte, 2010).
In Bezug auf die dynamische Restrukturierung und Hierarchisierung von Inhalten eignen sich Media-Queries primär für den makroskopischen Layout-Wechsel. Dies umfasst die grobe Neuanordnung großer Seitenbereiche, wie das Umschalten von einer mehrspaltigen Desktop-Ansicht mit integrierter Sidebar auf ein einspaltiges mobiles Layout (Marcotte, 2010). Darüber hinaus wird die Technologie für eine selektive Inhaltssteuerung genutzt, um die Informationsdichte über das Ein- und Ausblenden ganzer Sektionen mittels `display: none` an kleine Displays anzupassen (Frain, 2015). Auf typografischer Ebene ermöglicht das Prinzip die Etablierung einer globalen Hierarchie durch die schrittweise, seitenweite Anpassung von Basisschriftgrößen und Layoutabständen an vordefinierten Viewport-Schwellenwerten (World Wide Web Consortium (W3C), 2026). Architektonisch prädestiniert dieses Leistungsprofil Media-Queries für Ganzseiten- und Makro-Layouts, um das übergeordnete Seitenskelett, die Hauptnavigation sowie globale Footer-Bereiche zu strukturieren und klassische, geräteorientierte Seiten-Templates, also Smartphone, Tablet und Desktop, zu verwalten (Marcotte, 2010; Frain, 2015).

5.4.2. Komponentenarchitektur und Modularität

Im Kontext moderner Frontend-Architekturen offenbaren Media-Queries eine mangelhafte Unterstützung unabhängiger Komponenten. Da die visuelle Darstellung einer Komponente strikt an den globalen Viewport statt an den real zur Verfügung stehenden Platz gekoppelt ist, wird die Entwicklung isolierter, plattformunabhängiger Module erheblich erschwert (Frain, 2015).
Daraus resultiert ein hohes Risiko für ein visuelles „Zerschießen“ von UI-Elementen beim Kontextwechsel. Das Steuerungsprinzip bietet keinerlei Schutz vor strukturellen Layoutbrüchen, wenn ein Modul in unterschiedliche Layoutkontexte eingebettet wird. Eine für den Desktop-Viewport optimierte Komponente bricht visuell zusammen, sobald sie auf einem großen Bildschirm in eine schmale, restriktive Sidebar integriert wird, da sie fälschlicherweise die globalen Desktop-Styles statt der lokal erforderlichen Reduktion aufruft (Marcotte, 2010).

5.4.3. Technische Effizienz und Performance

Unter softwarearchitektonischen Gesichtspunkten führt der exklusive Einsatz von Media-Queries zu einer erhöhten CSS-Code-Redundanz. Da Layoutbedingungen kaskadierend über separate Breakpoint-Blöcke hinweg definiert werden müssen, ist das wiederholte Überschreiben identischer CSS-Regeln für verschiedene Geräteklassen unumgänglich (Frain, 2015). Dies bedingt eine fundamentale Verletzung der CSS-Kapselung, da globale Viewport-Bedingungen und komponenten-spezifische Styles innerhalb des Stylesheets unweigerlich miteinander vermischt werden (Frost, 2022).
Hinsichtlich der Client-Performance weisen Media-Queries hingegen eine hohe native Rendering-Performance auf. Die Verarbeitung der Viewport-Berechnungen erfolgt hocheffizient direkt innerhalb der Layout-Engine des Browsers, ohne den JavaScript-Haupt-Thread, auch *Main Thread* genannt, bei einer Veränderung des Browserfensters zu belasten (World Wide Web Consortium (W3C), 2026). Als JavaScript-Ersatz fungiert das Prinzip jedoch nur unvollständig. Aufgrund der Unfähigkeit, lokale Größenänderungen von Elementen zu detektieren, bleibt für echte komponentenbasierte Layoutwechsel ohne den Einsatz von Container-Queries die Implementierung rechenintensiver `ResizeObserver`-Skripte in JavaScript zwingend erforderlich (Frain, 2015).

5.4.4. Wartbarkeit und Developer Experience

Die Kausalität der responsiven Bedingungen zeichnet sich bei Media-Queries durch eine hohe Verständlichkeit aus. Das mentale Modell ist für Entwickler intuitiv, da die visuelle Layoutänderung direkt an eine greifbare und im Browser leicht nachvollziehbare Gesamtbreite des Viewports gekoppelt ist (Marcotte, 2010). Demgegenüber steht jedoch ein erheblich ansteigender Pflegeaufwand bei der Skalierung großer Systeme. Updates an zentralen Breakpoints innerhalb eines Design-Systems erweisen sich als fehleranfällig, da Modifikationen an den globalen Schwellenwerten unerwartete visuelle Seiteneffekte in weit entfernten, vermeintlich isolierten Komponenten auslösen können (Frost, 2022). Positiv zu bewerten ist der Browser-Tooling-Support. Moderne Entwicklungswerkzeuge bieten über spezialisierte Modi, zum Beispiel den *Responsive Design Mode* in Google Chrome (Kayce Basques, 2024), eine hervorragende visuelle

Unterstützung zur stufenlosen Simulation jeglicher Viewportgrößen und Geräteklassen im Entwicklungsprozess (Mozilla Developer Network (MDN), 2026c).

5.4.5. Nachhaltigkeit und Barrierefreiheit

Hinsichtlich der Barrierefreiheit bergen drastische, via Media-Query gesteuerte Layout-Sprünge ein inhärentes Fokus-Reihenfolge-Risiko. Wenn Elemente per Viewport-Breakpoint visuell stark umstrukturiert werden, die native Tastatur-Tab-Reihenfolge im HTML-Quelltext jedoch statisch unverändert bleibt, entstehen gravierende Defizite bei der Zugänglichkeit für motorisch oder visuell eingeschränkte Nutzer (World Wide Web Consortium (W3C), 2026). Ein wesentlicher Vorteil für die Nachhaltigkeit ist hingegen die uneingeschränkte Zukunftssicherheit des Prinzips. Als absolut etablierter Kern-Webstandard weist die Technologie einen flächendeckenden und fehlerfreien Browser-Support über alle Betriebssysteme und Geräteklassen hinweg auf (Frain, 2015).
Zudem zeichnen sich Media-Queries durch eine exzellente Funktionalität bei der Abfrage benutzerdefinierter Barrierefreiheits-Einstellungen aus. Sie erlauben die direkte, native Erkennung von Nutzer-Präferenzen im Betriebssystem, um beispielsweise Animationen über `prefers-reduced-motion` einzuschränken oder optimierte Kontrast-Modi für sehbehinderte Menschen bereitzustellen (Mozilla Developer Network (MDN), 2026c).

5.5. Analyse Container-Queries

5.5.1. Steuerungslogik und Layout-Wirkung

Im Gegensatz zu traditionellen Viewport-Ansätzen verfolgen Container-Queries einen reinen Komponenten-Fokus. Die Schaltung responsiver Bedingungen orientiert sich via `@container` ausschließlich an den physikalischen Dimensionen des direkt umschließenden Eltern-Containers anstelle des gesamten Bildschirms (World Wide Web Consortium (W3C), 2022b). Diese logische Neuausrichtung ermöglicht eine hochgradig lokale Restrukturierung von Inhalten. Innere Element-Hierarchien können vollkommen unabhängig, beispielsweise durch den dynamischen Wechsel von einer Listen- zu einer Card-Ansicht, rein basierend auf dem lokalen Platzgewinn oder -verlust des umschließenden Raums neu geordnet werden (web.dev, 2025).
Aufgrund dieser Charakteristika eignet sich das Prinzip primär für anspruchsvolle Isolations-Szenarien. Es ist optimiert für hochgradig modulare, kontextunabhängige UI-Komponenten wie Teaser-Elemente, Dashboard-Widgets oder Produktkarten innerhalb dynamischer, flexibler Layoutstrukturen (Graham, 2024). Das Makro-Layout der Seite wird hierbei bewusst ausgeblendet, um die gestalterische Kontrolle vollständig auf das Mikro-Layout des Moduls zu verlagern.

5.5.2. Komponentenarchitektur und Modularität

Hinsichtlich der Komponentenarchitektur gewähren Container-Queries UI-Modulen eine hohe Unabhängigkeit. Die Technologie erlaubt die Entwicklung plattformunabhängiger

Komponenten, die ihr responsives Verhalten inhärent steuern und ohne globale externe Schalter operieren (Graham, 2024). Da sowohl die visuellen Styles als auch die responsiven Bedingungen vollständig innerhalb des jeweiligen Komponentenordners gekapselt bleiben, wird die softwaretechnische Wiederverwendbarkeit maßgeblich gesteigert.
Ein wesentlicher Vorteil dieses Ansatzes ist der umfassende Kontext-Schutz der UI-Elemente. Container-Queries bieten einen absoluten Schutz vor visuellem „Zerschießen“ beim Einbetten in unterschiedliche Layoutbereiche. Eine Komponente adaptiert ihre Oberfläche in einer schmalen Sidebar vollautomatisch so, wie es für ein mobiles Smartphone-Display angemessen ist, während sie im raumgreifenden Hauptinhalt ohne Code-Duplizierung die visuelle Ausprägung eines Desktop-Layouts annimmt (World Wide Web Consortium (W3C), 2022b).

5.5.3. Technische Effizienz und Performance

Die softwaretechnische Effizienz dieses Prinzips äußert sich in einer radikalen CSS-Verschlankung. Seitenlange, verschachtelte Media-Query-Kaskaden sowie redundante, kontextspezifische Hilfsklassen für verschiedene Seitensektionen werden im Stylesheet obsolet (Graham, 2024). Layout-Bedingungen verbleiben exklusiv im Scope der jeweiligen Komponente, was eine gezielte Kapselung und eine klare Trennung der Zuständigkeiten begünstigt.
Auf Ebene der Client-Performance garantiert das Prinzip eine browsernative Performance. Obwohl die Überwachung lokaler Elementabmessungen mathematisch komplexer ist als die Abfrage des globalen Viewports, erfolgt die Berechnung direkt innerhalb der Layout-Engine des Browsers (World Wide Web Consortium (W3C), 2022b). Für die Developer Experience resultiert daraus eine vollständige JavaScript-Obsoleszenz im Bereich responsiver Komponentensteuerungen. Rechenintensive und fehleranfällige `ResizeObserver`-Skripte zur manuellen Überwachung lokaler Element-Breiten können restlos aus der Codebasis entfernt werden (Mozilla Developer Network (MDN), 2026a).

5.5.4. Wartbarkeit und Developer Experience

Trotz der technischen Vorteile erfordern Container-Queries anfangs eine erhöhte mentale Komplexität aufseiten der Entwickler. Das Prinzip verlangt ein grundlegendes Umdenken, da die Kausalität weniger intuitiv als bei Media-Queries ist. Um eine responsive Reaktion zu triggern, muss das umschließende Eltern-Element zuvor explizit über die Eigenschaft `container-type`, zum Beispiel mit `inline-size`, als struktureller Container deklariert werden (Mozilla Developer Network (MDN), 2026a).
Ist diese Hürde überwunden, bietet das Prinzip jedoch eine zukunftssichere Pflege bei globalen Design-System-Updates. Modifikationen am responsiven Verhalten einer einzelnen Komponente erfordern keinerlei Anpassungen am globalen Seitenskelett und eliminieren visuelle Seiteneffekte in Fremdmodulen (Graham, 2024). Unterstützt wird die Implementierung durch einen modernen DevTools-Support in aktuellen Browser-Inspektoren, welche dedizierte Container-Badges und Achsenindikatoren zur gezielten Fehlersuche bereitstellen (Mozilla Developer Network (MDN), 2026a).

5.5.5. Nachhaltigkeit und Barrierefreiheit

Unter dem Gesichtspunkt der Barrierefreiheit bergen Container-Queries ebenfalls, wie Media-Queries (siehe Unterkapitel 5.4.5) ein inhärentes Fokus-Reihenfolge-Risiko. Wenn innerhalb kleiner Container extreme visuelle Layout-Sprünge vollzogen werden, besteht die Gefahr, dass die visuelle Anordnung nicht mehr mit der statischen DOM- und Tab-Reihenfolge des HTML-Quelltexts übereinstimmt, was Barrierefreiheitsdefizite für Tastaturnutzer nach sich zieht (World Wide Web Consortium (W3C), 2022b). Demgegenüber steht eine Flexibilität bei benutzerdefinierten Barrierefreiheits-Einstellungen, insbesondere beim User-Zoom. Da Container-Queries auf den real entstehenden Platz innerhalb des Elements reagieren und nicht starr den physischen Bildschirm-Maßstab abfragen, passen sich Komponenten bei einer Vergrößerung des Text-Zooms harmonisch und fließend an den verknappten Raum an (Graham, 2024). Die technologische Zukunftssicherheit ist langfristig gesichert. Der aktuelle Browser-Support ist in allen modernen Haupt-Engines flächendeckend stabil gegeben, wodurch das Prinzip uneingeschränkt im produktiven Webbereich einsetzbar ist (Mozilla Developer Network (MDN), 2026a).

6. Herleitung der Layoutanforderungen im responsive Webdesign

Basierend auf der in Unterkapitel 4.3 vorgestellten Human-Centered-Design-Methodik und den dort definierten Spezifikationswerkzeugen werden nachfolgend die typischen Layoutanforderungen für das responsive Webdesign hergeleitet. Die Deduktion erfolgt systematisch aus den spezifischen Nutzungskontexten über die situativen Erfordernisse hin zu den technologischen Layoutanforderungen. Die dazugehörige ausgefüllte Tabelle ist im Anhang zu finden (siehe A1.

6.1. Dimensionierung und proportionale Raumverteilung

Der erste spezifische Nutzungskontext adressiert die strukturelle Flexibilität digitaler Oberflächen: Endanwender greifen in hochgradig dynamischen Situationen auf Webinhalte zu, wechseln zwischen Smartphones, Tablets und Desktop-Monitoren oder nutzen moderne Anzeigeformen wie den Split-Screen-Modus (Marcotte, 2010; Bhanarkar u. a., 2023).
Aus diesem Szenario resultiert ein grundlegendes ergonomisches Erfordernis aufseiten des Nutzers. Unter Anwendung der syntaktischen Struktur des User Experience Qualifications Board (UXQB) (2023) wird dieses wie folgt spezifiziert:

> „Als mobiler Endanwender muss man Webinhalte stufenlos an die Displayränder angepasst und ohne horizontales Scrollen verfügbar haben, um Informationen auf Bildschirmen jeder Dimension barrierefrei lesen zu können."

Um diese rein nutzerzentrierte Perspektive in eine funktionale Systemvorgabe zu überführen, wird das Erfordernis mithilfe der Rupp'schen Satzschablone in den Lösungsraum transformiert (Rupp u. die SOPHISTen, 2020). Daraus ergibt sich die verbindliche, technologische Layoutanforderung für ein **Fluid Layout**:

> „Die Webkomponente auf der Webseite **muss** dem Endanwender die Möglichkeit bieten, die Breite von Inhaltselementen proportional und stufenlos entlang relativer Einheiten zu skalieren."

6.2. Ergonomie und Interaktionsflächen auf Touch-Endgeräten

Der darauffolgende Nutzungskontext betrifft die physische Interaktion mit der Benutzeroberfläche: Anwender bedienen Webseiten auf mobilen Geräten häufig unter Ablenkung; zum Beispiel unterwegs in öffentlichen Verkehrsmitteln und einhändig ausschließ-

lich mit dem Daumen auf einem Touchscreen. Der damit begrenzte zu erreichende Screen wird in Abbildung 6.1 in grün angezeigt.

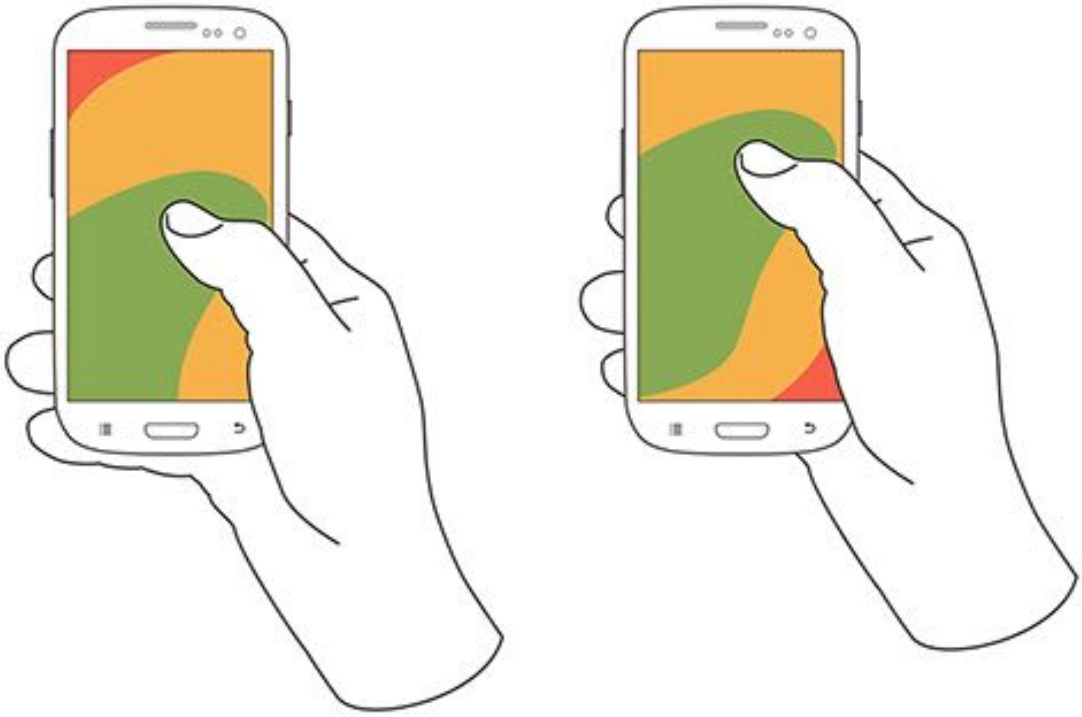


Abbildung 6.1.: Darstellung des begrenzten zu erreichenden Screens eines Smartphones (Hoober, 2013)

Aus dieser motorischen Einschränkung leitet sich ein ergonomisches Erfordernis ab, das nach der Syntax des User Experience Qualifications Board (UXQB) (2023) wie folgt definiert wird:

> „Als mobiler Endanwender muss man Steuerungselemente in einer ergonomisch erreichbaren Größe und Anordnung verfügbar haben, um Links zielsicher und ohne Fehlklicks mit dem Finger aktivieren zu können."

Die Transformation dieser Nutzerwünsche in den Lösungsraum erfolgt über die Rupp'sche Satzschablone (Rupp u. die SOPHISTen, 2020). Sie definiert die verbindliche Systemvorgabe für die **Navigation**:

> „Das Navigations-Modul muss dem Endanwender die Möglichkeit bieten, Steuerungselemente bei unzureichendem horizontalem Platzangebot elastisch in die Vertikale umzubrechen."

6.3. Größenunterschiede von Medien auf Webseiten

Ein weiterer kritischer Nutzungskontext ergibt sich bei der Darstellung von verschiedenen Größen von Medien: Endanwender konsumieren auf Webseiten eine Vielzahl an

visuellen Medien wie Bilder oder Videos, welche auf Geräten mit stark differenzierenden Bildschirmbreiten und Pixeldichten fehlerfrei dargestellt werden müssen (Firtman, 2010).
Um die visuelle Konsistenz für den Betrachter zu wahren, entsteht ein strukturelles Erfordernis, das nach UXQB-Standard (User Experience Qualifications Board (UXQB), 2023) formuliert wird:

> „Als mobiler Endanwender muss man eingebettete Medien und Container stufenlos an den jeweils verfügbaren Platz angepasst vorfinden, um visuelle Inhalte ohne Beschneidung, Verzerrung des Seitenverhältnisses oder horizontalen Layout-Overflow erfassen zu können."

Mithilfe der Rupp'schen Schablone (Rupp u. die SOPHISTen, 2020) wird diese fachliche Notwendigkeit in eine funktionale Systemanforderung für eine
Flexible Größenanpassung übersetzt:

> „Die Webkomponente auf der Webseite **muss** fähig sein, die Dimensionen von Containern und eingebetteten Medien proportional zu skalieren, ohne das inhärente Aspektverhältnis zu verzerren."

6.4. Kontextübergreifende Modularität in Design-Systemen

Aus der Perspektive moderner Softwarearchitektur ergibt sich ein komponentenbasierter Nutzungskontext: Webapplikationen werden nach dem Komponenten-Paradigma aufgebaut, wodurch identische UI-Module, zum Beispiel Teaser-Karten, flexibel an unterschiedlichen Positionen der Applikation, wie einer schmalen Sidebar oder dem breiten Hauptbereich, eingesetzt werden (Frost, 2016).
Daraus resultiert ein visuelles Erfordernis für eine konsistente Informationsdarstellung, spezifiziert nach User Experience Qualifications Board (UXQB) (2023):

> „Als Endanwender muss man UI-Komponenten unabhängig von ihrer Platzierung auf der Webseite in einem visuell fehlerfreien und an den verfügbaren Platz angepassten Layout dargeboten bekommen, um Informationen kontextübergreifend konsistent erfassen zu können."

Die Überführung in den technischen Lösungsraum mittels der Rupp'schen Satzschablone liefert die Systemanforderung für die **Wiederverwendbarkeit von Komponenten** (Rupp u. die SOPHISTen, 2020):

> „Das Layout-System der Webseite **muss** fähig sein, die visuelle Darstellung einer Webkomponente ausschließlich auf Basis des lokal verfügbaren Platzes ihres direkt umschließenden Eltern-Elements zu steuern."

6.5. Varianz und Dynamik datengetriebener Inhalte

Ein weiterer kritischer Nutzungskontext ergibt sich aus der Datenarchitektur moderner Webanwendungen: Das Interface wird mit dynamischen Daten, zum Beispiel über

APIs, Datenbanken oder unterschiedliche Sprachübersetzungen befüllt, deren exakte Textlängen und Zeichenanzahlen dem Layout zur Designzeit unbekannt sind (Kholmatova, 2017).
Um die visuelle Konsistenz für den Betrachter zu wahren, entsteht ein strukturelles Erfordernis, das nach UXQB-Standard (User Experience Qualifications Board (UXQB), 2023) formuliert wird:

> „Als Endanwender muss man visuell zusammengehörige Informationen, zum Beispiel Produktmerkmale oder Funktionstasten, auf einer einheitlichen horizontalen Fluchtlinie wahrnehmen können, um Inhalte strukturiert vergleichen zu können."

Mithilfe der Rupp'schen Schablone (Rupp u. die SOPHISTen, 2020) wird diese fachliche Notwendigkeit in eine funktionale Systemanforderung für **Dynamische Inhaltsmengen** übersetzt:

> „Die Webkomponente auf der Webseite **muss** fähig sein, die Höhen verschachtelter Kindelemente über Komponentengrenzen hinweg synchron an den längsten Inhalt anzupassen."

6.6. Divergenz zwischen visueller Hierarchie und Semantik

Der fünfte Nutzungskontext betrachtet die Barrierefreiheit und die gleichzeitige Bereitstellung für unterschiedliche Ausgabemedien: Eine Webseite wird simultan von sehenden Nutzern auf mobilen Geräten, die eine daumenfreundliche Anordnung der wichtigsten Elemente erwarten und von sehbehinderten Nutzern via Screenreader konsumiert (**?**).
Um beiden Nutzergruppen gerecht zu werden, lässt sich nach der UXQB-Syntax folgendes Erfordernis definieren (User Experience Qualifications Board (UXQB), 2023):

> „Als Screenreader-Nutzer muss man die logische und semantische Reihenfolge des Quelltextes unverändert linear vorfinden, während man als mobiler Nutzer eine visuell optimierte Anordnung benötigt, um die Anwendung barrierefrei und ergonomisch bedienen zu können."

Die datenseitige Entkopplung wird durch die Rupp'sche Schablone (Rupp u. die SOPHISTen, 2020) als bindende Systemanforderung für die **Layout-Unabhängigkeit von der HTML-Struktur** formuliert:

> „Die Webkomponente der Webseite **muss** dem Entwickler die Möglichkeit bieten, die visuelle Anordnung von UI-Elementen auf dem Bildschirm vollständig von ihrer semantischen Reihenfolge im HTML-Quelltext zu entkoppeln."

6.7. Software-Qualität und strukturelle Wartbarkeit

Der letzte Nutzungskontext fokussiert die softwarequalitativen Ansprüche der internen Stakeholder nach ISO/IEC 25010 (ISO, 2011): Entwickler-Teams müssen im Rahmen des Software-Lebenszyklus komplexe, tief verschachtelte Layoutstrukturen nach

ISO/IEC 25010 anpassen, erweitern oder refaktorieren.
Aus der Wartungsperspektive ergibt sich ein unmissverständliches Erfordernis für den Entwicklungsprozess, formalisiert nach UXQB (User Experience Qualifications Board (UXQB), 2023):

> „Als Entwickler muss man eine schlanke, semantische und redundanzfreie Quelltext-Struktur verfügbar haben, um das CSS-Layout ohne unvorhergesehene Seiteneffekte und ohne das manuelle Hinzufügen rein visueller HTML-Hilfs-Wrapper erweitern zu können.“

Die finale Ableitung in den technischen Lösungsraum beschreibt mithilfe der Rupp'schen Satzschablone (Rupp u. die SOPHISTen, 2020) die Anforderung an die **Skalierbarkeit und Wartbarkeit** des Codes:

> „Die Webkomponente der Webseite **muss** fähig sein, die Layout-Rastersysteme von tief verschachtelten Kindelementen direkt an die Achsenstrukturen des übergeordneten Eltern-Rasters zu binden.“

7. Auswertung und Einordnung der Ergebnisse

Nach der ausführlichen Analyse der CSS-Layouttechnologien war es nun möglich die in Unterkapitel 6 hergeleiteten Layoutanforderungen im RWD mit den Layouttechnologien zu verknüpfen und den zweiten Teil der Forschungsfrage: „...für welche Layoutanforderungen eignet sich welche Technologie am besten?" zu beantworten. Dazu wurde die in Unterkapitel 4.4 vorgestellte Tabelle 4.5 genutzt.
Wichtig zu beachten ist, dass einige Layoutanforderungen die Verwendung von mehr als einer CSS-Layouttechnologie erwarten. Dies liegt unter anderem an den primären Eigenschaften der Anforderungen, die die Kombination, meist zweier CSS-Layoutanforderungen erfordern.

7.1. Entscheidungsmodell Ergebnisse

Aus zeitlichen Gründen wird im folgenden Unterkapitel nur die jeweilige Auswahl der passenden CSS-Technologien pro Layoutanforderung zusammengefasst. Die ausgefüllten Tabellen der einzelnen Layoutanforderungen kann im Anhang eingesehen werden, sodass die Schlussfolgerung der ausgewählten CSS-Technologien verstanden wird.

Für die Layoutanforderung **Fluid Layout** (siehe Tabelle B2) sind vor allem die proportionale und relative Skalierung von Webkomponenten und die damit verbundene stufenlose Anpassung dieser Komponenten, ohne die Angabe fester Pixelwerte, von Wichtigkeit. Somit wären die technischen Eigenschaften von CSS Grid hilfreich, um den Anforderungen des Fluid Layouts gerecht zu werden. CSS-Grid (5.2) bricht mit der Beschränkung linearer Inhaltsflüsse und verteilt Räume über native Einheiten wie fr oder minmax() (World Wide Web Consortium (W3C), 2015). Um zusätzlich die stufenlose Anpassung von Webkomponeten auf der Webseite zu erreichen sollte das Grid mit den Media-Queries kombiniert werden. Media-Queries (5.4) bilden hierzu die makroskopische Steuerungszentrale, indem sie das globale Seitenskelett an vordefinierten Viewport-Schwellenwerten stufenweise anpassen (World Wide Web Consortium (W3C), 2026; Marcotte, 2010).

Für die Umsetzung der Layoutanforderung **Navigation** (siehe Tabelle B3) erweist sich vor allem eine Kombination aus Flexbox und Media Queries als idealer technologischer Ansatz. Da die responsive Gestaltung von Navigationsstrukturen primär eine eindimensionale Ausrichtung erfordert, bietet Flexbox hierfür das passende funktionale Fundament (MDN Web Docs, 2025a; Coyier, 2026). Flexbox sorgt innerhalb dieser Navigationsebenen für ein fehlerfreies, elastisches Mitatmen und ein zeilenwei-

ses Umbrechen der Menüglieder, was die berührungsfreundliche Bedienung auf Touch-Endgeräten maßgeblich unterstützt (Firtman, 2010). Um den Anforderungen im RWD vollumfänglich gerecht zu werden, steuern Media Queries ergänzend den makroskopischen Systemwechsel: Bei kritischem Platzmangel wird das Desktop-Menü vollständig ausgeblendet und eine mobile Navigation aktiviert.

Passend zu der Anforderung im RWD der **flexiblen Größenanpassung** (siehe Tabelle B4), bei der das elastische Wachsen bzw. Schrumpfen von Elementen, sowie das Verhindern von Layout Overflows zu beachten sind, können Flexbox-Eigenschaften mit Container-Queries kombiniert werden. Während Flexbox über Eigenschaften wie `flex-grow` und `flex-shrink` das elastische Verhalten von Boxen auf Mikro-Ebene steuert, eliminieren Container-Queries den Layout Overflow, indem sie Medien und Container rein auf Basis des lokal verfügbaren Platzes im direkt umschließenden Eltern-Element skalieren (World Wide Web Consortium (W3C), 2025a; Marcotte, 2010; World Wide Web Consortium (W3C), 2022b).

Die kontextunabhängige Kapselung und das Einhalten der Modulbauweise nach dem Atomic Design (Frost, 2016) sind primäre Eigenschaften der Layoutanforderung der **Wiederverwendbarkeit von Komponenten** (siehe Tabelle B5), welche durch den Einsatz von Container Queries umgesetzt werden können. Container Queries (5.5) ermöglichen hierbei eine maximale Unabhängigkeit, da Komponenten ihr Layout inhärent und ohne globale, starre Breakpoint-Klassen kontrollieren können. Sie bilden das logische Fundament, indem sie der Komponente die Information über ihren lokal verfügbaren Platz bereitstellen. Innerhalb der Komponente werden dann Flexbox eingesetzt, um das eigentliche Interface-Layout passgenau umzustrukturieren. Da sämtliche Styles und die Responsivität vollständig im Komponentenordner gekapselt bleiben, lassen sich die Module nahtlos und fehlerfrei in moderne Design-Systeme integrieren (Frost, 2022; Graham, 2024).

Im RWD sind ebenfalls **dynamische Inhaltsmengen** (siehe Tabelle B6) als Layoutanforderungen fest verankert (Marcotte, 2010). Ihre primären Eigenschaften bestehen aus der inhaltsgetriebenen Höhen-Synchronisation, sowie der Vermeidung von Layout-Sprüngen beim Wechsel von verschiedenen Bildschrimgrößen. Diese Eigenschaften können durch das Verwenden von CSS-Subgrid eingehalten werden. CSS-Subgrid (5.3) löst das Problem variierender Inhaltsmengen in verschachtelten, modularen Einheiten, wie zum Beispiel Cards. Kindelemente erben das Raster des Eltern-Grids, wodurch Zeilenhöhen bei dynamischem Textzuwachs synchron mitziehen und horizontale Fluchtlinien über Komponentengrenzen hinweg perfekt erhalten bleiben. Außerdem werden Layout-Sprüngen vermieden, indem eine sofortige Platzreservierung und synchrone Anpassung aller Teilstrukturen bei dynamischen API-Inhaltsänderungen stattfindet.(Andrew, 2017; House, 2021).

Die Anforderung der **Layout-Unabhängigkeit von der HTML-Struktur** (siehe Tabelle B7) beschreibt die vollständige Trennung der visuellen von der logischen Dokumentenhierarchie, um Barrierefreiheit und Ergonomie simultan zu bedienen. CSS selbst garantiert diese strikte Entkopplung. Über Eigenschaften von CSS Grid, wie

beispielsweise `grid-template-areas`, lassen sich radikale Layout-Sprünge im Responsive Design visuell umsetzen (World Wide Web Consortium (W3C), 2025b; Marcotte, 2010). Die logische und barrierefreie HTML-Struktur bleibt für Screenreader dabei unangetastet im Quellcode verankert. Während CSS Grid die visuelle Neuanordnung im zweidimensionalen Raum übernimmt, dienen Media Queries ergänzend als Steuerinstanz, um diese Layout-Verschiebungen exakt auf Basis des jeweiligen Endgeräts auszuführen.

Für die Erfüllung der Kriterien **Skalierbarkeit und Wartbarkeit** (siehe Tabelle B8), insbesondere im Hinblick auf die Reduktion von Code-Redundanz und die Vermeidung von verschachtelten DOM-Strukturen gemäß ISO/IEC 25010 (ISO, 2011), empfiehlt sich der Einsatz von CSS Subgrid. CSS Subgrid (5.3) eliminiert rein visuelle Hilfs-Wrapper, indem Kindelemente die Achsenstrukturen des Eltern-Rasters direkt erben und somit keine künstlichen HTML-Ebenen für die Ausrichtung benötigen. (Garrett, 2022; Andrew, 2017).

7.2. Auswertung

Die Verknüpfung der hergeleiteten Layoutanforderungen mit den analysierten CSS-Technologien zeigt, dass sich eine performante und wartbare Umsetzung responsiver Websysteme heute nicht mehr auf eine einzelne, alles umfassende Technologie stützen lässt. Stattdessen zeichnet sich eine klare funktionale Arbeitsteilung ab, die eine softwarearchitektonische Trennung zwischen Makro- und Mikro-Ebene des Layouts nahelegt. Auf der Makro-Ebene, also dem globalen Seitengerüst, bleibt eine Kombination aus zweidimensionaler Rastersteuerung (`CSS-Grid`) und viewportbasierten Regeln (`Media-Queries`) weiterhin erforderlich, wie das Beispiel des *Fluid Layouts* zeigt. Auf der Ebene einzelner Komponenten hingegen lässt sich eine deutliche Verschiebung hin zu größerer technologischer Eigenständigkeit beobachten.

Getragen wird diese Entwicklung vor allem vom Zusammenspiel aus Flexbox,
CSS-Subgrid und Container-Queries. Die Analyseergebnisse zeigen, dass einzelne UI-Bausteine durch inhalts- und kontextbasierte Steuerungsprinzipien von sich aus responsiv reagieren können. Anforderungen wie die *Wiederverwendbarkeit von Komponenten* oder der Umgang mit *dynamischen Inhaltsmengen* lassen sich dadurch direkt in der jeweiligen Komponente lösen, ohne dass das globale Stylesheet zusätzlich mit sich wiederholenden Breakpoint-Regeln aufgebläht werden muss. Für die Frontend-Architektur ergibt sich daraus, im Sinne der ISO/IEC 25010 (ISO, 2011), ein spürbarer Gewinn an Übersichtlichkeit: Weil Layout-Logik lokal gekapselt bleibt, ziehen Änderungen an einer Komponente keine unerwarteten Folgen in entfernten Teilen des DOM-Baums nach sich, was sich positiv auf Wartbarkeit und Analysierbarkeit des Codes auswirkt (Frost, 2022; Graham, 2024).

Gleichzeitig verdient das Spannungsverhältnis zwischen gestalterischer Freiheit und Barrierefreiheit eine kritische Betrachtung. Die durch CSS-Grid und Flexbox ermöglichte *Layout-Unabhängigkeit von der HTML-Struktur* ist nicht ohne Risiko. Das W3C macht

in seinen Spezifikationen deutlich, dass eine rein visuelle Umsortierung über `order` oder Rasterpositionierung keinen Ersatz für die tatsächliche Reihenfolge im Quelltext darstellt, da weder Tastaturnavigation noch nicht-visuelle Ausgabemedien davon erfasst werden (World Wide Web Consortium (W3C), 2015; Mozilla Developer Network (MDN), 2025a). Wird die visuelle Darstellung zu weit von der logischen Dokumentstruktur entkoppelt, entstehen trotz eleganter technischer Lösung erhebliche Barrieren für betroffene Nutzergruppen. Daraus folgt für die praktische Umsetzung, dass die semantische Ordnung stets im HTML verankert bleiben muss, während CSS-Layouttechnologien sich auf ihre Rolle als reine Präsentationsschicht beschränken sollten.

In der Gesamtschau lässt sich der Stand des Responsive Webdesigns im Jahr 2026 treffend als ein ,,kontextsensitives Ökosystem" charakterisieren. An die Stelle starrer, gerätebezogener Viewport-Breakpoints tritt zunehmend ein fluid-elastischer Ansatz. Durch das Zusammenwirken einer zweidimensionalen Rastersteuerung auf Makro-Ebene mit einer gekapselten, kontextabhängigen Komponentenlogik auf Mikro-Ebene löst sich die frühere Fixierung auf feste Viewport-Grenzen zunehmend auf (Carter, 2025; Seyfferth, 2026). Für moderne Design-Systeme eröffnet diese technologische Reife die Möglichkeit, langlebige, performante und barrierefreie Benutzeroberflächen zu schaffen, die strukturelle Komplexität sowohl im HTML-Markup als auch in der CSS-Architektur deutlich reduzieren.

8. Prototypische Umsetzung einer Demonstrationswebseite

Die im Rahmen dieser Arbeit durchgeführte Analyse der CSS-Layouttechnologien (5) sowie das entwickelte Verknüpfungsmodell (7) bilden die theoretische Grundlage, auf deren Basis sich geeignete Technologien für unterschiedliche Layoutanforderungen im Responsive Webdesign auswählen lassen. Um diese Erkenntnisse praxisnah erfahrbar zu machen, wird im Folgenden eine prototypische Demonstrationswebseite vorgestellt, die zunächst mit der Gestaltungssoftware Figma (Figma, 2016) konzipiert und anschließend mithilfe von HTML, CSS und JavaScript in VS-Code umgesetzt sowie in einem Repository auf GitHub bereitgestellt wurde.

8.1. Zielsetzung der Demonstrationswebseite

Ziel der Demonstrationswebseite ist es, die Ergebnisse der vorangegangenen Analyse visuell erfahrbar zu machen und den Zusammenhang zwischen Layoutanforderungen und geeigneten CSS-Technologien greifbar darzustellen.
Ergänzend zu jeder UI-Komponente werden die relevanten HTML- und/oder CSS-Codeausschnitte eingeblendet. Auf diese Weise wird die technische Umsetzung transparent gemacht, sodass sich Unterschiede hinsichtlich Implementierungsaufwand, Lesbarkeit und Wartbarkeit unmittelbar nachvollziehen lassen.

8.2. Designprozess

Die Konzeption der Demonstrationswebseite basiert auf dem in Kapitel 7.1 entwickelten Verknüpfungsmodell, welches typische Layoutanforderungen des Responsive Webdesigns den jeweils am besten geeigneten CSS-Layouttechnologien zuordnet. Die theoretischen Ergebnisse sollen in eine visuell nachvollziehbare und praxisnahe Form überführt werden. Hierzu wird jede identifizierte Layoutanforderung durch eine eigenständige Webkomponente repräsentiert. Die Komponenten orientieren sich an realen Anwendungsszenarien moderner Webseiten und bilden typische Herausforderungen des Responsive Webdesigns nach. Dazu zählen unter anderem Navigationsleisten, Kartenraster, Dashboard-Widgets oder komponentenbasierte Inhaltsbereiche. Durch die Verwendung solcher praxisnahen Beispiele soll die Übertragbarkeit der Analyseergebnisse auf reale Entwicklungsprojekte verdeutlicht werden. Die entwickelten Skizzen für die Demonstrationswebseite, als Teil des Designprozesses sind im Anhang zu finden (siehe C9, C10, C11, C12).

Jede Webkomponente kann durch geeignete Interaktionskomponenten in einer Side-

bar verändert bzw. angepasst werden. Mithilfe dieser Interaktionsschittstellen können die einzelnen Webkomponenten so angepasst werden, dass ihre Auswirkungen im responsive Webdesign und die Vorteile der verwendeten Layoutanforderungen erfahrbar gemacht werden können.

8.2.1. Definition der Webkomponenten

Die Layoutanforderung des **Fluid Layouts** wird auf der Demowebseite exemplarisch anhand eines responsiven, initial dreispaltigen Kartenrasters demonstriert (8.1). Die technische Implementierung basiert auf einer Synthese aus CSS Grid und Media Queries. Über einen interaktiven Schieberegler können die Viewport-Dimensionen dreier repräsentativer Endgeräteklassen simuliert werden. Diese Versuchsumgebung veranschaulicht empirisch, wie das Rastersystem auf unterschiedlichen Zielgeräten skaliert und die einzelnen Layoutkomponenten flüssig an den verfügbaren Raum angepasst werden.

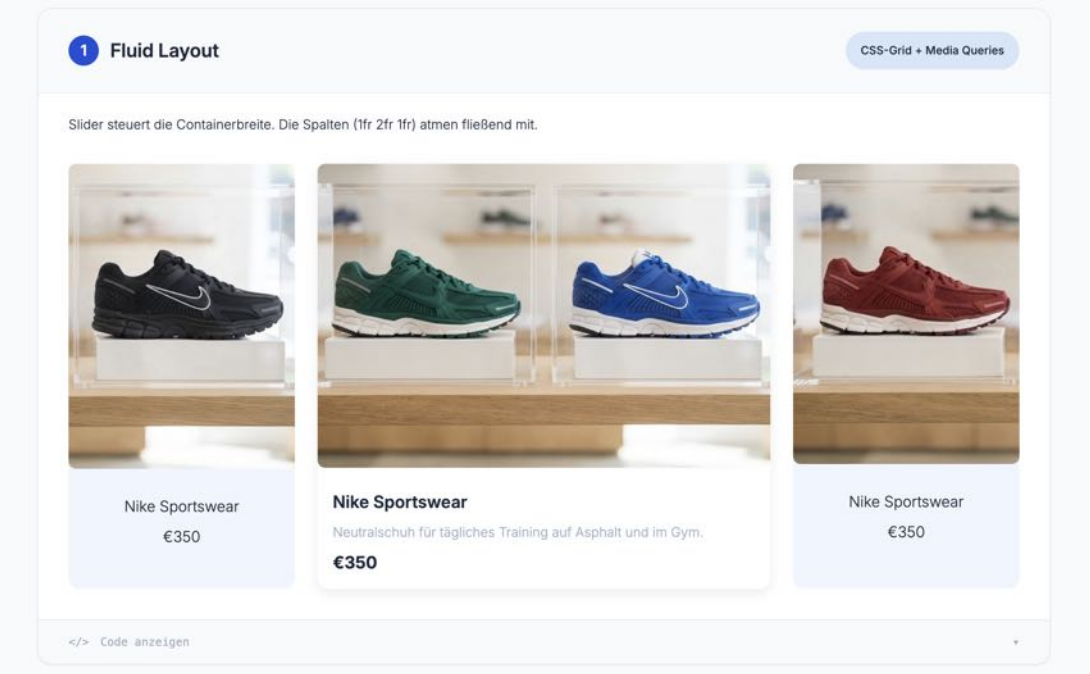


Abbildung 8.1.: Section für die Layoutanforderung ,,Fluid Layout" 6.1

Die Layoutanforderung der **Navigation** wird auf der Demowebseite anhand einer interaktiven Header-Menüleiste (8.2) demonstriert, welche die Kombination aus Flexbox und Media-Queries abbildet. Die Komponente verfügt über zwei komplementäre Interaktionselemente, um die Bewältigung unvorhersehbarer Platzverhältnisse und Inhaltsmengen empirisch zu überprüfen. Zum einen erlaubt ein Schieberegler die Simulation des gesamten Layouts in einer Desktop-, Tablet- und Mobile-Version. Zum anderen ermöglicht eine dynamische Schaltfläche das Einfügen drei neuer Steuerungselemente zur Laufzeit. Dadurch wird visuell nachvollziehbar, wie Flexbox die Menüglieder innerhalb der Navigationsebene elastisch mitatmen und bei unzureichendem horizontalem

Raum fehlerfrei per `flex-wrap: wrap` in die nächste Zeile umbrechen lässt. Dies stellt sicher, dass die Kriterien der Barrierefreiheit und Berührungssicherheit auf Touchscreens ohne Datenverlust oder horizontalen Layout-Overflow gewahrt bleiben.

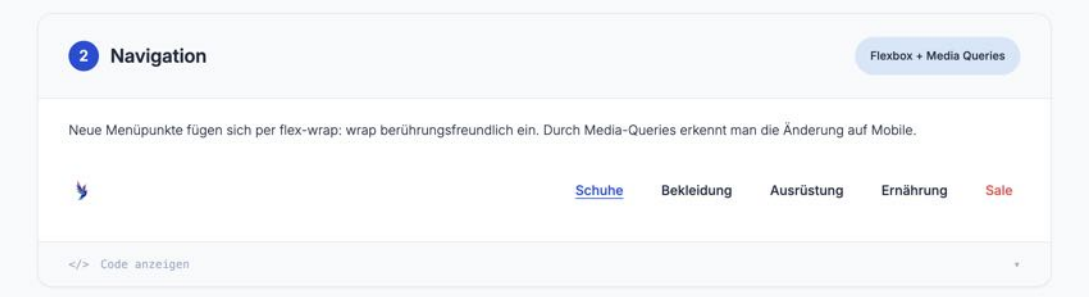


Abbildung 8.2.: Section für die Layoutanforderung „Navigation" 6.2

Die Layoutanforderung der **flexiblen Größenanpassung** wird exemplarisch anhand einer isolierten Produktkarten-Komponente visualisiert (8.3). Die technische Umsetzung basiert auf Container Queries in Kombination mit Flexbox-Eigenschaften zur proportionalen Medienskalierung. Mithilfe einer integrierten Interaktionskomponente lässt sich die Breite des umschließenden Eltern-Containers stufenlos manipulieren. Diese experimentelle Anordnung macht die technologischen Vorteile einer containerbasierten Responsivität direkt nachvollziehbar: Es wird empirisch demonstriert, wie sich die Dimensionen des Containers sowie der eingebetteten Medien vollkommen autark und fließend an den lokal verfügbaren Platz anpassen, ohne dass ein globaler Viewport-Wechsel über Media Queries erzwungen werden muss.

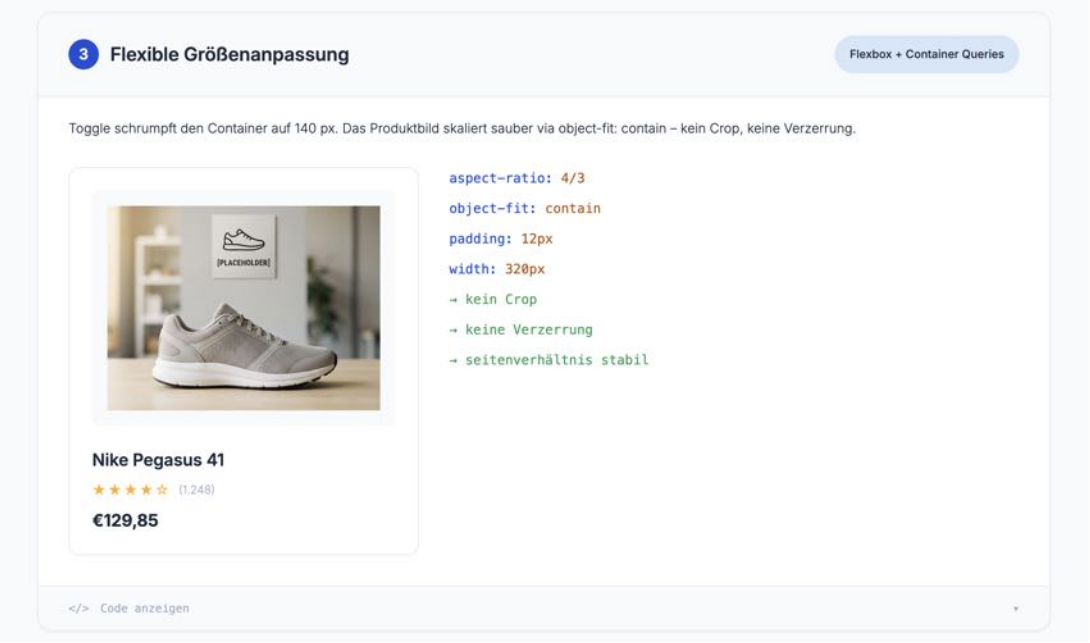


Abbildung 8.3.: Section für die Layoutanforderung „flexiblen Größenanpassung" 6.3

Die Layoutanforderung der **Wiederverwendbarkeit von Komponenten** wird auf der Demowebseite anhand eines modularen Teaser-Moduls demonstriert 8.4. Die technologische Grundlage bildet eine Kombination aus Container Queries zur lokalen Raumüberwachung sowie Flexbox für die interne Strukturanpassung. Anhand eines zugehörigen Code-Beispiels wird veranschaulicht, wie die identische Teaser-Komponente anhand eines definierten Breakpoints von 191px innerhalb ihres direkt umschließenden Containers zwischen einem horizontalen und einem vertikalen Layout umschaltet, ohne dass dabei eine konkrete Einbettungsposition wie Hauptinhaltsbereich, Seitenleiste, Grid-Zelle oder modales Fenster vorausgesetzt werden muss. Dieses Vorgehen veranschaulicht, dass das Modul seine visuelle Darstellung ausschließlich auf Basis des lokal verfügbaren Platzes des umschließenden Eltern-Elements anpasst, wodurch eine kontextunabhängige Kapselung ohne visuelle Defekte realisiert wird.

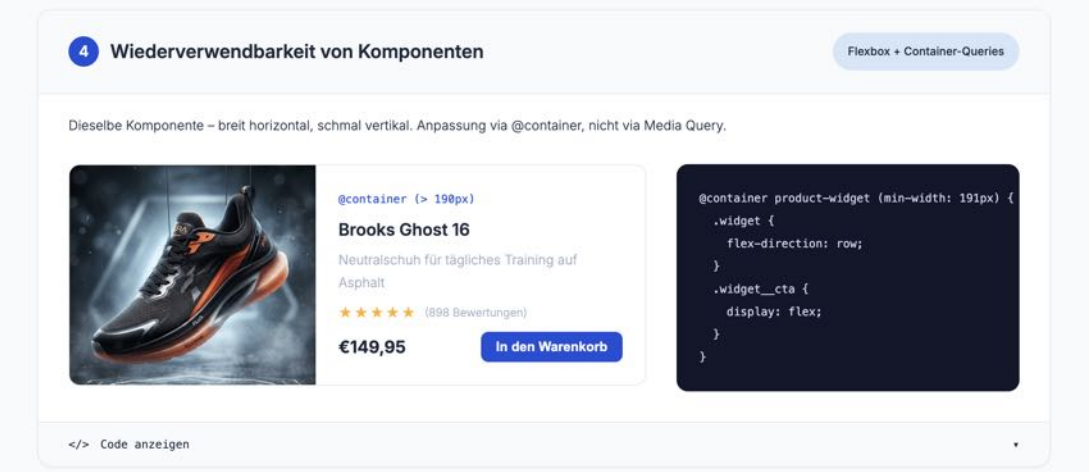


Abbildung 8.4.: Section für die Layoutanforderung „Wiederverwendbarkeit von Komponenten" 6.4

Die Layoutanforderung bezüglich **dynamischer Inhaltsmengen** wird auf der Demowebseite anhand einer Reihe mehrspaltiger Produktkarten demonstriert, die mit ungleichen Textlängen befüllt sind, um die Unvorhersehbarkeit von API- oder Datenbankdaten zur Designzeit zu simulieren 8.5. Die technologische Umsetzung basiert auf einer Synthese aus einem übergeordneten CSS Grid und dem darin aktivierten `CSS Subgrid`. Als interaktive Komponente wird ein dynamischer Text-Umschalter integriert, mit dem sich die Textlängen der Beschreibungen zur Laufzeit manipulieren lassen. Diese Versuchsanordnung veranschaulicht empirisch, wie das übergeordnete Grid die globale Spaltenverteilung übernimmt, während das Subgrid die Höhen der verschachtelten Kindelemente über Komponentengrenzen hinweg synchron an den jeweils längsten Inhalt anpasst. Dadurch wird nachgewiesen, dass visuell zusammengehörige Informationen und Funktionstasten selbst bei variierenden Datenmengen stets auf einer einheitlichen, horizontalen Fluchtlinie ausgerichtet bleiben.

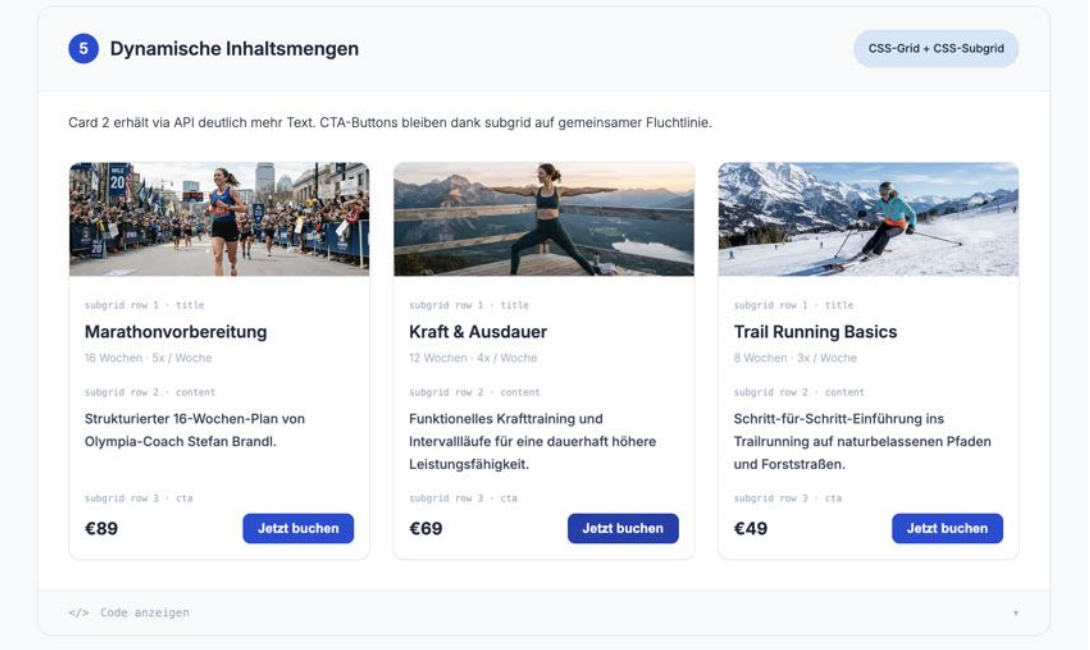


Abbildung 8.5.: Section für die Layoutanforderung ,,Dynamische Inhaltsmengen" 6.5

Die Layoutanforderung der **Layout-Unabhängigkeit von der HTML-Struktur** wird auf der Demowebseite anhand eines flexiblen Magazin- bzw. Dashboard-Layouts demonstriert 8.6. Die technische Umsetzung basiert auf der Kombination von CSS Grid und Media Queries, um die simultane Nutzung durch sehende und sehbehinderte Endanwender zu simulieren. Als interaktive Komponente wird ein Viewport-Umschalter integriert, welcher zusätzlich die zugrundeliegende, lineare DOM-Quelltextreihenfolge visualisiert. Diese Versuchsanordnung veranschaulicht empirisch, wie die logische und barrierefreie HTML-Struktur für Screenreader unangetastet im Quellcode verankert bleibt, während CSS Grid die visuelle Neuanordnung der UI-Elemente im zweidimensionalen Raum übernimmt. Die Media Queries dienen dabei als Steuerinstanz, um diese drastischen Layout-Verschiebungen exakt auf Basis des simulierten Endgeräts auszuführen.

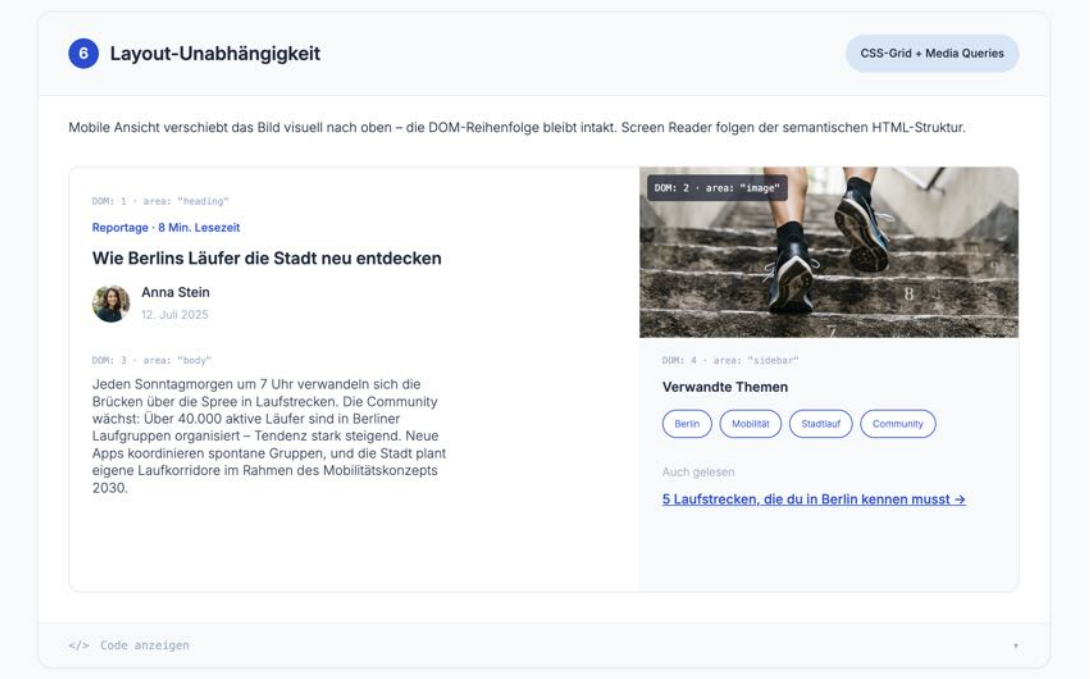


Abbildung 8.6.: Section für die Layoutanforderung ,,Layout-Unabhängigkeit von der HTML-Struktur" 6.6

Die Layoutanforderung der **Skalierbarkeit und Wartbarkeit** wird auf der Demowebseite anhand eines komplexen Account-Einstellungsmenüs veranschaulicht 8.7. Die technologische Architektur basiert auf einer Kombination aus CSS Grid und CSS Subgrid zur strukturellen Begradigung. Als interaktive Komponente wird ein funktioneller „HTML-Wrapper-Toggle" implementiert, der es dem Nutzer erlaubt, zwischen zwei Markup-Zuständen umzuschalten: Einem Zustand mit tief verschachtelten, rein visuellen Hilfs-Wrappern und einer optimierten, semantisch flachen HTML-Struktur. Diese Versuchsanordnung demonstriert im Sinne der Qualitätskriterien nach ISO/IEC 25010 (ISO, 2011) empirisch, wie das übergeordnete Rastersystem dank `subgrid` tiefer liegende Kindelemente direkt an seine Achsenstrukturen bindet. Dadurch wird nachgewiesen, dass redundanzfreie, schlanke Quelltexte ohne unvorhergesehene visuelle Seiteneffekte wartbar und erweiterbar bleiben.

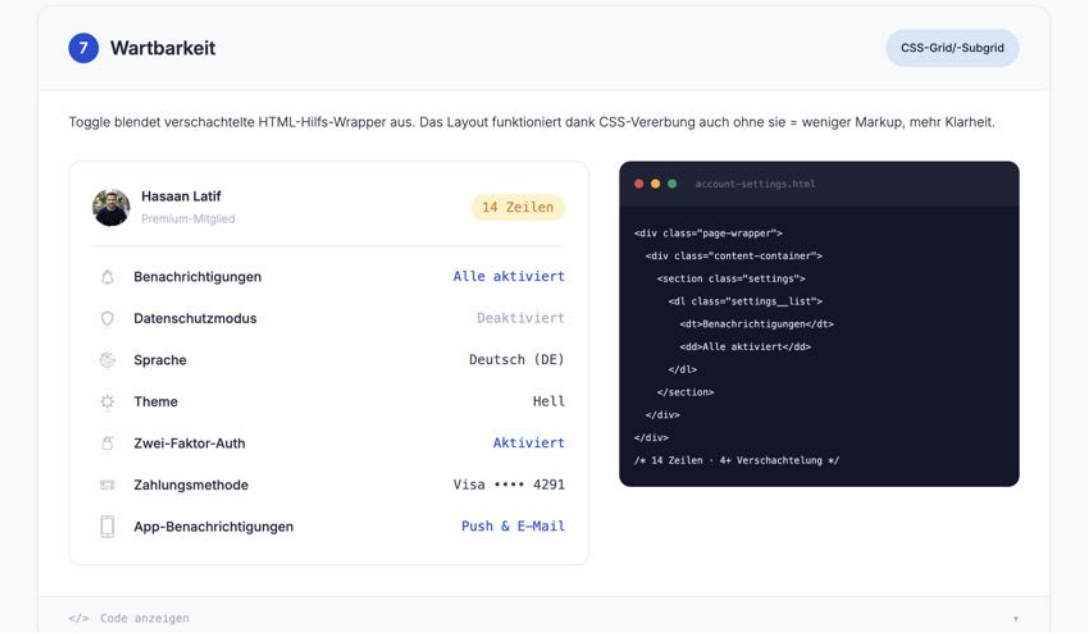


Abbildung 8.7.: Section für die Layoutanforderung „Skalierbarkeit und Wartbarkeit" 6.7

8.2.2. Darstellung der Responsivität und Code-Struktur

Unterhalb jeder Webkomponente werden die zugehörigen HTML- und CSS-Codeausschnitte eingeblendet, wie exemplarisch in Abbildung 8.8 zu sehen ist. Auf diese Weise lässt sich die technische Implementierung unmittelbar nachvollziehen.

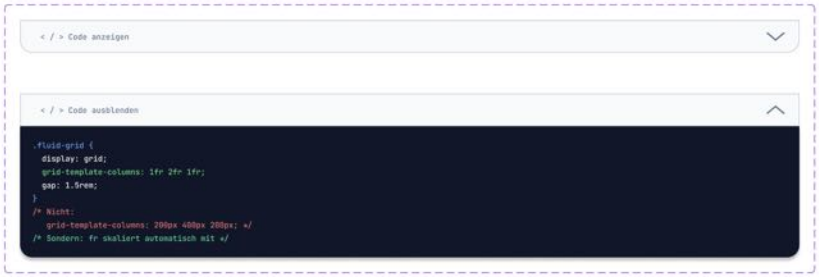


Abbildung 8.8.: Beispiel Code-Drop-Down von Section 1 aus dem Figma-Design

Über die Sidebar mit ihren Interaktionskomponenten lassen sich die Eigenschaften der verwendeten CSS-Layouttechnologien spielerisch und live testen. Nutzerinnen und Nutzer der Demowebseite können dabei selbst festlegen, welche Layoutanforderung sie in der responsiven Ansicht betrachten möchten. Die einzelnen Komponenten der Sidebar wurden zunächst separat entworfen und anschließend in einer Gesamtansicht in Figma zusammengeführt, wie in Abbildung 8.9 dargestellt.

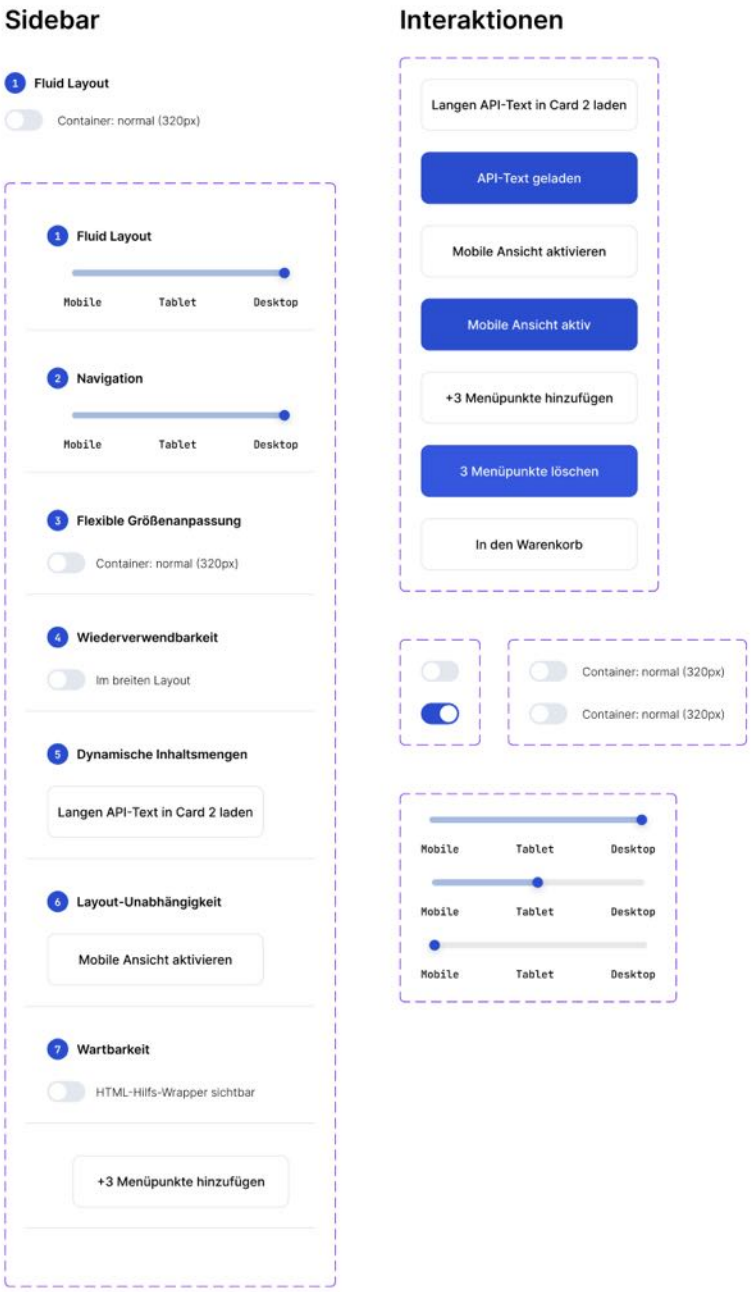


Abbildung 8.9.: Sidebar-Komponenten und die Zusammengeführte Sidebar in Figma

8.2.3. Figma Umsetzung

Auf Grundlage sämtlicher entworfener Komponenten ließ sich schließlich ein Webseiten-Mockup zusammenführen, der in Abbildung 8.11 dargestellt ist.

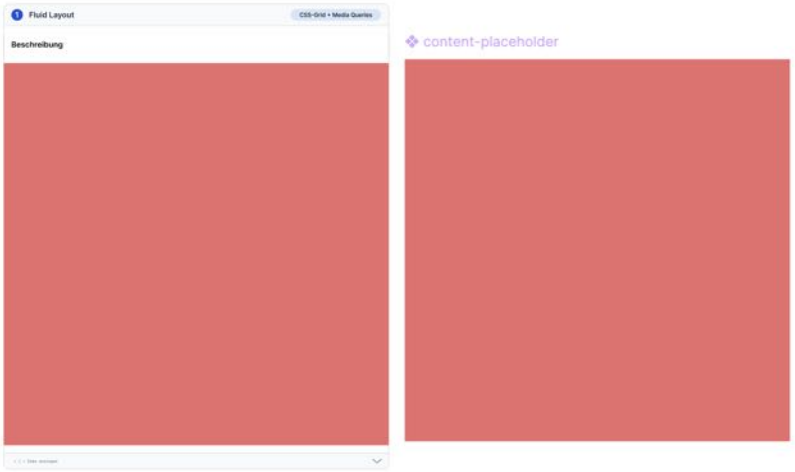


Abbildung 8.10.: Base-Section Komponente für einheitliche Sections

Im Anschluss wurden die einzelnen Sections mithilfe von Auto-Layouts in Figma entworfen und, nach Umwandlung in Komponenten, in das Base-Layout eingebettet. Die Designs der einzelnen Sections sind im Anhang hinterlegt (siehe C13). Die im Figma-File verwendeten Bilder wurden mithilfe der Figma-AI erzeugt, welche ein Rechteck anhand eines Prompts mit einem passenden Bild füllt.
Auf Grundlage sämtlicher entworfener Komponenten ließ sich schließlich ein Webseiten-Mockup zusammenführen, der in Abbildung 8.11 dargestellt ist.

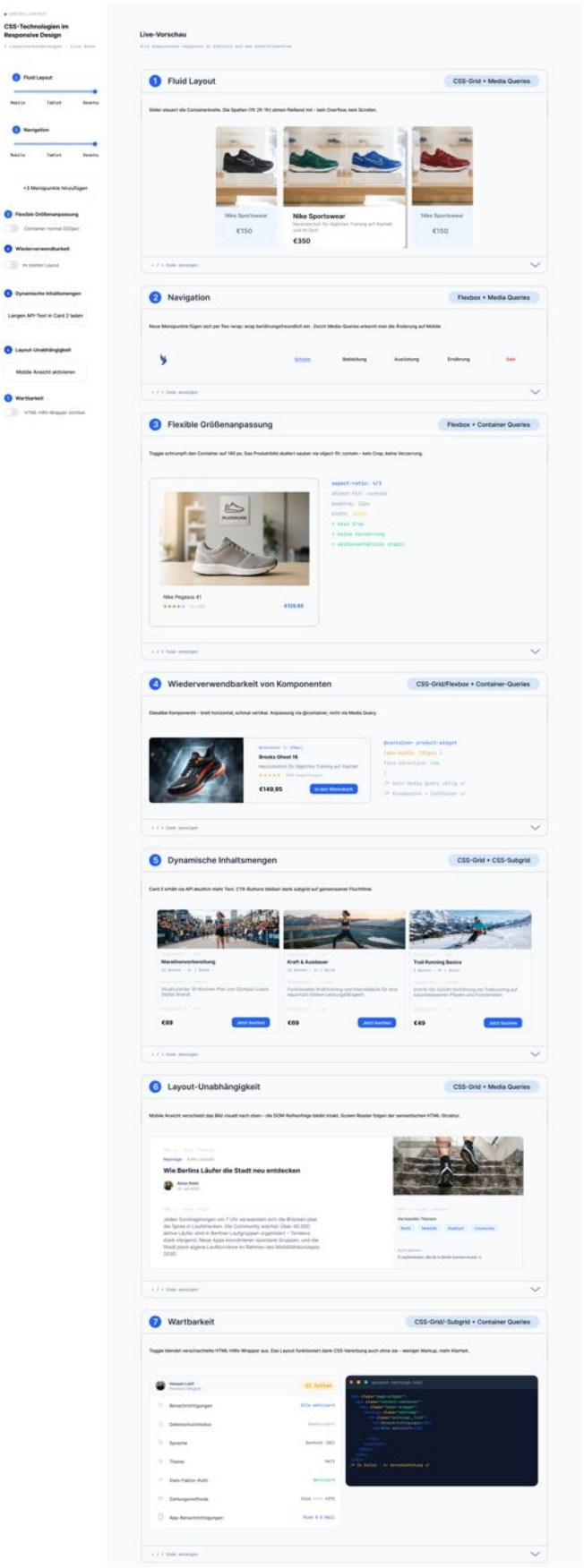


Abbildung 8.11.: Figma Webseiten-Mockup

8.2.4. Programmieren des Figma-Designs

Nachdem das Figma-Design erstellt wurde, konnte dieses in eine funktionsfähige Webseite überführt werden. Der technische Aufbau der Demonstrationswebseite basiert auf den Kerntechnologien HTML5, CSS3 sowie JavaScript, bewusst ohne den Einsatz eines Frontend-Frameworks. Diese Entscheidung wurde getroffen, um die in Kapitel 5 untersuchten CSS-Layouttechnologien möglichst unverfälscht und ohne zusätzliche Abstraktionsebenen durch ein Framework zu demonstrieren. Der gesamte Code wurde im Quelltext-Editor Visual Studio Code (vscode, 2026) entwickelt und anschließend auf GitHub (github, 2026) hochgeladen. Das GitHub-Repository kann von jedem Interessierten geklont und lokal auf dem eigenen Rechner ausgeführt werden. Derzeit besteht keine öffentlich zugängliche Internetadresse, unter der die Webseite global abrufbar ist.

Mithilfe der KI-Plattform Claude (2026) wurde zunächst das HTML-Grundgerüst der `index.html` erstellt, da sich auf diesem Weg die grundlegende Dokumentstruktur sowie die semantische Aufteilung der einzelnen Sections zeitlich effizient umsetzen ließen. Die Erstellung erfolgte iterativ: Das erstellte Figma-Design wurde als Datei exportiert und anschließend in dem prompt mitgeschickt, sodass die KI wusste, welches Grundgerüst aufgebaut werden sollte. Außerdem wurde in dem Prompt erwähnt, das bereits ID´s für die einzelnen Bestandteile des HTML´s definiert werden sollten, um diese im späteren Projektverlauf im Javascript auffangen zu können. Der auf diese Weise generierte HTML-Code wurde im Anschluss manuell überprüft, semantisch bereinigt und an die tatsächlichen Anforderungen der jeweiligen CSS-Layouttechnologie angepasst, um sicherzustellen, dass die Dokumentstruktur mit den in Kapitel 5 untersuchten Eigenschaften der Technologien, insbesondere im Hinblick auf Barrierefreiheit und logische Reihenfolge, übereinstimmt. Der KI-Einsatz beschleunigte somit vor allem die initiale Grundgerüst-Erstellung; die konzeptionelle Verknüpfung zwischen Layoutanforderung und konkreter Technologie sowie die feingranulare CSS-Umsetzung erfolgten hingegen eigenständig, da hierfür ein tieferes Verständnis der spezifischen Analyseergebnisse notwendig war.

Im Anschluss an die HTML-Struktur war es möglich, die CSS-Dateien für die einzelnen Designs der Webseite zu programmieren. Für jede Layoutanforderung wurde dabei eine eigene, klar abgegrenzte CSS-Datei angelegt, um die in Kapitel 7.2 beschriebene Kapselung der Komponenten auch im Quellcode selbst nachvollziehbar abzubilden und die Zuordnung zwischen Technologie und Anforderung transparent zu halten.

Das Figma-Design wurde während der Programmierung an einigen Stellen abgewandelt, was jedoch nicht die Kernfunktionalitäten der Demonstrationswebseite beeinträchtigt, sondern lediglich aus gestalterischer Sicht geringfügige Abweichungen mit sich bringt, etwa bei Abständen, Farbnuancen oder der konkreten Ausprägung einzelner UI-Elemente.

Durch den Einsatz von JavaScript war es zudem möglich, Interaktionen zwischen der Sidebar und den jeweiligen Webkomponenten zu erzeugen, um deren Vorteile beziehungsweise Potenziale unmittelbar erfahrbar zu machen. Die implementierten Interak-

tionselemente umfassen unter anderem Schieberegler zur Simulation unterschiedlicher Viewport-Breiten beziehungsweise Container-Breiten, Toggle-Schaltflächen zum Umschalten zwischen Desktop-, Tablet- und Mobile-Ansichten sowie dynamische Buttons, über die zur Laufzeit zusätzliche Inhalte, wie neue Navigationspunkte oder verlängerte Textmengen, in die jeweilige Komponente eingefügt werden können. Technisch wurden diese Interaktionen über einfache DOM-Manipulationen sowie Event-Listener realisiert, welche auf Nutzereingaben, wie das Verschieben eines Sliders oder das Betätigen eines Buttons, reagieren und daraufhin gezielt Inline-Styles oder CSS-Klassen der betroffenen Elemente verändern.

Wichtig ist hierbei die Unterscheidung zwischen zwei unterschiedlichen Funktionsebenen des Codes. Zum einen dient JavaScript ausschließlich der Simulation von Nutzungsszenarien innerhalb der Demonstrationswebseite selbst, beispielsweise um ohne tatsächliche Größenänderung des Browserfensters zu veranschaulichen, wie sich eine Komponente bei unterschiedlichen Viewport- oder Containerbreiten verhält. Diese Interaktionsebene ist somit ein rein didaktisches Hilfsmittel, das ausschließlich für die Präsentation im Rahmen dieser Arbeit erforderlich ist. Zum anderen wird unterhalb jeder Webkomponente der zugehörige HTML- und CSS-Codeausschnitt angezeigt, welcher der eigentlichen, zu untersuchenden CSS-Layouttechnologie entspricht. Dieser angezeigte Code ist bewusst so gestaltet, dass er unabhängig von der JavaScript-gesteuerten Simulationsumgebung funktioniert: Die responsive Anpassung der Komponenten, etwa das Umbrechen einer Flexbox-Navigation, das Skalieren eines CSS-Grids oder das kontextabhängige Verhalten einer per Container-Query gesteuerten Karte, wird in Produktivumgebungen ausschließlich durch den Browser und die native CSS-Engine ausgelöst, sobald sich der tatsächliche Viewport oder die tatsächliche Containergröße verändert. Der dargestellte CSS-Code lässt sich somit direkt und ohne jegliche JavaScript-Abhängigkeit in reale Webprojekte übernehmen, während die JavaScript-Logik der Demonstrationswebseite lediglich dazu dient, verschiedene Größenzustände simulierbar zu machen, ohne dass Nutzern hierfür manuell das Browserfenster verändern oder unterschiedliche Endgeräte verwenden müssen. Dadurch bleibt die zentrale Aussage der Arbeit gewahrt, dass moderne CSS-Layouttechnologien responsive Anpassungen nativ und ohne den Einsatz von JavaScript realisieren können.

9. Fazit

9.1. Zusammenfassung der Ergebnisse

Im Rahmen dieses Praxisprojekts wurden die CSS-Layouttechnologien Flexbox, CSS Grid und CSS Subgrid sowie die responsiven Steuerungsprinzipien Media Queries und Container Queries anhand eines einheitlichen, sechs- beziehungsweise fünfdimensionalen Analysemodells systematisch untersucht. Die Analyse zeigte, dass jede Technologie über spezifische gestalterische und technische Stärken verfügt: Flexbox eignet sich besonders für eindimensionale, komponentenorientierte Layoutmuster, CSS Grid für zweidimensionale Makro-Layouts, CSS Subgrid für die synchrone Ausrichtung verschachtelter Komponenten, Media Queries für globale, viewportbasierte Layout-Wechsel und Container Queries für kontextsensitive, kapselungsfähige UI-Komponenten.

Aufbauend auf einem methodischen Rahmen aus Human-Centered Design, ISO/IEC 25064 und der Rupp'schen Satzschablone wurden sieben konkrete Layoutanforderungen aus typischen Nutzungskontexten des responsiven Webdesigns hergeleitet. Das daraus entwickelte Verknüpfungsmodell zeigt, dass sich die untersuchten Layoutanforderungen selten durch eine einzelne Technologie, sondern meist durch das gezielte Zusammenspiel mehrerer Technologien lösen lassen. Auf Grundlage dieser Erkenntnisse wurde abschließend eine prototypische Demonstrationswebseite entwickelt, die die theoretischen Ergebnisse anhand von sieben interaktiven Webkomponenten praktisch nachvollziehbar macht.

9.2. Beantwortung der Forschungsfrage

Die eingangs gestellte Forschungsfrage lautete, welche gestalterischen und technischen Potenziale CSS-Layouttechnologien im Responsive Webdesign bieten und für welche Layoutanforderungen sich welche Technologie am besten eignet.

Die Analyse zeigt, dass das gestalterische Potenzial der untersuchten Technologien vor allem in der Möglichkeit liegt, Layouts flüssig, kontextsensitiv und ohne feste Pixelwerte zu gestalten, wodurch Webseiten auf beliebigen Bildschirmgrößen konsistent nutzbar bleiben. Technisch zeichnen sich die Technologien insbesondere durch eine Reduktion von JavaScript-Abhängigkeiten, eine geringere DOM-Komplexität sowie eine native, performante Verarbeitung durch die Rendering-Engines der Browser aus.

In Bezug auf die konkrete Eignung für einzelne Layoutanforderungen lässt sich festhalten: Für globale, seitenweite Strukturen (Fluid Layout) eignet sich die Kombination aus CSS Grid und Media Queries am besten. Für eindimensionale, berührungsfreundliche

Komponenten wie Navigationen ist die Kombination aus Flexbox und Media Queries am geeignetsten. Für kontextunabhängige, wiederverwendbare Mikro-Komponenten (flexible Größenanpassung, Wiederverwendbarkeit von Komponenten) bietet das Zusammenspiel aus Flexbox beziehungsweise Container Queries den größten Nutzen. Für dynamische, variierende Inhaltsmengen und die strukturelle Wartbarkeit tief verschachtelter Komponenten stellt schließlich die Kombination aus CSS Grid und CSS Subgrid die geeignetste Lösung dar. Damit wird deutlich, dass die Forschungsfrage nicht durch die Empfehlung einer einzelnen „besten“ Technologie, sondern durch ein kontextabhängiges Entscheidungsmodell zu beantworten ist, das Makro- und Mikro-Ebene des Layouts bewusst trennt.

9.3. Reflexion der Analyse und Methodik

Die im Rahmen dieser Arbeit entwickelte Demonstrationswebseite dient ausschließlich dazu, die Ergebnisse der theoretischen Analyse und des Verknüpfungsmodells anschaulich zu vermitteln, und ist entsprechend in ihrem Umfang begrenzt. Zunächst ist zu beachten, dass der auf der Webseite eingeblendete Code-Dropdown aus didaktischen Gründen nicht vollständig mit dem tatsächlichen Quellcode im zugehörigen GitHub-Repository übereinstimmt. Während der angezeigte Code-Ausschnitt bewusst auf die jeweils relevante CSS-Layouttechnologie reduziert wurde, um deren Funktionsweise isoliert nachvollziehbar zu machen, enthält der produktive Code zusätzliche Selektoren, Hilfsklassen und JavaScript-Logik, die für die Simulationsumgebung der Webseite notwendig sind, jedoch keinen Mehrwert für das Verständnis der eigentlichen Layouttechnologie liefern.

Des Weiteren handelt es sich bei der entwickelten Webseite lediglich um einen Prototyp. Eine gezielte Optimierung für mobile Endgeräte, etwa im Sinne echter Breakpoint-Strategien oder performanter Ladezeiten auf Smartphones, wurde bewusst nicht vorgenommen, da die Simulationskomponenten der Sidebar diese Endgeräteklassen bereits interaktiv nachbilden. Ebenso wurden keine vertiefenden Usability-Anpassungen, etwa im Hinblick auf Ladezeiten, Interaktionsdesign oder Nutzerführung, vorgenommen. Die Demonstrationswebseite verfolgt somit ausdrücklich nicht das Ziel, eine produktionsreife Anwendung darzustellen, sondern soll die identifizierten Vorteile der untersuchten CSS-Layouttechnologien für ausgewählte Layoutanforderungen grob und nachvollziehbar visualisieren.

Im Verlauf der Recherche wurden zudem zwei Onlineartikel identifiziert, die inhaltlich und strukturell Überschneidungen mit dieser Arbeit aufweisen: „CSS Grid Mastery: Real Layouts Without Hacks“(Rupasinghe, 2026) sowie „CSS Grid and Flexbox: Complete Layout Reference“(Team, 2026). Beide Beiträge behandeln, ähnlich wie die vorliegende Arbeit, Flexbox, CSS Grid, Subgrid und Container Queries und stellen ebenfalls Entscheidungshilfen („Grid vs. Flexbox“) sowie Code-Patterns für gängige Layoutprobleme bereit.

Bei genauerer Betrachtung zeigt sich jedoch ein grundlegender Unterschied im me-

thodischen Anspruch. Beide Artikel sind als praxisorientierte, werbe- beziehungsweise SEO-getriebene Fachblogs konzipiert: Sie präsentieren isolierte Code-Snippets und eine informelle, aus Erfahrungswerten abgeleitete Entscheidungstabelle, ohne eine systematische, nachvollziehbare Herleitung, wie diese Empfehlungen zustande kommen. Die vorliegende Arbeit unterscheidet sich davon in mehreren Punkten:

Statt punktueller Praxistipps wird jede Technologie anhand derselben sechs, beziehungsweise bei den Steuerungsprinzipien fünf, klar definierten Bewertungsdimensionen untersucht, wodurch eine objektive Vergleichbarkeit der Technologien untereinander entsteht.
Außerdem werden die Layoutanforderungen nicht als gesetzt vorausgesetzt, sondern mittels eines Human-Centered-Design-Prozesses sowie standardisierter Requirements-Engineering-Werkzeuge nachvollziehbar aus konkreten Nutzungskontexten abgeleitet.
Desweiteren liefert diese Arbeit anstelle einer allgemeinen „Grid vs. Flexbox“-Tabelle eine begründete, nach Layoutanforderung gegliederte Entscheidungsmatrix mit Punktebewertung und expliziter technologischer Begründung.
Während Barrierefreiheit in den beiden Artikeln allenfalls am Rande als Stolperfalle erwähnt wird, ist sie in dieser Arbeit für jede Technologie eine eigenständige, gleichrangig behandelte Analysedimension.
Abschließend überführt diese Arbeit statt statischer Codebeispiele die Analyseergebnisse in eine funktionsfähige, interaktive Demonstrationswebseite, an der sich die hergeleiteten Zusammenhänge unmittelbar erfahrbar nachvollziehen lassen.

Das Alleinstellungsmerkmal dieser Arbeit liegt somit weniger in der Auswahl der behandelten Technologien selbst, sondern in der wissenschaftlich fundierten, methodisch nachvollziehbaren Herleitung und Verknüpfung von Nutzeranforderungen und Technologieeigenschaften sowie deren praktischer Demonstration.

10. Ausblick

Die in dieser Arbeit entwickelte Analyse- und Verknüpfungsmethodik lässt sich in mehrere Richtungen weiterführen. Zum einen könnte das entwickelte Entscheidungsmodell in zukünftigen Projekten als Grundlage für ein automatisiertes Linting- oder Empfehlungstool dienen, das Entwicklern während der Implementierung basierend auf erkannten Layoutmustern direkt die passende CSS-Layouttechnologie vorschlägt. Zum anderen wäre eine empirische Validierung der Ergebnisse sinnvoll, etwa durch quantitative Performance-Messungen der einzelnen Technologien in realen Produktionsumgebungen sowie durch Usability-Tests mit echten Screenreader-Nutzerinnen und -Nutzern, um die in Kapitel 5 diskutierten Barrierefreiheitsrisiken empirisch zu überprüfen.

Schließlich könnte die entwickelte Demonstrationswebseite in einer weiterführenden Arbeit zu einer produktionsreifen, mobil optimierten Referenzanwendung ausgebaut werden, die als Lehrmittel in der Ausbildung von Webentwicklerinnen und Webentwicklern oder als Grundlage für unternehmensinterne Design-System-Dokumentationen dient. Insgesamt liefert diese Arbeit damit sowohl eine theoretische Grundlage als auch einen praktischen Ausgangspunkt für weiterführende Forschung im Bereich der responsiven Webentwicklung.

Abbildungsverzeichnis

4.1. Vorlage zur Analyse der CSS-Layouttechnologien 10
4.2. Vorlage zur Analyse der responsiven Steuerungsprinzipien 12
4.3. Syntaxstruktur der Erfordernis-Satzschablone (User Experience Qualifications Board (UXQB), 2023) 13
4.4. Schablone für funktionale Anforderungen ohne Bedingung (Rupp u. die SOPHISTen, 2020) 13
4.5. Vorlage Entscheidungsmatrix 14

5.1. Abbildung des Flex-Models aus der MDN-Dokumentation (MDN Web Docs, 2025a) 16
5.2. DevTool-Option von Chrome zur Einstellung von Flexbox-Eigenschaften (Chrome for Developers, 2024) 17
5.3. Diagramm zur Veranschaulichung der Browserunterstützung von Flexbox (Alexis Deveria, 2026a) 18
5.4. DevTool-Option von Chrome zur Anzeige der verwendeten Grids am Beispiel von der Webkomponente der eigenen Ausarbeitung 8 21
5.5. DevTool-Layouteditor von Chrome zur Echtzeitänderung von CSS-Grid-Eigenschaften (Chrome for developers, 2021) 21
5.6. Visuelle Inspektion eines CSS Subgrids in den Firefox DevTools inklusive Badge-Kennzeichnung und Overlay-Linien (Mozilla Foundation & Mozilla Corporation, 2026) 24
5.7. Diagramm zur Veranschaulichung der Browserunterstützung von Subgrid (Alexis Deveria, 2026b) 25

6.1. Darstellung des begrenzten zu erreichenden Screens eines Smartphones (Hoober, 2013) 32

8.1. Section für die Layoutanforderung ,,Fluid Layout” 6.1 41
8.2. Section für die Layoutanforderung ,,Navigation” 6.2 42
8.3. Section für die Layoutanforderung ,,flexiblen Größenanpassung” 6.3 . . 42
8.4. Section für die Layoutanforderung ,,Wiederverwendbarkeit von Komponenten” 6.4 43
8.5. Section für die Layoutanforderung ,,Dynamische Inhaltsmengen” 6.5 . . 44
8.6. Section für die Layoutanforderung ,,Layout-Unabhängigkeit von der HTML-Struktur” 6.6 45
8.7. Section für die Layoutanforderung ,,Skalierbarkeit und Wartbarkeit” 6.7 46
8.8. Beispiel Code-Drop-Down von Section 1 aus dem Figma-Design 46
8.9. Sidebar-Komponenten und die Zusammengeführte Sidebar in Figma . . 47
8.10. Base-Section Komponente für einheitliche Sections 48
8.11. Figma Webseiten-Mockup 49

A1. „Tabelle zur Herleitung der Layoutanforderungen" 65
B2. „Entscheidungsmatrix: Fluid Layout" 66
B3. „Entscheidungsmatrix: Navigation" 66
B4. „Entscheidungsmatrix: Flexible Größenanpassung" 67
B5. „Entscheidungsmatrix: Wiederverwendbarkeit von Komponenten" . . . 68
B6. „Entscheidungsmatrix: dynamische Inhaltsmengen" 68
B7. „Entscheidungsmatrix: Layout-Unabhängigkeit von der HTML-Struktur" 69
B8. „Entscheidungsmatrix: Skalierbarkeit und Wartbarkeit" 69
C9. „Erste Skizze während des Designprozesses" 70
C10. „Zweite Skizze während des Designprozesses" 71
C11. „Dritte Skizze während des Designprozesses" 72
C12. „Finale Skizzen während des Designprozesses" 73
C13. „Design der einzelnen Sections zum Einfügen in das Base-Layout" . . . 74

Literaturverzeichnis

[ISO 2011] ; International Organization for Standardization (Veranst.): *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. Genf, 2011

[ISO 2013] ; International Organization for Standardization (Veranst.): *Systems and software engineering – Software product Quality Requirements and Evaluation (SQuaRE) – Common Industry Format (CIF) for usability: User needs report*. Genf, 2013

[Alexis Deveria 2026a] Alexis Deveria: *Can I use... Support tables for CSS Flexible Box Layout*. `https://caniuse.com/flexbox`. Version: 2026. – Letzter Zugriff: 19 Juni 2026

[Alexis Deveria 2026b] Alexis Deveria: *Can I use... Support tables for CSS Subgrid Layout*. `https://caniuse.com/?search=CSS+Subgrid`. Version: 2026. – Letzter Zugriff: 27 Juli 2026

[Andrew 2017] Andrew, Rachel: *The New CSS Layout*. New York : A Book Apart, 2017. – ISBN 978–1–937557–62–1

[Bhanarkar u. a. 2023] Bhanarkar, Noorkaran ; Paul, Aditi ; Mehta, Ashima: Responsive web design and its impact on user experience. In: *International Journal of Advanced Research in Science, Communication and Technology* 3 (2023), Nr. 4, S. 50–55

[Carter 2025] Carter, Krish: *Mastering CSS Container Queries: Break Free from Viewport Tyranny*. `https://dev.to/krish_kakadiya_5f0eaf6342/mastering-css-container-queries-break-free-from-viewport-tyranny-2lgo?` Version: 2025. – Letzter Zugriff: 09 Mai 2026

[Chrome for developers 2021] Chrome for developers: *CSS-Rasterlayouts prüfen*. `https://developer.chrome.com/docs/devtools/css/grid?hl=de`. Version: 2021. – Letzter Zugriff: 23 Juni 2026

[Chrome for Developers 2024] Chrome for Developers: *Inspect CSS Grid and Flexbox layouts – Inspect Flexbox*. `https://developer.chrome.com/docs/devtools/css/flexbox`. Version: 2024. – Letzter Zugriff: 27. Juli 2026

[Claude 2026] *Claude*. 2026. – `https://claude.com/de`

[Coyier 2026] Coyier, Chris: *A Complete CSS Flexbox Layout Guide*. `https://css-tricks.com/snippets/css/a-guide-to-flexbox/`, 2026. – Letzter Zugriff: 17 Juni 2026

[Department 2025] Department, Statista R.: *Smartphone-Nutzung in Deutschland – Daten Fakten*. https://de.statista.com/themen/6137/smartphone-nutzung-in-deutschland. Version: 2025. – Letzter Zugriff: 09 Mai 2026

[Figma 2016] *Figma*. 2016. – https://www.figma.com/de/design/?utm_source=google&utm_medium=cpc&utm_campaign=18085381823&utm_term=design-software&utm_content=617062504255&utm_adgroup=143044330769&gclid=CjwKCAiA1fqrBhA1EiwAMU5m_zx3TUWuZeANhu840AjZ3fCP0lTLHK3P6eOjUoJxvII2z0ryQKi6JhoCBgwQAvD_BwE

[Firtman 2010] Firtman, Maximiliano: *Programming the mobile web*. Ö'Reilly Media, Inc.", 2010

[Frain 2015] Frain, Ben: *Responsive web design with HTML5 and CSS3*. Packt Publishing Ltd, 2015

[Frost 2016] Frost, Brad: *Atomic Design*. Eigenverlag, 2016 https://atomicdesign.bradfrost.com/

[Frost 2022] Frost, Brad: *Layout & Grid in Design Systems*. https://bradfrost.com/blog/post/layout-grid-in-design-systems/. Version: November 2022. – Letzter Zugriff: Juni 2026

[Garrett 2022] Garrett, Jesse J.: *The elements of user experience*. , 2022

[github 2026] *GitHub Repo*. 2026. – https://github.com/Anna130903/praxisprojekt-css-potenziale

[Graham 2024] Graham, Geoff: *CSS Container Queries*. https://css-tricks.com/css-container-queries/. Version: 2024

[Gupta 2025] Gupta, Satyam: *: Container Queries in CSS: A Complete Guide to the Future of Responsive Design*. https://dev.to/satyam_gupta_0d1ff2152dcc/-container-queries-in-css-a-complete-guide-to-the-future-of-responsive-design-24om Version: 2025. – Letzter Zugriff: 24 Mai 2026

[Haim 2026] Haim, Itamar: *CSS-Flex-Eigenschaft: Ein umfassender Leitfaden zu Flexbox in CSS*. https://elementor.com/blog/de/css-flex-eigenschaft-ein-umfassender-leitfaden-zu-flexbox-in-css/. Version: 2026. – Letzter Zugriff: 17 Juni 2026

[Hayward 2020] Hayward, Nathan: *Advanced CSS Layouts with Flexbox for Component Frameworks*. Berkeley : Apress, 2020. http://dx.doi.org/10.1007/978-1-4842-6233-7. http://dx.doi.org/10.1007/978-1-4842-6233-7

[Hazaël-Massieux 2005] Hazaël-Massieux, Dominique: *Tabellenloses Webseitendesign*. https://www.w3.org/2005/12/tableless-de/. Version: 2005. – Letzter Zugriff: 03 Mai 2026

[Hellbusch 2021] Hellbusch, Jan: *Layout-Tabellen sind wie Briefe in Excel*. https://www.barrierefreies-webdesign.de/knowhow/layout-tabellen/. Version: 2021. – Letzter Zugriff: 04 Mai 2026

[Hoffmann 2017] Hoffmann, Jay: *Tables for Layout? Absurd.* https://thehistoryoftheweb.com/tables-layout-absurd/. Version: Mai 2017. – Letzter Zugriff: 03 Mai 2026

[Hoober 2013] Hoober, Steven: How Do Users Really Hold Mobile Devices? In: *UXmatters* (2013), Februar. https://www.uxmatters.com/mt/archives/2013/02/how-do-users-really-hold-mobile-devices.php. – Letzter Zugriff: 27. Juli 2026

[House 2021] House, Chris: *A Complete CSS Grid Layout Guide*. https://css-tricks.com/snippets/css/complete-guide-grid/, 2021. – Letzter Zugriff: 19 Juni 2026

[IONOS Redaktion 2024] IONOS Redaktion: *Was ist Mobile First?* https://www.ionos.de/digitalguide/websites/webdesign/mobile-first-neuer-ansatz-im-webdesign/. Version: 2024. – Letzter Zugriff: 27 Juli 2026

[Kayce Basques 2024] Kayce Basques, Sofia E.: *Simulate mobile devices with device mode*. https://developer.chrome.com/docs/devtools/device-mode?hl=de. Version: 2024. – Letzter Zugriff: 27. Juli 2026

[Kholmatova 2017] Kholmatova, Alla: *Design Systems: A practical guide to creating design languages for digital products*. Freiburg : Smashing Magazine, 2017

[Marcotte 2010] Marcotte, Ethan: *Responsive Web Design*. https://alistapart.com/article/responsive-web-design/. Version: 2010. – Letzter Zugriff: 19 Mai 2026

[MDN Web Docs 2025a] MDN Web Docs: *CSS flexible box layout*. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Flexible_box_layout. Version: 2025. – Letzter Zugriff: 19 Mai 2026

[MDN Web Docs 2025b] MDN Web Docs: *Responsive Web Design*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design. Version: 2025. – Letzter Zugriff: 19 Mai 2026

[MDN Web Docs 2026a] MDN Web Docs: *CSS Grid Layout*. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout. Version: 2026. – Letzter Zugriff: 19 Mai 2026

[MDN Web Docs 2026b] MDN Web Docs: *Using container size and style queries*. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_size_and_style_queries. Version: 2026. – Letzter Zugriff: 19 Mai 2026

[Mohamed u. a. 2014] Mohamed, Abdulrehman A. ; Cheruiyot, WK ; Rimiru, Richard ; Ondago, C: Responsive web design inFluid grid concept literature survey. In: *The International Journal Of Engineering And Science (IJES)* 3 (2014), Nr. 7

[Mozilla Developer Network (MDN) 2025a] Mozilla Developer Network (MDN): *Grid layout and accessibility*. `https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout/Accessibility`, 2025. – Letzter Zugriff: Mai 2026

[Mozilla Developer Network (MDN) 2025b] Mozilla Developer Network (MDN): *Subgrid*. `https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Subgrid`, 2025. – Letzter Zugriff: 19 Juni 2026

[Mozilla Developer Network (MDN) 2026a] Mozilla Developer Network (MDN): *CSS Container Queries*. `https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_container_queries`, 2026. – Letzter Zugriff: 21 Juni 2026

[Mozilla Developer Network (MDN) 2026b] Mozilla Developer Network (MDN): *CSS Layout Performance and Browser Rendering*. `https://developer.mozilla.org/en-US/docs/Web/Performance/Fundamentals`, 2026. – Letzter Zugriff: Juni 2026

[Mozilla Developer Network (MDN) 2026c] Mozilla Developer Network (MDN): *Using media queries*. `https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries`, 2026. – Letzter Zugriff: Juni 2026

[Mozilla Foundation & Mozilla Corporation 2026] Mozilla Foundation & Mozilla Corporation: *Examine CSS Grid layouts – Subgrid inspector*. `https://firefox-source-docs.mozilla.org/devtools-user/page_inspector/how_to/examine_grid_layouts/index.html`. Version: 2026. – Letzter Zugriff: 27. Juli 2026

[Rupasinghe 2026] Rupasinghe, Sanjeewa: *CSS Grid Mastery: Real Layouts Without Hacks (2027)*. `https://sanjewa.com/blogs/css-grid-mastery-real-layouts-without-hacks/`. Version: 2026. – Letzter Zugriff: 27 Juli 2026

[Rupp u. die SOPHISTen 2020] Rupp, Chris ; SOPHISTen die: *Requirements-Engineering und -Management: Aus der Praxis von Projekten zum Erfolg*. 7. München : Hanser, 2020. – ISBN 978–3–446–46432–2

[Schoner 2024] Schoner, Stefan: *Container Queries vs. Media Queries: Die Evolution des responsiven Webdesigns*. `https://www.webwilli.at/blog/eintrag/container-queries-vs-media-queries-die-evolution-des-responsiven-webdesigns`. Version: 2024. – Letzter Zugriff: 27 Juli 2026

[Seyfferth 2026] Seyfferth, Markus: *Ist Responsive Design 2026 noch Media Queries?* `https://www.drweb.de/responsive-design/`. Version: 2026. – Letzter Zugriff: 09 Mai 2026

[Siegel 1996] Siegel, David: *Creating killer websites*. Hayden Books Indianapolis, 1996

[Team 2026] Team, Digital A.: *CSS Grid and Flexbox: Complete Layout Reference 2026*. https://www.digitalapplied.com/blog/css-grid-flexbox-complete-layout-reference. Version: 2026. – Letzter Zugriff: 27 Juli 2026

[Think11 2020] Think11, Redaktion: *Why You Should Avoid Tables for Web Page Layouts*. https://www.thoughtco.com/dont-usetables-for-layout-3468941. Version: 2020. – Letzter Zugriff: 06 Juni 2026

[Think11 2026] Think11, Redaktion: *Responsive Design – Websites für alle Geräte*. https://think11.de/glossar/responsive-design/. Version: 2026. – Letzter Zugriff: 24 Mai 2026

[UnravelJS 2026] UnravelJS: *CSS Grid vs Flexbox - The Ultimate Guide to Choosing the Right Layout Tool*. https://www.unraveljs.com/tips/2026-02-14-css-grid-vs-flexbox-when-to-use-what. Version: 2026. – Letzter Zugriff: 27 Juli 2026

[User Experience Qualifications Board (UXQB) 2023] User Experience Qualifications Board (UXQB): *Lehrplan CPUX-UR – Zertifizierter Professional für Usability und User Experience – User Requirements Engineering*. https://uxqb.org/. Version: 2023. – Offizieller Lehrplan zur Spezifikation von Nutzungsanforderungen

[vscode 2026] *Visual Studio Code*. 2026. – https://code.visualstudio.com/

[W3C 2024] W3C: *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/. Version: 2024. – Letzter Zugriff: 06 Juni 2026

[web.dev 2025] web.dev: *Container queries*. https://web.dev/learn/css/container-queries?hl=en. Version: 2025. – Letzter Zugriff: 21 Juni 2026

[Wilson 2023] Wilson, Kevin: The Absolute Beginner's Guide to HTML and CSS. In: *The Absolute Beginner's Guide to HTML and CSS* (2023)

[World Wide Web Consortium (W3C) 2015] World Wide Web Consortium (W3C): *CSS Grid Layout Module Level 1 – Section 10.1: Reordering and Accessibility*. https://www.w3.org/TR/css-grid-1/#order-accessibility, 2015. – W3C Recommendation, Letzter Zugriff: 19 Juni 2026

[World Wide Web Consortium (W3C) 2022a] World Wide Web Consortium (W3C): *CSS Cascading and Inheritance Module Level 4*. https://www.w3.org/TR/css-cascade-4/. Version: 2022. – Letzter Zugriff: 27 Juli 2026

[World Wide Web Consortium (W3C) 2022b] World Wide Web Consortium (W3C): *CSS Containment Module Level 3*. https://www.w3.org/TR/css-contain-3/, 2022. – W3C Candidate Recommendation, Letzter Zugriff: 21 Juni 2026

[World Wide Web Consortium (W3C) 2025a] World Wide Web Consortium (W3C): *CSS Flexible Box Layout Module Level 1*. https://www.w3.org/TR/css-flexbox-1/. Version: 2025. – Letzter Zugriff: 17 Juni 2026

[World Wide Web Consortium (W3C) 2025b] World Wide Web Consortium (W3C): *CSS Grid Layout Module Level 1*. `https://www.w3.org/TR/css-grid-1/`, 2025. – W3C Recommendation, Letzter Zugriff: 19 Juni 2026

[World Wide Web Consortium (W3C) 2025c] World Wide Web Consortium (W3C): *CSS Grid Layout Module Level 2*. `https://www.w3.org/TR/css-grid-2/`, 2025. – W3C Candidate Recommendation, Letzter Zugriff: 21 Juni 2026

[World Wide Web Consortium (W3C) 2026] World Wide Web Consortium (W3C): *Media Queries Level 4*. `https://www.w3.org/TR/mediaqueries-4/`, 2026. – W3C Recommendation, Letzter Zugriff: Juni 2026

Anhang

A. Anhang 6 Herleitung der Layoutanforderungen im responsive Webdesign

Layoutanforderungen			
Überbegriff der Anforderung	**Nutzungskontext**	**Erfordernis**	**Anforderung**
Fluid Layout	Endanwender greifen in hochgradig dynamischen Situationen auf Webinhalte zu, wechseln zwischen Smartphones, Tablets und Desktop-Monitoren oder nutzen moderne Anzeigeformen wie den Split-Screen-Modus.	Als mobiler Endanwender muss man Webinhalte stufenlos an die Displayränder angepasst und ohne horizontales Scrollen verfügbar haben, um Informationen auf Bildschirmen jeder Dimension barrierefrei lesen zu können.	Die Webkomponente auf der Webseite **muss** dem Endanwender die Möglichkeit bieten, die Breite von Inhaltselementen proportional und stufenlos entlang relativer Einheiten zu skalieren.
Navigation	Anwender bedienen Webseiten auf mobilen Geräten häufig unter Ablenkung (z. B. unterwegs in öffentlichen Verkehrsmitteln) und einhändig ausschließlich mit dem Daumen auf einem Touchscreen.	Als mobiler Endanwender muss man Steuerungselemente in einer ergonomisch erreichbaren Größe und Anordnung verfügbar haben, um Links zielsicher und ohne Fehlklicks mit dem Finger aktivieren zu können.	Das Navigations-Modul **muss** dem Endanwender die Möglichkeit bieten, Steuerungselemente bei unzureichendem horizontalem Platzangebot elastisch in die Vertikale umzubrechen.
Flexible Größenanpassung	Endanwender konsumieren auf Webseiten eine Vielzahl an visuellen Medien wie Bilder oder Videos, welche auf Geräten mit stark differenzierenden Bildschirmbreiten und Pixeldichten fehlerfrei dargestellt werden müssen.	Als mobiler Endanwender muss man eingebettete Medien und Container stufenlos an den jeweils verfügbaren Platz angepasst vorfinden, um visuelle Inhalte ohne Beschneidung, Verzerrung des Seitenverhältnisses oder horizontalen Layout-Overflow erfassen zu können.	Die Webkomponente auf der Webseite muss fähig sein, die Dimensionen von Containern und eingebetteten Medien proportional zu skalieren, ohne das inhärente Aspektverhältnis zu verzerren.
Wiederverwendbarkeit von Komponenten	Webapplikationen werden nach dem Komponenten-Paradigma aufgebaut, wodurch identische UI-Module (z. B. Teaser-Karten) flexibel an unterschiedlichen Positionen der Applikation, wie einer schmalen Sidebar oder dem breiten Hauptbereich, eingesetzt werden.	Als Endanwender muss man UI-Komponenten unabhängig von ihrer Platzierung auf der Webseite in einem visuell fehlerfreien und an den verfügbaren Platz angepassten Layout dargeboten bekommen, um Informationen kontextübergreifend konsistent erfassen zu können.	Das Layout-System der Webseite **muss** fähig sein, die visuelle Darstellung einer Webkomponente ausschließlich auf Basis des lokal verfügbaren Platzes ihres direkt umschließenden Eltern-Elements zu steuern.
Dynamische Inhaltsmengen	Das Interface wird mit dynamischen Daten (z. B. über APIs, Datenbanken oder unterschiedliche Sprachübersetzungen) befüllt, deren exakte Textlängen und Zeichenanzahlen dem Layout zur Designzeit unbekannt sind.	Als Endanwender muss man visuell zusammengehörige Informationen (z. B. Produktmerkmale oder Funktionsfasten) auf einer einheitlichen horizontalen Fluchtlinie wahrnehmen können, um Inhalte strukturiert vergleichen zu können.	Die Webkomponente auf der Webseite **muss** fähig sein, die Höhen verschachtelter Kindelemente über Komponentengrenzen hinweg synchron an den längsten Inhalt anzupassen.
Layout-Unabhängigkeit von der HTML-Struktur	Eine Webseite wird simultan von sehenden Nutzern auf mobilen Geräten, die eine daumenfreundliche Anordnung der wichtigsten Elemente erwarten und von sehbehinderten Nutzern via Screenreader konsumiert.	Als Screenreader-Nutzer muss man die logische und semantische Reihenfolge des Quelltextes unverändert linear vorfinden, während man als mobiler Nutzer eine visuell optimierte Anordnung benötigt, um die Anwendung barrierefrei und ergonomisch bedienen zu können.	Die Webkomponenteder Webseite **muss** dem Entwickler die Möglichkeit bieten, die visuelle Anordnung von UI-Elementen auf dem Bildschirm vollständig von ihrer semantischen Reihenfolge im HTML-Quelltext zu entkoppeln.
Skalierbarkeit und Wartbarkeit	Entwickler-Teams müssen im Rahmen des Software-Lebenszyklus komplexe, tief verschachtelte Layoutstrukturen nach ISO/IEC 25010 anpassen, erweitern oder refaktorieren.	Als Entwickler muss man eine schlanke, semantische und redundanzfreie Quelltext-Struktur verfügbar haben, um das CSS-Layout ohne unvorhergesehene Seiteneffekte und ohne das manuelle Hinzufügen rein visueller HTML-Hilfs-Wrapper erweitern zu können.	Die Webkomponente der Webseite **muss** fähig sein, die Layout-Rastersysteme von tief verschachtelten Kindelementen direkt an die Achsenstrukturen des übergeordneten Eltern-Rasters zu binden.

Abbildung A1.: ,,Tabelle zur Herleitung der Layoutanforderungen”

B. Anhang 7 Auswertung und Einordnung der Ergebnisse

B.1. Entscheidungsmodell Ergebnisse 7.1

Entscheidungsmatrix: Fluid Layout			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Fluid Layout (Skalierung entlang relativer Einheiten ohne Overflow)	CSS Grid	++	Erlaubt über die Bruchteil-Einheit (fr) und Funktionen wie minmax() eine mathematisch exakte, stufenlose Aufteilung des Raums auf Mikro- und Makro-Ebene, ohne dass feste Breiten definiert werden müssen.
	Subgrid	o	Bietet selbst keine globalen flüssigen Einheiten, sondern erbt die Rasterlinien des Elternelements. Es ist hochgradig flüssig, wenn das Eltern-Grid flüssig ist, dient primär jedoch der internen Ausrichtung verschachtelter Elemente, nicht dem globalen Viewport-Atmen.
	Flexbox	+	Unterstützt flüssiges Verhalten über flex-grow und flex-shrink. Ist jedoch eindimensional und stößt an Grenzen, wenn starre zweidimensionale Rasterstrukturen flüssig gehalten werden müssen.
	Media Queries	+	Bietet selbst keine flüssigen Einheiten, ist aber als makroskopische Steuereinheit (Breakpoints) essenziell, um das flüssige Layout an extremen Bildschirmgrenzen (z. B. Wechsel von Mobile zu Desktop) neu zu strukturieren.
	Container Queries	o	Löst das Problem der Wiederverwendbarkeit von Komponenten, ist aber für das globale, seitenweite flüssige Atmen des Layouts primär nicht zuständig.
	KOMBINATION: CSS-Grid + Media-Queries	+++	**Während CSS Grid für das stufenlose, proportionale „Atmen" der Inhalte zwischen den Breakpoints sorgt, steuern die Media Queries den makroskopischen Strukturwechsel (z. B. den Sprung von 1-spaltig auf 3-spaltig) an kritischen Viewport-Grenzen. Erst die Synthese beider Technologien löst die Anforderung vollumfänglich.**

Abbildung B2.: ,,Entscheidungsmatrix: Fluid Layout"

Entscheidungsmatrix: Navigation			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Navigation (Elastisches, vertikales Umbrechen bei Platzmangel)	Flexbox	++	Ideal für eindimensionale Komponentenketten. Über flex-wrap: wrap brechen Menüpunkte automatisch und fließend in die nächste Zeile um, wenn der horizontale Platz nicht mehr ausreicht.
	Grid	+	Kann Menüs über implizite Raster steuern, ist jedoch starrer als Flexbox, da Elemente standardmäßig in ein fixes Zwei-Wege-Raster gezwungen werden, statt organisch zu fließen.
	Subgrid	o	Spielt für den Umbruch und das dynamische Fließen von Menüketten keine Rolle, da es Layoutstrukturen starr vererbt, statt sie flexibel umzubrechen.
	Media Queries	++	Essenziell, um ab einem exakten Schwellenwert (Breakpoint) das komplette Navigationsmuster radikal zu verändern (z. B. Umschalten von einer horizontalen Desktop-Leiste zu einem mobilen Hamburger-Dropdown).
	Container Queries	+	Erlauben das Umschalten des Menü-Styles basierend auf der Breite des Header-Containers. Oft reicht hier jedoch die Betrachtung des globalen Viewports aus.
	KOMBINATION: Flexbox + Media-Queries	+++	**Flexbox sorgt innerhalb der Navigationsebenen für ein fehlerfreies, elastisches Mitatmen und zeilenweises Umbrechen der Menüglieder. Media Queries steuern den makroskopischen Systemwechsel, um bei kritischem Platzmangel das Desktop-Menü vollständig auszublenden und die mobile Navigation (z. B. ein Off-Canvas-Menü) zu aktivieren.**

Abbildung B3.: ,,Entscheidungsmatrix: Navigation"

Entscheidungsmatrix: Flexible Größenanpassung			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Flexible Größenanpassung (Skalierung von Medien/Containern ohne Verzerrung)	Flexbox	+	Verhält sich analog zu CSS Grid: Erlaubt über flex-shrink das Verkleinern von Containern, benötigt für Medienobjekte aber flankierende Größen- und Verhältnisregeln.
	Grid	+	Kann Containern über relative Einheiten flexible Breiten zuweisen, verhindert aber ohne zusätzliche CSS-Eigenschaften (wie max-width oder aspect-ratio) nicht das Ausbrechen oder Verzerren von Medien.
	Subgrid	o	Vererbt lediglich Rasterspalten und hat keinen direkten Einfluss auf die elastische Größenkontrolle oder die Aspektverhältnisse eingebetteter Medien.
	Media Queries	+	Können genutzt werden, um Bildgrößen oder Seitenverhältnisse an harten Breakpoints stufenweise anzupassen, lösen das Problem des stufenlosen, fließenden Atmens von Medien jedoch nicht.
	Container Queries	++	Ermöglichen es, Medien und Container hochgradig präzise an den Platz anzupassen, den ihr direktes Eltern-Element ihnen bietet. Kombiniert mit modernen CSS-Properties (z. B. aspect-ratio: 16/9) die modernste Art der Skalierung.
	KOMBINATION: Container Queries + Flexbox	+++	**Container Queries erlauben es, die Container-Dimensionen dynamisch zu überwachen. In Kombination mit Flexbox wird sichergestellt, dass Medien stufenlos skalieren und ihr proportionales Seitenverhältnis wahren, ohne jemals aus ihrem umschließenden Element auszubrechen.**

Abbildung B4.: „Entscheidungsmatrix: Flexible Größenanpassung"

Entscheidungsmatrix: Wiederverwendbarkeit von Komponenten			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Wiederverwendbarkeit von Komponenten (Komponenten-Layout basierend auf lokalem Platz)	Flexbox	+	Erlaubt durch den flexiblen Fluss ein gutes Einfügen in unterschiedliche Breiten, kann aber komplexe Layout-Strukturwechsel innerhalb der Komponente nicht isoliert steuern.
	Grid	+	Unterstützt die Wiederverwendbarkeit implizit, wenn mit auto-fit und minmax() gearbeitet wird, bietet jedoch keine echten Style- oder Layout-Wechsel basierend auf dem Eltern-Element.
	Subgrid	+	Hilft dabei, dass wiederverwendbare Komponenten sich perfekt in das Rastersystem ihres jeweiligen Einsatzortes einfügen, steuert deren internen Strukturwechsel aber nicht selbst.
	Media Queries	o	Schlagen für die modulare Wiederverwendbarkeit fehl. Da sie nur den globalen Viewport (@media) abfragen, zerstört eine in die schmale Sidebar gesetzte Komponente das Layout, wenn sich der Viewport auf Desktop-Größe befindet.
	Container Queries	++	Die einzige native Technologie, die Komponenten vollkommen autark macht. Das Layout der Komponente (z. B. Teaser-Karte) reagiert ausschließlich auf die Breite des umschließenden Elements (@container), wodurch sie überall (Sidebar und Hauptbereich) fehlerfrei funktioniert.
	KOMBINATION: Container Queries + CSS Grid/Flexbox	+++	**Container Queries bilden das logische Fundament, indem sie der Komponente die Information über ihren lokal verfügbaren Platz bereitstellen. Innerhalb der Komponente werden dann CSS Grid oder Flexbox eingesetzt, um das eigentliche Interface-Layout (z. B. Wechsel von horizontaler zu vertikaler Ausrichtung) passgenau umzustrukturieren.**

Abbildung B5.: ,,Entscheidungsmatrix: Wiederverwendbarkeit von Komponenten”

Entscheidungsmatrix: Dynamische Inhaltsmengen			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Dynamische Inhaltsmengen (Horizontale Fluchtlinien bei ungleichen Textlängen)	Flexbox	+	Kann über align-items: stretch die äußeren Container einer Reihe auf die gleiche Höhe zwingen. Eine zeilenübergreifende synchrone Ausrichtung verschachtelter Kindelemente ist damit jedoch nicht möglich.
	Grid	+	Garantiert standardmäßig nur, dass die Hauptelemente (die Karten selbst) in einer Reihe exakt gleich hoch sind. Sobald die Elemente innere Verschachtelungen aufweisen, stößt das normale Grid an seine Grenzen.
	Subgrid	++	Die technologische Ideallösung. Erlaubt es tief verschachtelten Kindelementen (z. B. den Card-Überschriften in einer Grid-Reihe), sich an denselben Rasterlinien auszurichten. Wächst ein Text dynamisch an, wachsen alle anderen Karten-Header in dieser Reihe
	Media Queries	o	Haben keinen Einfluss auf dynamische Dateninhalte oder die vertikale Synchronisation von Elementhöhen.
	Container Queries	o	Können zwar auf Platzänderungen reagieren, bieten aber keine Möglichkeit, die Höhen unvollständiger, nebeneinanderstehender Boxen datenabhängig zu synchronisieren.
	KOMBINATION: CSS Grid + CSS Subgrid	+++	**Während das übergeordnete CSS Grid die globale, mehrspaltige Verteilung der Inhaltskarten übernimmt, sorgt das aktivierte grid-template-rows: subgrid auf den Kindelementen dafür, dass alle inneren Segmente (Titel, Beschreibung, Buttons) trotz variierender Textlängen auf exakt denselben horizontalen Fluchtlinien liegen.**

Abbildung B6.: ,,Entscheidungsmatrix: dynamische Inhaltsmengen”

Entscheidungsmatrix: Layout-Unabhängigkeit von der HTML-Struktur			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Layout-Unabhängigkeit von der HTML-Struktur (Entkopplung der visuellen Anordnung vom HTML-Code)	Flexbox	+	Erlaubt über die Eigenschaft order oder flex-direction: column-reverse das Umsortieren von Elementen, ist jedoch auf eine Dimension (entlang einer Achse) beschränkt und bei komplexen Layoutsprüngen unzureichend.
	Grid	++	Bietet maximale Unabhängigkeit. Über grid-template-areas oder die explizite Zuweisung von Start- und Endlinien (grid-column, grid-row) können Elemente visuell völlig frei im zweidimensionalen Raum platziert und verschoben werden, ohne das HTML zu berühren.
	Subgrid	o	Ist an das Rastersystem des Elternelements gebunden. Es dient der Ausrichtung und nicht dem großflächigen, strukturellen Umsortieren von Elementen entgegen dem Quelltext.
	Media Queries	++	*Unverzichtbar, um den visuellen Strukturwechsel zu triggern. Sie definieren, wann (z. B. ab Mobile-Viewport) das CSS Grid die visuellen Bereiche neu anordnen soll.*
	Container Queries	+	Können Umsortierungen innerhalb einer Komponente triggern, steuern aber nicht das seitenweite, makroskopische Umsortieren von Hauptlayouts.
	KOMBINATION: CSS Grid + Media Queries	+++	**Die logische und barrierefreie HTML-Struktur bleibt für Screenreader unangetastet im Quellcode verankert. CSS Grid übernimmt die visuelle Neuanordnung im zweidimensionalen Raum, während die Media Queries als Steuerinstanz dienen, um diese Layout-Verschiebungen exakt auf Basis des Endgeräts auszuführen.**

Abbildung B7.: ,,Entscheidungsmatrix: Layout-Unabhängigkeit von der HTML-Struktur”

Entscheidungsmatrix: Skalierbarkeit und Wartbarkeit			
Layoutanforderung	**CSS Layout-Technologie**	**Score**	**Technologische Begründung (Warum passt es?)**
Skalierbarkeit und Wartbarkeit (Schlanker, redundanzfreier Code ohne Hilfs-Wrapper)	Flexbox	+	Führt bei komplexen, verschachtelten Rastern oft zu massiven Verschachtelungen (Box-in-Box-Prinzip) im HTML-Code, was die Wartbarkeit und Lesbarkeit verschlechtert.
	Grid	++	Reduziert den CSS-Code drastisch, da komplexe Gestaltungsraster zentral im Eltern-Element definiert werden, statt jedem Kindelement manuelle Breiten, Margins und Paddings mitzugeben.
	Subgrid	++	Eliminiert die Notwendigkeit von rein visuell motivierten HTML-Hilfs-Containern („div-Suppen"). Kindelemente nutzen einfach das Raster des Eltern-Grids, wodurch der HTML-Code semantisch sauber und das CSS schlank bleibt.
	Media Queries	+	Bei exzessiver Nutzung droht ein unübersichtlicher „Code-Spaghetti"-Effekt, da Layout-Logiken über dutzende Breakpoints hinweg verstreut und überschrieben werden müssen.
	Container Queries	+	Erhöhen die Wartbarkeit im hohen Maße, da die gesamte Layout-Logik gekapselt in der Komponente liegt. Änderungen am globalen Seitenlayout haben keine unvorhergesehenen Seiteneffekte auf die Komponente.
	KOMBINATION: CSS Grid + CSS Subgrid	+++	**CSS Grid und Subgrid garantieren eine saubere, flache HTML-Struktur ohne visuelle Hilfs-Wrapper.**

Abbildung B8.: ,,Entscheidungsmatrix: Skalierbarkeit und Wartbarkeit”

C. Anhang 8 Prototypische Umsetzung einer Demonstrationswebseite

C.1. Designprozess 8.2

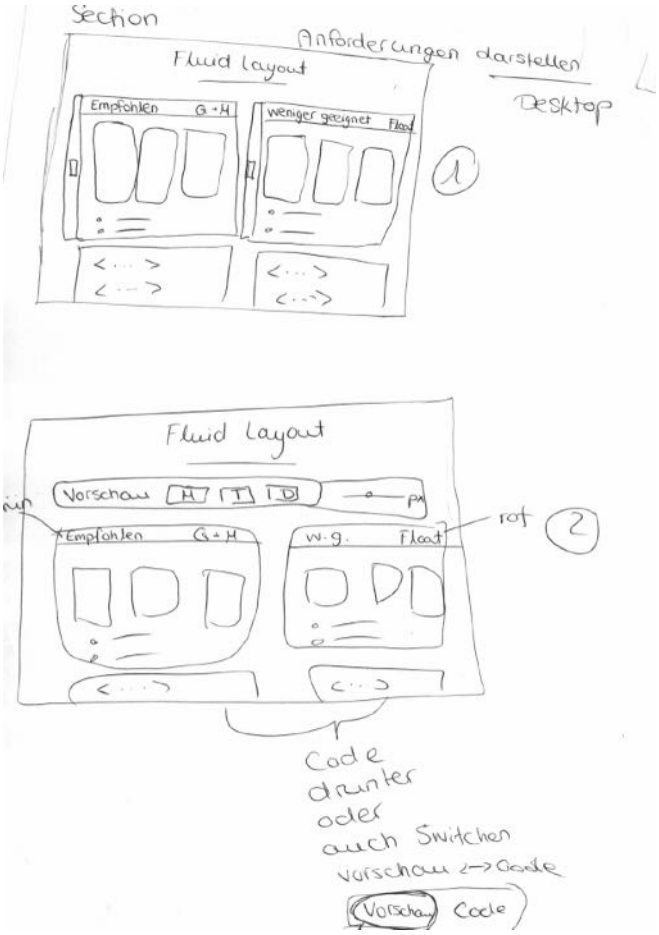


Abbildung C9.: ,,Erste Skizze während des Designprozesses”

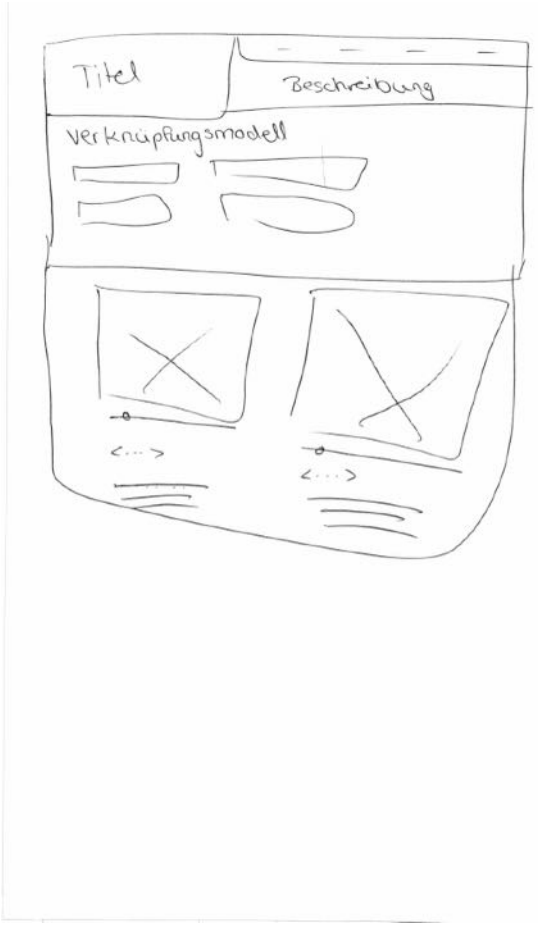


Abbildung C10.: „Zweite Skizze während des Designprozesses"

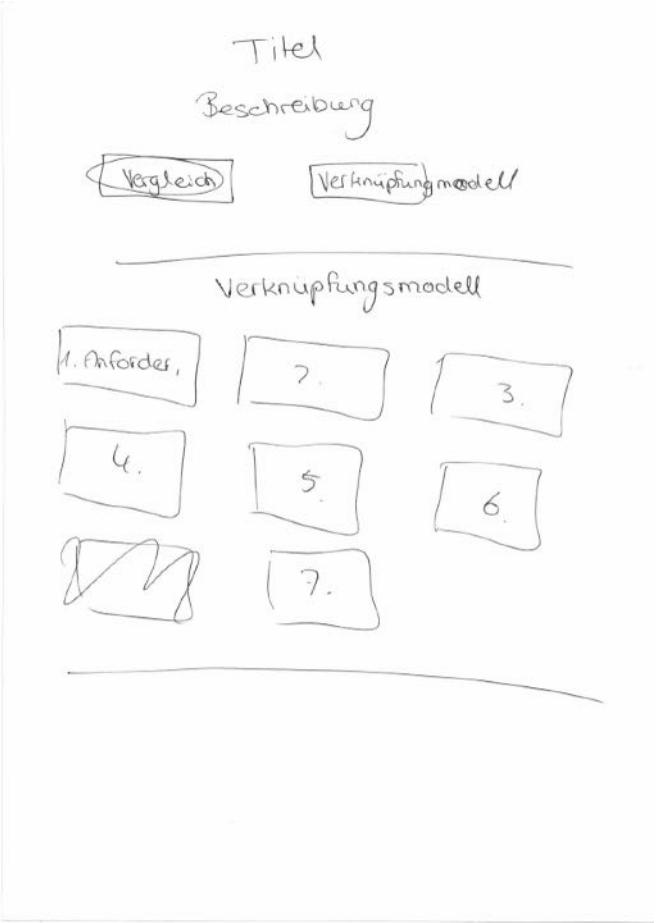


Abbildung C11.: „Dritte Skizze während des Designprozesses"

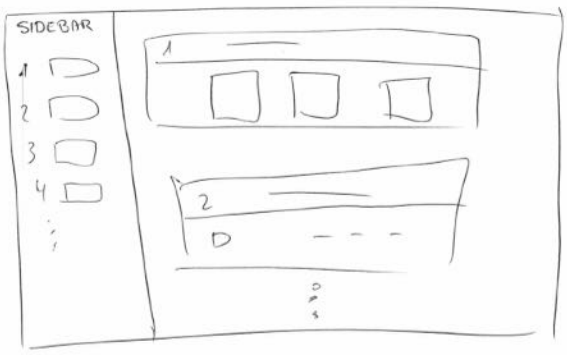


Abbildung C12.: ,,Finale Skizzen während des Designprozesses”

C.2. Figma Umsetzung 8.2.3

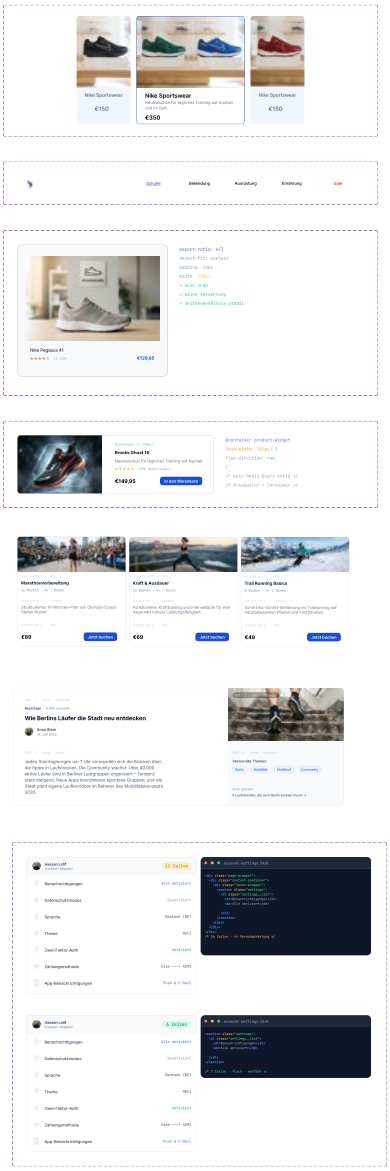


Abbildung C13.: „Design der einzelnen Sections zum Einfügen in das Base-Layout"